



Universidad Peruana de Ciencias Aplicadas

Carrera de Ingeniería de Software

1ACC0238 – Aplicaciones para Dispositivos Móviles

NRC: 13981

Informe del Trabajo Final

Docente: Mayta Guillermo, Jorge Luis

Equipo: TuxLogic

Proyecto: ShiftIq

Sección: 1ACC0238

Integrantes

Código	Apellidos y Nombres
u20241e299	Mamani Vilca, Alan Jaivi
u202419485	Machacca Soto, Aldo Jeanfranco
U202319440	Mallqui Vilca, Dhilsen Armil
u202224130	Ramos Hinoestroza, Diego Antonio
u202316878	Vidal Malaga, Jareth Beycker

Período 202620 – Julio 2026

Registro de Versiones

Versión	Fecha	Autor	Descripción de modificación
1.0	30/08/2026	Diego Antonio Ramos Hinostroza	Estructura inicial del informe para el Primer Hito AV1.
1.1	05/09/2026	Diego Antonio Ramos Hinostroza	Se añadió todo el contenido del Capítulo I

Student Outcome

El curso contribuye al cumplimiento del Student Outcome ABET:

ABET – EAC - Student Outcome 7

Criterio: La capacidad de adquirir y aplicar nuevos conocimientos según sea necesario, utilizando estrategias de aprendizaje apropiadas.

En este primer hito, el equipo TuxLogic aplicó metodologías de análisis, investigación y diseño para comprender el problema del usuario, definir la propuesta de valor y plantear una solución viable con una base sólida para futuras iteraciones.

Criterio específico Acciones realizadas Conclusiones Actualiza conceptos y conocimientos necesarios para su desarrollo profesional y en especial para su proyecto en soluciones de software. Alan Jaivi Mamani Vilca AV1 ... Aldo Jeanfranco Machacca Soto AV1 ... Diego Antonio Ramos Hinostroza AV1 ... Dhilsen Armil Mallqui Vilca AV1 ... Jareth Beycker Vidal Malaga AV1 ... AV1 ... Reconoce la necesidad del aprendizaje permanente para el desempeño profesional y el desarrollo de proyectos en soluciones de software. Alan Jaivi Mamani Vilca AV1 ... Aldo Jeanfranco Machacca Soto AV1 ... Diego Antonio Ramos Hinostroza AV1 ... Dhilsen Armil Mallqui Vilca AV1 ... Jareth Beycker Vidal Malaga AV1 ... AV1 ...

Tabla de Contenidos

Registro de Versiones	2
Student Outcome	2
Objetivos SMART	4
Alan Jaivi Mamani Vilca	4
Aldo Jeanfranco Machacca Soto	4
Diego Antonio Ramos Hinostroza	4

Dhilsen Armil Mallqui Vilca	5
Jareth Beycker Vidal Malaga	5
1.1. Startup Profile	6
1.1.1. Descripción de la Startup	6
1.1.2. Perfiles de integrantes del equipo	6
1.2. Solution Profile	7
1.2.1. Antecedentes y problemática	7
1.2.2. Lean UX Process	12
1.3. Segmentos Objetivo	17
Segmento Objetivo 1: Dueños o administradores de talleres automotrices independientes en Lima	17
Segmento Objetivo 2: Conductores de vehículos de Lima	18
2.1. Competidores	19
2.1.1. Análisis competitivo	19
2.1.2. Estrategias y tácticas frente a competidores	19
2.2. Entrevistas	19
2.2.1. Diseño de entrevistas	19
2.2.2. Registro de entrevistas	25
2.2.3. Análisis de entrevistas	26
2.3. Needfinding	26
2.3.1. User Personas	26
2.3.2. User Task Matrix	30
2.3.3. User Journey Mapping	31
2.3.4. Empathy Mapping	35
2.3.5. Big Picture EventStorming	39
2.3.6. Ubiquitous Language	41
2.4. Requirements Specification	45
2.4.1. User Stories	45
2.4.2. Impact Mapping	45
2.4.3. Product Backlog	45
2.5. Strategic-Level Domain-Driven Design	45
2.5.1. EventStorming	45
2.5.2. Context Mapping	68
2.5.3. Software Architecture	71
2.6. Tactical-Level Domain-Driven Design	75
2.6.1. Bounded Context: Shared Kernel	75
2.6.2. Bounded Context: Identity & Access Management (IAM)	81
2.6.3. Bounded Context: Operations (Work Orders & Services)	89
2.6.4. Bounded Context: Inventory (Stock & Products Management)	100
2.6.5. Bounded Context: IoT (Telemetry, Vehicles & OBD-II Devices)	110
2.6.6. Bounded Context: Core (Profiles, Workshops & Branches)	118
2.6.7. Bounded Context: Fleet (Appointments & Registrations)	128
2.6.8. Bounded Context: Billing (Quotes, Vouchers & Payments)	135

Objetivos SMART

Los objetivos SMART son una herramienta fundamental para definir metas claras, alcanzables y verificables dentro de cualquier proyecto. El acrónimo SMART corresponde a Specific (específico), Measurable (medible), Achievable (alcanzable), Relevant (relevante) y Time-bound (con un plazo definido). Este enfoque permite establecer objetivos con un nivel de precisión que facilita su seguimiento, evaluación y alineación con las estrategias del proyecto. Al mantener una estructura clara y medible, los objetivos SMART contribuyen a la planificación efectiva, la toma de decisiones y la ejecución ordenada del trabajo durante el desarrollo del producto.

Alan Jaivi Mamani Vilca

Campo	Descripción
Descripción del objetivo	...
Fecha de inicio	...
Fecha de cumplimiento esperada	...
S - Specific	...
M - Measurable	...
A - Achievable	...
R - Relevant	...
T - Time-bound	...

Aldo Jeanfranco Machacca Soto

Campo	Descripción
Descripción del objetivo	...
Fecha de inicio	...
Fecha de cumplimiento esperada	...
S - Specific	...
M - Measurable	...
A - Achievable	...
R - Relevant	...
T - Time-bound	...

Diego Antonio Ramos Hinostroza

Campo	Descripción
Descripción del objetivo	...
Fecha de inicio	...
Fecha de cumplimiento esperada	...
S - Specific	...
M - Measurable	...
A - Achievable	...
R - Relevant	...
T - Time-bound	...

Dhilsen Arnil Mallqui Vilca

Campo	Descripción
Descripción del objetivo	...
Fecha de inicio	...
Fecha de cumplimiento esperada	...
S - Specific	...
M - Measurable	...
A - Achievable	...
R - Relevant	...
T - Time-bound	...

Jareth Beycker Vidal Malaga

Campo	Descripción
Descripción del objetivo	...
Fecha de inicio	...
Fecha de cumplimiento esperada	...
S - Specific	...
M - Measurable	...
A - Achievable	...
R - Relevant	...
T - Time-bound	...

1.1. Startup Profile

1.1.1. Descripción de la Startup

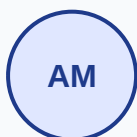
TuxLogic es una iniciativa tecnológica concebida y desarrollada por estudiantes de la carrera de Ingeniería de Software de la Universidad Peruana de Ciencias Aplicadas (UPC). Nuestra propuesta se materializa a través de Shiftlq, un ecosistema digital diseñado para transformar el modelo operativo tradicional de los talleres automotrices en el Perú, impulsando una transición estructural desde un esquema de mantenimiento reactivo hacia un modelo predictivo, preventivo e inteligente.

Para articular esta propuesta de valor, Shiftlq integra una arquitectura multiplataforma compuesta por aplicaciones web y móviles sincronizadas con dispositivos de diagnóstico a bordo. A través de esta integración de hardware, el sistema recolecta telemetría y códigos de diagnóstico de fallas (DTC) en tiempo real, los cuales son procesados de manera segura para anticipar incidencias mecánicas, optimizar los cronogramas de intervención técnica y traducir métricas complejas en información comprensible tanto para el personal técnico como para el conductor.

Misión: La misión de TuxLogic como empresa es innovar en sector del mantenimiento y servicio automotriz en el país mediante una plataforma tecnológica integral que facilite la transición hacia un mantenimiento preventivo y predictivo basado en datos. En TuxLogic buscamos profesionalizar la administración de los talleres mecánicos, optimizar sus procesos operativos y comerciales, y fortalecer la confianza y transparencia en la relación con los conductores.

Visión: En TuxLogic, tenemos la visión de consolidarnos como la plataforma referente en el mercado nacional y regional en soluciones de software para la gestión de talleres mecánicos y el monitoreo vehicular inteligente, promoviendo la rentabilidad de las empresas del sector, la extensión de la vida útil del parque automotor y una mayor seguridad vial.

1.1.2. Perfiles de integrantes del equipo



Mamani Vilca, Alan Jaivi

u20241e299

Estudiante de Ingeniería de Software – Desarrollo & UI/UX

Estudiante de Ingeniería de Software de la Universidad Peruana de Ciencias Aplicadas (UPC). Enfocado en el diseño e implementación de interfaces de usuario intuitivas y el desarrollo frontend para aplicaciones móviles y plataformas web.

Machacca Soto, Aldo Jeanfranco

u202419485

Estudiante de Ingeniería de Software – Backend Developer & AI Architecture



Estudiante de Ingeniería de Software centrado en el diseño, desarrollo y mantenimiento de soluciones tecnológicas eficientes, escalables y de calidad. Especializado en el desarrollo backend y la arquitectura de APIs bajo Domain-Driven Design (DDD) con Java (Spring Boot), Python, C++, JavaScript y TypeScript, junto con React y Next.js. Experiencia en gestión de bases de datos optimizadas, procesos de despliegue e integración de agentes de inteligencia artificial en flujos de trabajo.

Mallqui Vilca, Dhilsen Armil

U202319440

Estudiante de Ingeniería de Software – QA & Análisis de Requerimientos



Estudiante de Ingeniería de Software de la Universidad Peruana de Ciencias Aplicadas (UPC). Con habilidades orientadas al análisis de procesos, aseguramiento de la calidad de software (QA) y diseño funcional de soluciones tecnológicas orientadas al usuario.

Ramos Hinostroza, Diego Antonio

u202224130

Estudiante de Ingeniería de Software – Backend & DevOps



Estudiante de sexto ciclo de Ingeniería de Software en la UPC. Cuenta con dominio en diseño de arquitectura, desarrollo backend y consumo de servicios web. Experiencia en construcción e integración de APIs REST utilizando Java (Spring Boot), C# y Python, modelado de bases de datos relacionales y NoSQL, estructuración de comunicación cliente-servidor en apps móviles, Docker y Git.

Vidal Malaga, Jareth Beycker

u202316878

Estudiante de Ingeniería de Software – Mobile & Frontend Developer



Estudiante de Ingeniería de Software de la Universidad Peruana de Ciencias Aplicadas (UPC). Especializado en la implementación de aplicaciones móviles y soluciones de interacción para usuario en entornos multiplataforma.

1.2. Solution Profile

El perfil de la solución define los fundamentos estratégicos, operativos y técnicos de Shiftlq, ecosistema desarrollado por TuxLogic. Esta sección sintetiza la problemática del mantenimiento vehicular en Lima Metropolitana mediante el análisis de datos estadísticos oficiales, diagramas de causa-raíz y el marco contextual de las 5W's y 2H's, articulando la formulación del producto a través del marco de trabajo Lean UX.

1.2.1. Antecedentes y problemática

En Lima Metropolitana, la gestión del mantenimiento automotriz opera predominantemente bajo un esquema reactivo, en el cual los usuarios acuden a los centros de reparación únicamente cuando la falla mecánica ha inhabilitado el funcionamiento del vehículo. Esta dinámica genera sobrecostos imprevistos para los propietarios y compromete la sostenibilidad operativa de los talleres mecánicos independientes, los cuales sufren de una demanda altamente volátil y bajos niveles de fidelización.

De acuerdo con las cifras de la Asociación Automotriz del Perú (AAP, 2026), durante el primer bimestre del año 2026 se comercializaron 41,584 unidades de vehículos livianos, lo que representó una expansión interanual del 36.8% frente al mismo periodo del año anterior. Este dinamismo sostenido en las matriculaciones acentúa la presión operativa sobre el parque automotor existente, demandando capacidades de servicio técnico que garanticen la vida útil de las unidades en circulación.

Figura 1
Venta de vehículos livianos y pesados en el periodo enero-febrero 2026

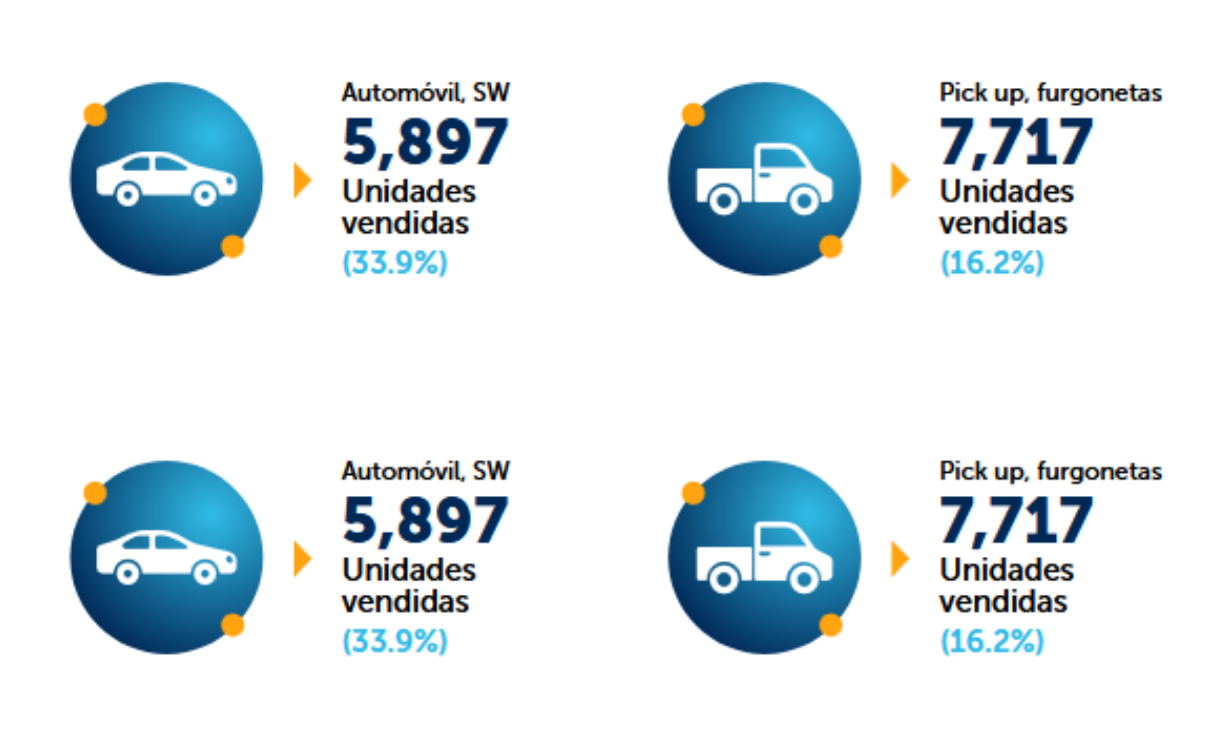
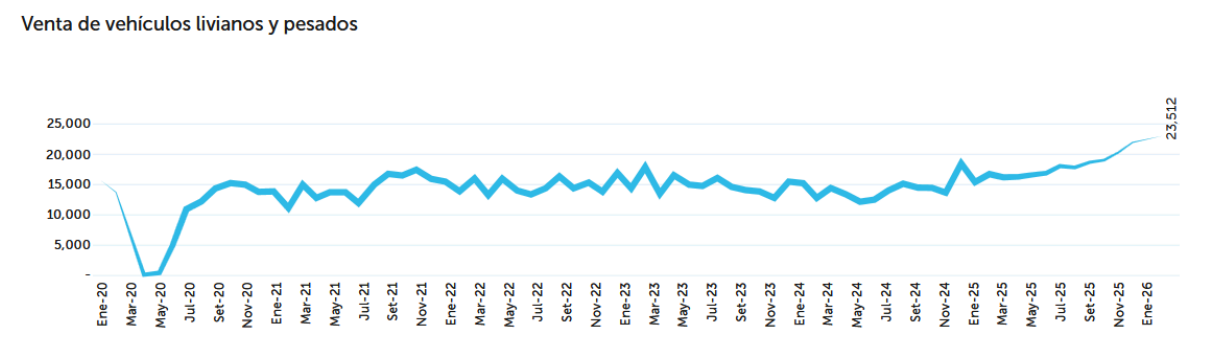
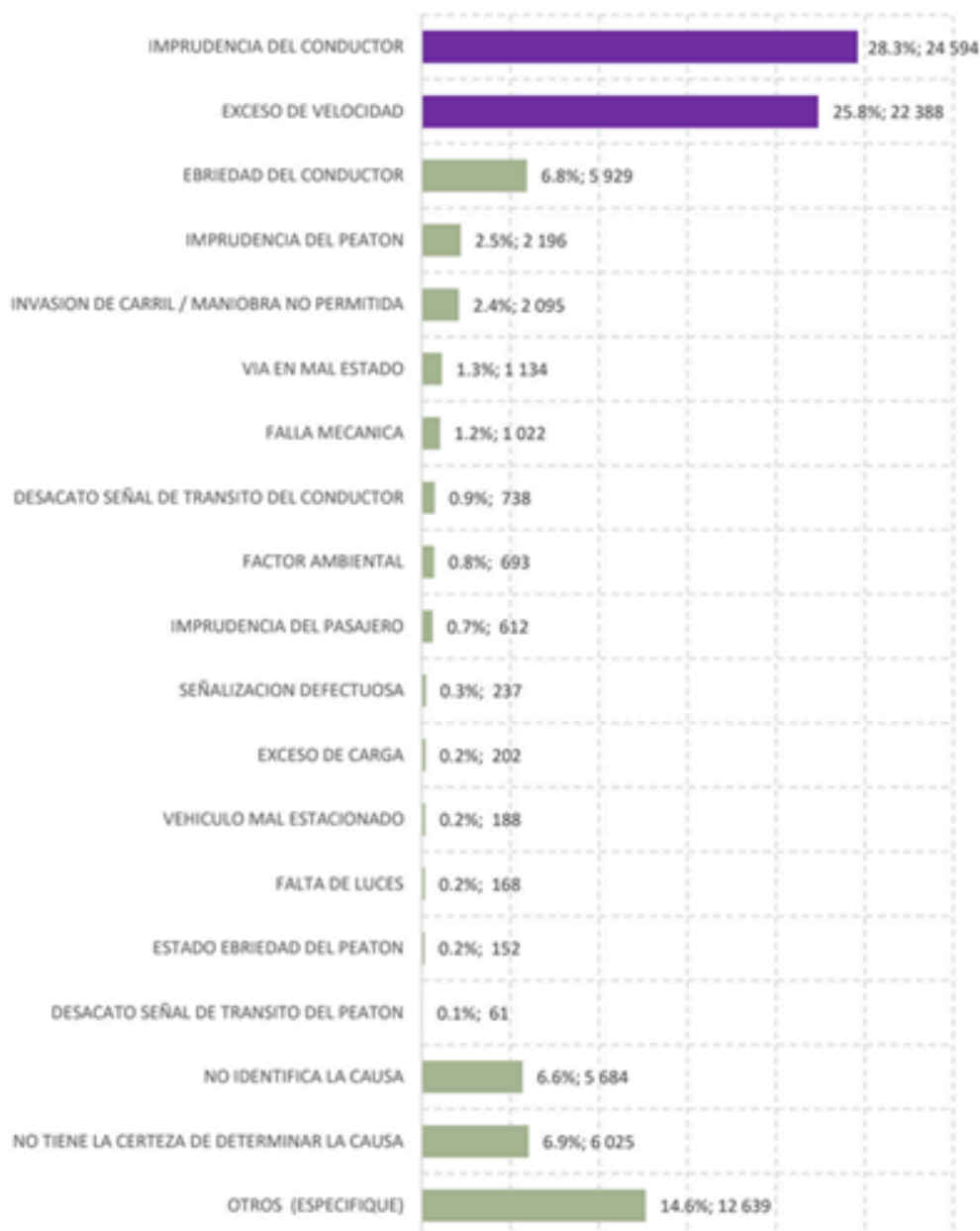


Figura 2
Evolución de ventas de vehículos livianos y pesados (2020 - 2026)



A pesar de este crecimiento en el parque vehicular, el sector de servicios técnicos presenta una marcada ineficiencia estructural. En Lima Metropolitana y el Callao operan aproximadamente 38,000 talleres mecánicos independientes, de los cuales una proporción significativa registra pérdidas operativas de entre el 30% y el 40% debido a la falta de herramientas tecnológicas de gestión, la asimetría de precios y la reducida recurrencia de los clientes (Fiestas et al., 2021). La ausencia de canales de comunicación directos y transparentes entre el taller y el cliente perpetúa un clima de desconfianza que inhibe el mantenimiento regular.

Esta desatención preventiva impacta de manera crítica en la seguridad vial. Según el Boletín Estadístico Anual de Siniestralidad Vial publicado por el Observatorio Nacional de Seguridad Vial (ONSV, 2025), durante el año 2024 se registraron 87,757 siniestros de tránsito a nivel nacional, de los cuales el 1.4% tuvieron como factor causal directo fallas mecánicas e inoperatividad de sistemas de iluminación. Esto equivale a 1,190 siniestros viales que pudieron haberse prevenido mediante diagnósticos mecánicos oportunos y esquemas de mantenimiento preventivo basados en el estado real del vehículo.

Figura 3
Distribución porcentual de siniestros viales según causa raíz (2024)


Con la finalidad de aislar los componentes estructurales de esta problemática, TuxLogic desarrolló dos diagramas de Ishikawa, los cuales delimitan las causas origen del mantenimiento reactivo y la fragmentación en la relación taller-conductor.

Figura 4
Diagrama de Ishikawa: Adopción predominante de un modelo de mantenimiento reactivo en

Lima

Adopción del Mantenimiento Reactivo

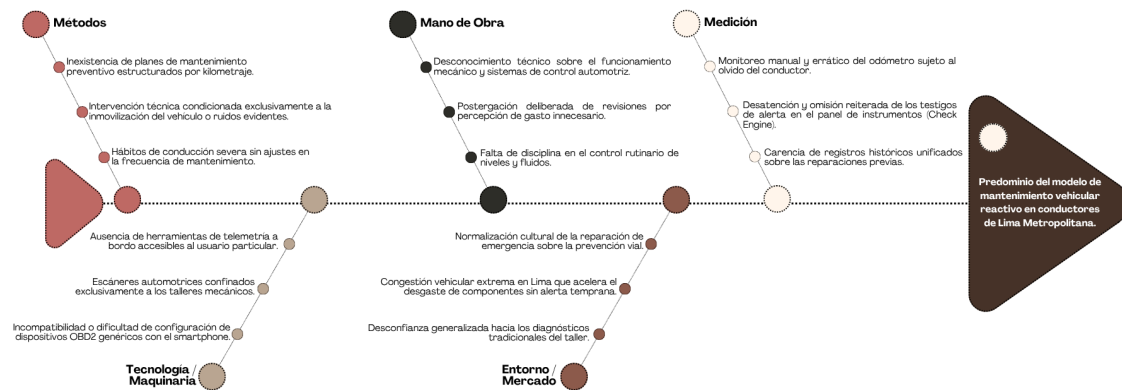
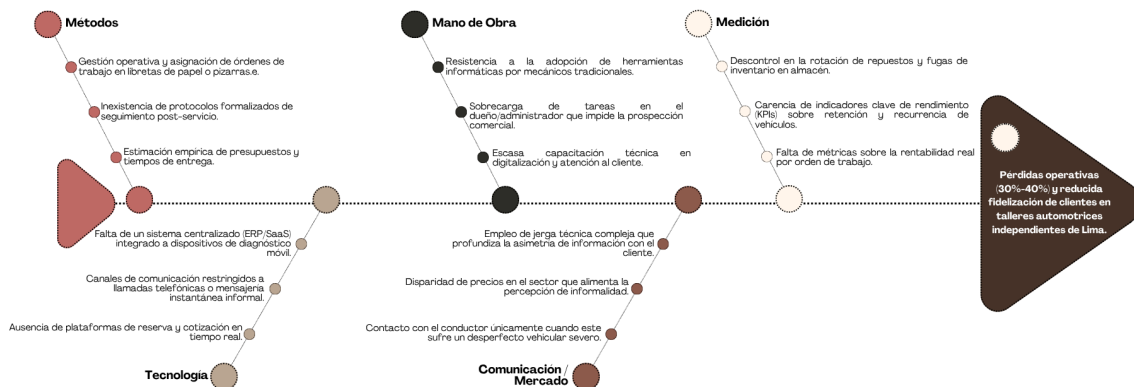


Figura 5

Diagrama de Ishikawa: Baja fidelización y pérdida de recurrencia en talleres automotrices independientes

Baja Fidelización y Pérdidas Operativas en Talleres



Adicionalmente, se delimitó el alcance operativo mediante el análisis de las 5W's y 2H's:

- **Who (Quién):** Afecta a los dueños y administradores de talleres automotrices independientes y a los conductores de vehículos particulares de Lima Metropolitana.
- **What (Qué):** Pérdidas económicas por ineficiencia operativa en talleres y sobrecostos por fallas vehiculares graves derivadas de la falta de diagnóstico temprano.

- **Where (Dónde):** En el parque automotor y los talleres mecánicos independientes de Lima Metropolitana.
- **When (Cuándo):** Ocurre de forma constante durante la circulación vehicular diaria y se agudiza cuando los componentes críticos sufren averías catastróficas por falta de advertencia previa.
- **Why (Por qué):** Debido a la brecha tecnológica en los talleres independientes y la carencia de herramientas móviles que traduzcan la telemetría del vehículo a información comprensible para el conductor.
- **How (Cómo):** Se evidencia en talleres con saturación desordenada, pérdidas operativas del 30% al 40%, y vehículos circulando con alertas mecánicas no atendidas.
- **How Much (Cuánto):** Compromete la rentabilidad de cerca de 38,000 talleres independientes en la región y repercute en más de 1,100 siniestros evitables al año.

Oportunidad Comercial y Restricciones Técnicas

La oportunidad para TuxLogic consiste en articular una solución digital SaaS integrada con hardware de telemetría OBD2 accesible, apalancada en aplicaciones móviles intuitivas que sincronicen la telemetría vehicular en tiempo real con la administración del taller.

Para garantizar la viabilidad técnica y operativa del sistema, se establecen las siguientes restricciones de diseño e ingeniería:

- **Heterogeneidad del hardware vehicular:** El parque automotor limeño presenta una marcada diversidad de marcas y años de fabricación. El dispositivo y los servicios móviles deben basarse en el estándar universal OBD2 (SAE J1979 / ISO 15031) para asegurar la captura uniforme de códigos de diagnóstico de fallas (DTC).
- **Usabilidad móvil en entornos operacionales:** Los administradores y mecánicos operan con altos niveles de fricción física. Las interfaces móviles deben regirse bajo principios de Material Design 3, minimizando la densidad de interacción y optimizando el flujo de trabajo en pantalla.
- **Tolerancia a fallos de conectividad:** Considerando la variabilidad en la cobertura de redes móviles en distintas zonas de la capital, las aplicaciones deben implementar persistencia local reactiva para almacenar lecturas de telemetría y transacciones locales, sincronizándose asíncronamente con los servicios en la nube al restablecerse el enlace de datos.

1.2.2. Lean UX Process

El diseño de la experiencia de usuario y el desarrollo de ShiftIQ se fundamentan en el marco de trabajo Lean UX, estructurado para iterar rápidamente sobre hipótesis validadas y reducir el riesgo de producto en función del valor entregado a los dos segmentos objetivo.

1.2.2.1. Lean UX Problem Statement

Problem Statement 1: Dueños o Administradores de Talleres Automotrices Independientes

El sector de mantenimiento automotriz independiente en Lima busca que los administradores y dueños de talleres gestionen sus operaciones, órdenes de servicio y cronogramas de mantenimiento de manera centralizada, manteniendo una comunicación transparente que asegure la recurrencia de sus clientes.

Sin embargo, estos negocios presentan pérdidas operativas de entre el 30% y el 40% y una reducida tasa de fidelización, debido a que administran sus procesos mediante cuadernos físicos, grupos de mensajería y hojas de cálculo desarticuladas, perdiendo capacidad para anticipar los mantenimientos de su base instalada de vehículos.

¿Cómo podríamos optimizar la gestión operativa y la capacidad proactiva de captación de los administradores de talleres automotrices independientes en Lima, de modo que incrementen su rentabilidad y fidelicen a sus clientes basándose en el estado técnico real de los vehículos?

Problem Statement 2: Conductores de Vehículos en Lima

Los conductores particulares en Lima tienen como objetivo mantener sus vehículos en condiciones óptimas de seguridad y funcionamiento al menor costo posible, requiriendo información oportuna para prevenir fallas que detengan imprevistamente sus traslados.

Sin embargo, los conductores posponen el mantenimiento debido a la desconfianza hacia los diagnósticos tradicionales, la complejidad del lenguaje mecánico y la falta de seguimiento preventivo, lo que desencadena gastos correctivos elevados y contribuye al índice de siniestralidad mecánica en la ciudad.

¿Cómo podríamos facilitar a los conductores de vehículos de Lima el acceso a información diagnóstica clara, comprensible y en tiempo real, de manera que puedan tomar decisiones de mantenimiento preventivo y coordinar servicios técnicos antes de que se produzcan averías severas?

1.2.2.2. Lean UX Assumptions

Business Assumptions

- **Clientes Iniciales:** Dueños y administradores de talleres mecánicos independientes en Lima que atienden un volumen promedio de 10 a 30 vehículos por semana, que no cuentan con herramientas de software especializadas y que demandan optimizar su retorno operativo en menos de 90 días.
- **Estrategia de Adquisición:** Despliegue presencial de programas piloto en talleres de Lima Metropolitana, ofreciendo 30 días de evaluación asistida. Los conductores se integrarán por recomendación y vinculación directa desde el taller afiliado, persiguiendo una meta base de 20 conductores vinculados por taller en el primer mes.
- **Modelo de Monetización:** Esquema de suscripción mensual (SaaS B2B) escalonado para el taller automotriz, complementado por la provisión del dispositivo de diagnóstico OBD2 (venta directa o comodato por suscripción).
- **Entorno Competitivo:** Existen sistemas ERP tradicionales de gestión comercial y escáneres automotrices industriales; sin embargo, no articulan la sincronización continua de telemetría IoT móvil con la recepción directa de alertas en la agenda del taller.

- **Riesgo Crítico de Producto:** Que la fricción en la instalación inicial del conector OBD2 o una interfaz compleja en la aplicación móvil provoquen el abandono de la plataforma durante los primeros 14 días.

Business Outcomes

- Reducción de la pérdida operativa de los talleres afiliados a menos del 18% al término de los primeros 6 meses de uso del ecosistema.
- Lograr que al menos el 45% de los ingresos por servicios del taller provengan de intervenciones preventivas planificadas en lugar de reparaciones reactivas.
- Tasa de conversión de alertas mecánicas OBD2 a órdenes de servicio confirmadas en la app igual o superior al 35%.
- Retención de usuarios conductores en la aplicación móvil a los 60 días superior al 60%.
- Tasa de renovación mensual de suscripciones por parte de los talleres superior al 85% al finalizar el primer trimestre.

User Assumptions

- **Perfil del Usuario - Taller:** Profesionales con amplia pericia en mecánica automotriz pero con bajo nivel de digitalización administrativa, acostumbrados a métodos empíricos y registros físicos.
- **Perfil del Usuario - Conductor:** Propietarios de vehículos particulares sin conocimientos técnicos avanzados, habituados al uso de smartphones para resolver gestiones cotidianas, pero con tendencia a postergar el mantenimiento por falta de tiempo o desconfianza.
- **Punto de Contacto (Touchpoint):**
 - Para el administrador: Una aplicación de supervisión y gestión operativa (móvil y web) consultada al inicio del día para coordinar citas, órdenes y alertas entrantes.
 - Para el conductor: Una aplicación móvil con notificaciones automáticas que solo interrumpe al usuario cuando se detecta un parámetro anómalo o se aproxima un kilometraje de servicio crítico.

User Outcomes

- **Para el Taller:** Control estructurado de citas y órdenes de servicio desde el dispositivo móvil, respaldado por la capacidad de contactar al cliente con el diagnóstico DTC exacto antes de que el vehículo ingrese a la rampa.
- **Para el Conductor:** Comprensión transparente del estado de salud del vehículo en una interfaz gráfica sin tecnicismos opacos, evitando emergencias mecánicas y sobrecostos inesperados.

Features

- **Módulo de Diagnóstico DTC Simplificado (Mobile):** Conversión automática de códigos OBD2 leídos vía Bluetooth a alertas en lenguaje natural con nivel de severidad (Bajo, Medio, Crítico).
- **Agendamiento y Presupuesto In-App:** Capacidad del conductor para agendar su cita preventiva con un solo clic tras recibir una alerta mecánica, visualizando el estimado del servicio.
- **Tablero Móvil de Alertas para Talleres:** Bandeja centralizada de telemetría que prioriza a los clientes cuyos vehículos presentan códigos de falla activos para su contacto preventivo inmediato.

1.2.2.3. Lean UX Hypothesis Statements

- **Hipótesis 1 (Gestión Proactiva del Taller):**

Creemos que lograremos **un incremento del 40% en citas de mantenimiento preventivo** si el **administrador del taller** dispone de un **tablero móvil de monitoreo de telemetría OBD2 que priorice las alertas mecánicas de sus clientes**, permitiéndole contactarlos de manera proactiva. Sabremos que hemos tenido éxito cuando **al menos el 50% de las órdenes de trabajo mensuales se originen a partir de alertas automáticas del sistema** durante los primeros 90 días de operación.

- **Hipótesis 2 (Adopción y Confianza del Conductor):**

Creemos que alcanzaremos **una retención a 60 días superior al 60% en la app móvil** si el **conductor particular** obtiene **notificaciones sobre el estado de su vehículo expresadas en lenguaje cotidiano con estimaciones de urgencia**, reduciendo la asimetría de información técnica. Sabremos que hemos tenido éxito cuando **el 70% de los usuarios activos abra la app dentro de las primeras 48 horas posteriores a la emisión de una notificación de advertencia vehicular**.

- **Hipótesis 3 (Flujo de Agendamiento Sin Fricción):**

Creemos que conseguiremos **una tasa de conversión de alerta a cita confirmada no menor al 35%** si el **conductor particular** cuenta con **un enlace directo de reserva y cotización referencial dentro de la notificación de falla**, evitando llamadas o trámites externos. Sabremos que hemos tenido éxito cuando **el tiempo promedio transcurrido entre la detección de un código DTC y la confirmación de la cita en el taller sea inferior a 72 horas**.

1.2.2.4. Lean UX Canvas

Business Problem

El modelo actual de mantenimiento automotriz en Lima es reactivo, lo que provoca que más de 38,000 talleres independientes experimenten pérdidas operativas de entre el 30% y el 40% debido a la falta de herramientas digitales y baja fidelización de clientes. A la par, la desatención preventiva genera sobrecostos a los conductores y deriva en 1,190 siniestros viales anuales atribuibles a fallas mecánicas evitables. ¿Cómo podríamos facilitar un ecosistema digital y móvil que conecte la telemetría vehicular con la administración del taller para transformar el mantenimiento reactivo en un modelo preventivo, oportuno y confiable?

Solutions

Desarrollar una aplicación móvil nativa para el conductor que se vincule vía Bluetooth con dispositivos OBD2, mostrando el estado de salud vehicular y alertas en lenguaje comprensible. Implementar un motor de interpretación que traduzca códigos de diagnóstico de fallas (DTC) y telemetría a advertencias claras con nivel de criticidad. Construir un módulo de gestión operativa y móvil para talleres que centralice la recepción de telemetría, el control de órdenes de trabajo y la programación de servicios preventivos. Integrar un sistema de notificaciones push proactivas con cálculo de costos estimados y botón de agendamiento directo de citas hacia el taller afiliado.

Business Outcomes

Reducir las pérdidas operativas de los talleres afiliados a menos del 18% al término de 6 meses de operación. Alcanzar una tasa de conversión de alertas OBD2 a citas confirmadas igual o superior al 35% en los primeros 7 días. Lograr una retención de usuarios conductores en la app móvil mayor o igual al 60% a los 60 días de registro. Mantener una tasa de renovación mensual de suscripciones SaaS por parte de los talleres superior al 85% al finalizar el primer trimestre.

Users

Dueños o administradores de talleres automotrices independientes en Lima: Negocios que atienden de 5 a 30 vehículos por semana y buscan modernizar su operación manual. Conductores de vehículos particulares de Lima: Propietarios de vehículos livianos con baja alfabetización mecánica que postergan revisiones por falta de tiempo o desconfianza diagnóstica.

User Outcomes & Benefits

Talleres: Control integral de la agenda y órdenes de servicio desde el móvil, reducción de tiempos muertos y capacidad de contactar al cliente con respaldo de datos técnicos objetivos. Conductores: Monitoreo transparente del estado real del vehículo sin jerga técnica, prevención de averías mecánicas críticas y ahorro económico en correctivos de emergencia. Ambos: Relación comercial transparente y libre de asimetrías de información durante el presupuesto y la ejecución de mantenimientos.

Hypotheses

Creemos que el ecosistema Shiftlq incrementará la adopción del mantenimiento preventivo y optimizará la rentabilidad de los talleres independientes al sincronizar alertas diagnósticas OBD2 con un canal directo de reserva técnica. Sabremos que esto es cierto cuando al menos el 50% de las citas registradas en la plataforma se originen proactivamente por alertas mecánicas y más del 65% de los conductores activos mantenga el dispositivo OBD2 sincronizado durante su segundo mes de uso.

What's the most important thing we need to learn first?

Validar si los conductores están dispuestos a conservar el dispositivo OBD2 conectado permanentemente en sus vehículos a cambio de telemetría en su smartphone. Identificar si los administradores de talleres independientes encuentran viable y rápida la adopción de una interfaz móvil para gestionar citas y órdenes frente a sus libretas físicas. Determinar si una notificación push con costo estimado y nivel de urgencia reduce significativamente la fricción de agendamiento en el conductor.

What's the least amount of work we need to do to learn the next most important thing?

Desplegar una prueba piloto de 30 días con 5 talleres independientes y 30 conductores en Lima utilizando el MVP móvil vinculado a emuladores y dongles OBD2 estándar. Evaluar el comportamiento y tiempo de respuesta de los conductores ante el envío de alertas diagnósticas simuladas y reales. Aplicar entrevistas breves y métricas in-app para medir la facilidad de uso y la tasa de agendamiento efectivo post-alerta.

1.3. Segmentos Objetivo

La propuesta de valor de TuxLogic a través del ecosistema Shiftlq se orienta a un modelo de mercado bilateral (*two-sided market*) dentro del sector automotriz de Lima Metropolitana. Este modelo articula a dos actores interdependientes cuyas necesidades operativas y diagnósticas son atendidas de forma sincronizada mediante plataformas móviles y servicios en la nube:

- **Segmento Oferente (B2B):** Dueños o administradores de talleres automotrices independientes en Lima.
- **Segmento Demandante (B2C):** Conductores de vehículos particulares de Lima Metropolitana.

Segmento Objetivo 1: Dueños o administradores de talleres automotrices independientes en Lima

Definición y Perfil Demográfico

Este segmento comprende a microempresarios, técnicos mecánicos y administradores generales a cargo de talleres automotrices independientes multimarca en Lima Metropolitana y el Callao. El perfil demográfico dominante corresponde a adultos de entre 30 y 58 años, con formación técnica o empírica consolidada en mecánica automotriz, pero con alfabetización formal reducida en herramientas de gestión digital. Administran establecimientos que procesan un flujo promedio de 5 a 30 vehículos semanales y dependen de métodos de registro físicos (cuadernos, notas) o canales de mensajería no estructurados (WhatsApp) para coordinar su operación.

Sustento Estadístico y Problemática del Dominio

La priorización estratégica de este sector radica en su volumen transaccional y cuota de mercado. Investigaciones empíricas demuestran que el **67.2% de los conductores en entornos urbanos prefiere acudir a talleres tradicionales e independientes** frente a los servicios postventa de las concesionarias oficiales, motivados principalmente por factores de cercanía y accesibilidad económica (Bimboza et al., 2023).

A pesar de concentrar la mayor parte de la demanda, este sector experimenta fallas operativas severas. Como señalan Fiestas et al. (2021), los cerca de 38,000 talleres independientes de la capital registran pérdidas operativas recurrentes de entre el 30% y el 40%. Esta ineficiencia se origina en:

- Descontrol de inventarios y presupuestación imprecisa de repuestos.
- Ausencia de seguimiento post-servicio y pérdida de recurrencia de los clientes.
- Asimetría de información que genera sospechas en el conductor respecto al diagnóstico emitido.

Articulación con la Solución Móvil Shiftlq

Shiftlq proporciona a este segmento un canal móvil y web de supervisión técnica en tiempo real. Al vincularse con la telemetría de los vehículos monitoreados, el administrador recibe códigos de diagnóstico de fallas (DTC) antes de que el automóvil ingrese al taller. Esto le

permite fundamentar sus cotizaciones en lecturas electrónicas objetivas, optimizar la asignación de repuestos y programar mantenimientos preventivos proactivos que elevan la tasa de retención del taller.

Fuente de referencia:

Bimboza, J., Cárdenas, L. y Mancheno, M. (2023). Calidad del servicio y satisfacción del cliente. El caso del mantenimiento vehicular liviano. *Religación: Revista de Ciencias Sociales y Humanidades*, 8(35), 2–10. <https://doi.org/10.46652/rng.v8i35.1019>

Segmento Objetivo 2: Conductores de vehículos de Lima

Definición y Perfil Demográfico

Este segmento está constituido por hombres y mujeres propietarios o usuarios habituales de vehículos livianos particulares (sedanes, hatchbacks, SUV) que transitan diariamente en Lima Metropolitana. Se trata de una población económicamente activa concentrada en un **70% entre los 26 y 45 años de edad** (distribuida en un 42% de 26 a 35 años y un 28% de 36 a 45 años) . Presentan un perfil altamente bancarizado y familiarizado con aplicaciones móviles cotidianas de navegación, delivery y banca por internet .

Sustento Estadístico y Hábitos de Mantenimiento

La conducta de este segmento evidencia una clara tendencia hacia el mantenimiento reactivo derivada de barreras temporales, cognitivas y económicas. De acuerdo con los hallazgos de Chávez y Moreno (2023), los motivos por los cuales estos conductores desatienden las revisiones preventivas periódicas se distribuyen de la siguiente forma :

- **32%** posterga el servicio por restricciones de tiempo en su rutina diaria .
- **28%** no realiza el mantenimiento por percibirlo como un sobre costo financiero imprevisto .
- **18%** incurre en desatención por simple olvido del kilometraje de cambio de componentes .
- **15%** posterga la intervención por desconfianza frente a cobros o diagnósticos inflados en los talleres .

Esta desatención tiene consecuencias directas: el Observatorio Nacional de Seguridad Vial (2025) reportó que durante el año 2024 se produjeron 1,190 siniestros de tránsito atribuibles de forma directa a fallas mecánicas e inoperatividad de sistemas del vehículo a nivel nacional.

Factibilidad y Adopción Tecnológica Móvil

La viabilidad técnica y conductual para que el conductor integre Shiftlq a su experiencia de manejo está respaldada por su interacción cotidiana con dispositivos móviles dentro del habitáculo:

- El **68% de los conductores utiliza su smartphone de forma frecuente y activa** durante la conducción como herramienta de asistencia (navegación GPS mediante Waze o Google Maps, monitoreo de tráfico y reproducción multimedia) .
- Un **22% declara un uso complementario u ocasional** de dispositivos móviles mientras se desplaza .

Este patrón conductual confirma que la vinculación pasiva mediante Bluetooth Low Energy entre el dispositivo OBD2 y la aplicación móvil Shiftlq se inserta de manera natural en la rutina

del usuario. La aplicación traduce códigos técnicos complejos a alertas en lenguaje natural con niveles claros de severidad y presupuestos estimados, eliminando la asimetría informativa y permitiendo agendar servicios con el taller afiliado en menos de tres toques.

Fuente de referencia:

Chávez, J. y Moreno, D. (2023). *Mejorar la disponibilidad en horas operativas de los equipos del departamento de mantenimiento y reparaciones utilizando el pilar de mejora enfocada de TPM* [Tesis de Licenciatura, Universidad Peruana de Ciencias Aplicadas]. Repositorio Académico UPC .

2.1. Competidores

2.1.1. Análisis competitivo

2.1.2. Estrategias y tácticas frente a competidores

2.2. Entrevistas

2.2.1. Diseño de entrevistas

El diseño metodológico de las entrevistas semiestructuradas desarrolladas por TuxLogic responde a la necesidad de investigar a profundidad las dinámicas operativas, hábitos, fricciones y expectativas de los dos actores clave del modelo de mercado bilateral (*two-sided market*) de **ShiftIQ**: el segmento oferente (Dueños o Administradores de Talleres Automotrices Independientes en Lima) y el segmento demandante (Conductores de Vehículos Particulares en Lima Metropolitana).

Para garantizar la rigurosidad científica y la validez empírica del estudio cualitativo, el guion de entrevista se formuló aplicando las siguientes buenas prácticas metodológicas: * **Preguntas Abiertas y No Inducidas:** Se diseñaron interrogantes orientadas a la exploración narrativa (“¿Cómo maneja actualmente...?”, “Describa la última vez que...”), evitando sesgar las respuestas del entrevistado o forzar la validación prematura de la propuesta de valor. * **Enfoque en Comportamientos Pasados y Presentes Reales (*The Mom Test*):** Se priorizó indagar sobre acciones, decisiones y costos reales experimentados en los últimos 6 a 12 meses, restringiendo las especulaciones futuras u opiniones hipotéticas sobre funcionalidades no construidas. * **Secuencia Gradual de Entrevista:** El flujo conversacional se estructuró en cuatro fases sucesivas: (*Rapport* y datos demográficos, Hábitos y rutina diaria, Dolores, frustraciones y barreras, Disposición al cambio y evaluación de canales digitales/IoT). * **Trazabilidad para la Construcción de Arquetipos (*User Personas*):** Cada bloque de preguntas está directamente calibrado para recolectar las dimensiones de información requeridas para los artefactos de Needfinding: variables demográficas (género, edad, distrito, estado civil, familia, ocupación), aspectos psicográficos (personalidad, metas, frustraciones), capacidades tecnológicas (habilidades digitales, dispositivos preferidos, aplicaciones en uso), influencias y

afinidad de marcas (marcas de escáneres, vehículos, apps de banca o navegación) y biografía operativa.

a. Matriz de Información Requerida para Arquetipos de Usuarios

La siguiente matriz delimita el propósito de indagación de cada variable requerida para modelar los arquetipos de usuario de ShiftlQ (*Felipe Hernández* para el Taller y *Raúl Jiménez* para el Conductor):

Dimensión del Arquetipo	Componentes Específicos a Recolectar	Propósito de Indagación en la Entrevista
Datos Demográficos y Biografía	Edad, género, distrito de residencia, estado civil, carga familiar, ocupación actual y años de experiencia en el rubro/manejo.	Delimitar el perfil sociocultural, nivel socioeconómico y contexto de vida cotidiana que enmarca las decisiones financieras y de gestión.
Personalidad y Actitudes	Tendencia pragmática, aversión al riesgo, apertura a la innovación, nivel de cautela en el gasto, nivel de desconfianza comercial.	Comprender la postura mental frente al cambio digital en la administración del taller o ante los diagnósticos mecánicos tradicionales.
Habilidades y Competencias	Habilidad en diagnóstico mecánico/empírico, nivel de alfabetización digital, experiencia en uso de smartphones y computadoras.	Diseñar interfaces de usuario (móvil y web) con la densidad de información y nivel de complejidad adecuados a su competencia real.
Dispositivos y Canales Digitales	Tipo de smartphone (Android/iOS), laptop/PC, aplicaciones de uso diario (WhatsApp, Chrome, Yape, Plin, Waze, Google Maps).	Identificar los ecosistemas de hardware y software donde el usuario pasa su tiempo para integrar notificaciones push y flujos de interacción naturales.
Afinidad por Marcas e Influencias	Marcas de vehículos (Toyota, Hyundai, Nissan), herramientas de diagnóstico (Autel, Launch, Bosch), marcas de repuestos y referencias de confianza.	Establecer el lenguaje visual, las alianzas de hardware (conectores OBD2 compatibles) y el posicionamiento de marca de ShiftIQ.
Objetivos y Motivaciones	Llenar agenda diaria, reducir tiempos muertos, prevenir averías críticas en ruta, transparencia presupuestal, mantener valor de reventa.	Diseñar las funcionalidades principales (<i>core features</i>) que entregan valor directo percibido en los primeros 14 a 30 días.
Frustraciones y Dolores (Pains)	Pérdidas del 30-40% por ineficiencia, cobros excesivos por sorpresa, desconfianza en la jerga técnica,	Justificar los requerimientos funcionales del MVP (alertas DTC en lenguaje natural, agendamiento in-

b. Preguntas de Entrevistas dirigidas al Segmento Objetivo 1: Dueños o Administradores de Talleres Automotrices Independientes (B2B)

Este cuestionario busca comprender a fondo la dinámica administrativa y técnica de los microempresarios mecánicos, evaluando cómo gestionan sus clientes, citas, presupuestos e inventarios en su operación diaria en Lima.

Bloque 1: Perfil Demográfico, Biografía y Entorno Operativo (Arquetipo: Felipe Hernández)

- **Pregunta Principal 1.1:** Para comenzar, coméntenos sobre usted: ¿cuál es su edad, en qué distrito reside, su estado civil y qué rol desempeña actualmente en el taller?
 - *Pregunta Complementaria 1.1a:* ¿Cuántos años de experiencia tiene en el rubro automotriz y cómo se constituyó su taller?
 - *Pregunta Complementaria 1.1b:* ¿Cuántas personas integran su núcleo familiar y cómo equilibra el tiempo de trabajo en el taller con su vida personal?
- **Pregunta Principal 1.2:** Describa la estructura y capacidad operativa de su establecimiento.
 - *Pregunta Complementaria 1.2a:* ¿Con cuántos mecánicos o técnicos cuenta y cuántas bahías o rampas de atención tiene habilitadas?
 - *Pregunta Complementaria 1.2b:* En promedio, ¿cuántos vehículos atienden a la semana y cuáles son los servicios de mayor demanda?

Bloque 2: Procesos de Gestión, Dispositivos y Canales de Interacción

- **Pregunta Principal 2.1:** ¿Cómo organiza actualmente el flujo diario de trabajo, la asignación de citas y el seguimiento de los vehículos atendidos?
 - *Pregunta Complementaria 2.1a:* ¿Qué herramientas utiliza para registrar la información del cliente y la orden de servicio (cuaderno físico, hojas de cálculo, WhatsApp, software de gestión)?
 - *Pregunta Complementaria 2.1b:* ¿Qué dispositivos tecnológicos utiliza personalmente durante la jornada de trabajo (smartphone Android/iOS, tablet, laptop, PC de escritorio)?
- **Pregunta Principal 2.2:** ¿Qué canales de comunicación digital utiliza de forma habitual para interactuar con sus clientes y proveedores?
 - *Pregunta Complementaria 2.2a:* ¿Cómo gestiona la recepción de solicitudes por WhatsApp y qué problemas encuentra al coordinar por este medio?
 - *Pregunta Complementaria 2.2b:* ¿Qué aplicaciones móviles o páginas web utiliza con mayor frecuencia en su rutina diaria de negocio o personal?

Bloque 3: Frustraciones, Dolores Operativos y Relación con el Cliente (Pains & Gains)

- **Pregunta Principal 3.1:** ¿Cuáles son los principales obstáculos o ineficiencias que enfrenta en la gestión diaria que afectan la rentabilidad del taller?
 - *Pregunta Complementaria 3.1a:* ¿Ha experimentado periodos de tiempo muerto o baja ocupación de rampas durante el mes? ¿A qué cree que se deba y cómo lo soluciona?
 - *Pregunta Complementaria 3.1b:* ¿Cómo maneja el control de inventario de repuestos e insumos? ¿Ha tenido pérdidas por falta de stock o repuestos mal presupuestados?

- **Pregunta Principal 3.2:** En su experiencia, ¿por qué razón suelen regresar los clientes a su taller y cómo promueve que realicen mantenimientos preventivos regulares?
 - *Pregunta Complementaria 3.2a:* ¿Qué tan frecuente es que un cliente llegue únicamente cuando el vehículo ya presenta una avería grave o inmovilizante?
 - *Pregunta Complementaria 3.2b:* ¿Cómo reaccionan los clientes ante la presentación de presupuestos adicionales o reparaciones no planificadas?

Bloque 4: Afinidad por Marcas, Equipamiento Técnico y Evaluación de la Solución ShiftIQ

- **Pregunta Principal 4.1:** Al realizar diagnósticos en los vehículos, ¿qué marcas de escáneres automotrices o herramientas electrónicas prefiere o utiliza (Autel, Launch, Bosch, otros)?
 - *Pregunta Complementaria 4.1a:* ¿En qué etapa de la atención utiliza el escáner y qué limitaciones encuentra en los equipos diagnósticos actuales?
 - *Pregunta Complementaria 4.1b:* ¿Qué marcas de vehículos (Toyota, Hyundai, Nissan, Kia, etc.) atienden con mayor frecuencia y qué tan complejo es acceder a su información de fallas?
- **Pregunta Principal 4.2:** Si dispusiera de una plataforma móvil conectada a dispositivos telemáticos OBD2 que le enviara alertas de fallas electrónicas (DTC) de sus clientes antes de que lleguen al taller, ¿cómo transformaría eso su manera de agendar y ofrecer servicios?
 - *Pregunta Complementaria 4.2a:* ¿Qué tan dispuesto estaría a adoptar una aplicación móvil para enviar cotizaciones digitales y confirmar citas en tiempo real?
 - *Pregunta Complementaria 4.2b:* ¿Qué condiciones o facilidades requeriría para confiar e integrar una solución SaaS como ShiftIQ en su taller?

c. Preguntas de Entrevistas dirigidas al Segmento Objetivo 2: Conductores de Vehículos Particulares en Lima (B2C)

Este cuestionario busca explorar las conductas de mantenimiento, las barreras cognitivas/financieras, la asimetría informativa y la competencia digital de los propietarios de vehículos particulares en Lima Metropolitana.

Bloque 1: Perfil Demográfico, Biografía y Hábitos de Conducción (Arquetipo: Raúl Jiménez)

- **Pregunta Principal 1.1:** Para comenzar, coméntenos sobre usted: ¿cuál es su edad, en qué distrito reside, su estado civil y a qué se dedica profesionalmente?
 - *Pregunta Complementaria 1.1a:* ¿Cuántas personas dependen de usted o forman parte de su hogar y qué papel cumple el vehículo en su dinámica familiar?
 - *Pregunta Complementaria 1.1b:* ¿Qué modelo, marca y año de fabricación tiene su vehículo particular, y cuál es el uso principal que le da (traslado laboral, viajes de fin de semana, aplicativo)?
- **Pregunta Principal 1.2:** ¿Cómo describiría su rutina de conducción semanal y cuántos kilómetros recorre aproximadamente al día en Lima?

Bloque 2: Experiencia y Hábitos de Mantenimiento Vehicular

- **Pregunta Principal 2.1:** ¿Cómo maneja actualmente el cuidado y mantenimiento de su vehículo? ¿Sigue un plan preventivo o acude al taller cuando nota un problema específico?
 - *Pregunta Complementaria 2.1a:* Describa la última vez que su vehículo presentó una falla mecánica o una luz encendida en el tablero. ¿Cómo reaccionó y qué pasos siguió?
 - *Pregunta Complementaria 2.1b:* ¿Con qué frecuencia realiza cambios de aceite, revisión de frenos o alineación, y cómo recuerda las fechas o kilometrajes correspondientes?
- **Pregunta Principal 2.2:** ¿Qué factores o motivos le han llevado en alguna ocasión a postergar o posponer la revisión mecánica de su automóvil?
 - *Pregunta Complementaria 2.2a:* ¿Ha influido la falta de tiempo, la incertidumbre económica o la falta de claridad en los costos para postergar una atención técnica?

Bloque 3: Relación con el Taller, Asimetría Informativa y Frustraciones (*Pains & Gains*)

- **Pregunta Principal 3.1:** ¿Cómo elige el taller mecánico al que lleva su automóvil y qué tan transparente considera la información técnica y los presupuestos que recibe?
 - *Pregunta Complementaria 3.1a:* ¿Ha experimentado dificultades para comprender la terminología técnica o los códigos de falla expuestos por el mecánico? ¿Cómo le hizo sentir esa situación?
 - *Pregunta Complementaria 3.1b:* ¿Ha tenido experiencias donde el costo final del servicio superó de forma imprevista la cotización inicial?
- **Pregunta Principal 3.2:** ¿Qué es lo que más valora al momento de solicitar un servicio técnico vehicular y qué le haría sentir total confianza en un taller independiente?

Bloque 4: Canales Digitales, Dispositivos Preferidos, Afinidad por Apps y Aceptación de Telemetría IoT

- **Pregunta Principal 4.1:** ¿Qué modelo de teléfono smartphone utiliza (Android/iOS) y qué aplicaciones móviles utiliza con mayor frecuencia en su vida diaria?
 - *Pregunta Complementaria 4.1a:* ¿Utiliza habitualmente aplicaciones de navegación GPS (Waze, Google Maps), mensajería (WhatsApp) o banca/billeteras digitales (Yape, Plin)?
 - *Pregunta Complementaria 4.1b:* ¿Qué tan cómodo se siente recibiendo notificaciones push relativas al estado de sus servicios o pagos en su smartphone?
- **Pregunta Principal 4.2:** Si contara con un pequeño dispositivo de diagnóstico (OBD2) conectado a su auto que se comuniquen con su smartphone para explicarle en español sencillo el nivel de urgencia de una falla y le permita agendar cita en un clic, ¿qué opinión le merecería?
 - *Pregunta Complementaria 4.2a:* ¿Estaría dispuesto a mantener este dispositivo conectado continuamente a cambio de recibir alertas de seguridad y estimados de costo preventivos?
 - *Pregunta Complementaria 4.2b:* ¿Qué garantías de privacidad y transparencia en el uso de los datos de su vehículo consideraría indispensables para adoptar la aplicación ShiftIQ?

2.2.2. Registro de entrevistas

Segmento Objetivo 1: Dueños o Administradores de Talleres Automotrices Independientes (B2B)

Entrevista 1: Gerente General y Propietario de Taller Automotriz Independiente Nombre Roberto Apellidos Silva Edad 37 años Distrito San Miguel, Juliaca Aplicaciones Usadas WhatsApp, BCP/Interbank, Excel y correo electrónico Tecnologías Smartphone Android, PC de escritorio, escáner Launch y Bosch KTS Browsers Google Chrome, Microsoft Edge Entrevistador Alan Mamani Evidencia Link Ver entrevista Duración 21:30 Resumen El entrevistado es gerente general y propietario de un taller automotriz independiente, con 14 años de experiencia en el sector. El taller cuenta con cuatro mecánicos y cuatro bahías de atención, y atiende entre 20 y 30 vehículos por semana. Actualmente gestiona las órdenes mediante documentos físicos, utiliza Excel para la contabilidad y caja, y WhatsApp para comunicarse con los clientes. Entre sus principales problemas se encuentran la baja ocupación de las bahías durante los primeros días de la semana, los vehículos que permanecen varios días en el taller y las pérdidas ocasionadas por repuestos solicitados que posteriormente son cancelados por el cliente. También señala que los clientes suelen desconfiar de reparaciones adicionales y que aproximadamente el 80 % llega cuando el vehículo ya presenta una falla grave. Respecto a ShiftIQ, considera interesante el uso de dispositivos OBD2 para detectar fallas antes de que el vehículo llegue al taller y ayudar a llenar las bahías en días de baja demanda. Sin embargo, requiere que la solución demuestre beneficios económicos concretos, tenga un costo inicial accesible, sea sencilla de instalar y no genere problemas para los clientes. Además, considera importante mantener canales tradicionales como llamadas y WhatsApp para aquellos clientes que tengan dificultades con una aplicación.

Entrevista 2: Administrador de Taller Automotriz Independiente y Gestión de Operaciones Nombre Sebastián Apellidos Rojas Espinoza Edad 30 años Distrito San Martín de Porres, Lima Aplicaciones Usadas Excel, WhatsApp Business, Google Sheets, banca móvil, Yape, Plin, BCP, Instagram, YouTube Tecnologías Smartphone Samsung Android, laptop con Windows, escáneres automotrices Autel y Launch Browsers Google Chrome Entrevistador Alan Mamani Evidencia Link Ver entrevista Duración 22:34 Resumen El entrevistado es administrador de un taller automotriz independiente y cuenta con aproximadamente 7 años de experiencia en el sector. Se encarga de las operaciones, recepción de clientes, planificación de agenda, inventario, cotizaciones, gestión financiera y coordinación del trabajo del equipo. El taller cuenta con 3 técnicos mecánicos, 1 asistente y 3 bahías de trabajo, de las cuales 2 tienen elevadores hidráulicos, además de áreas de diagnóstico computarizado y trabajos eléctricos. Actualmente utiliza Excel para gestionar historiales de servicios e ingresos, WhatsApp Business para comunicarse con clientes y proveedores, y progresivamente está migrando algunos registros a Google Sheets para facilitar el acceso compartido entre el personal administrativo. Entre sus principales dificultades se encuentran la variabilidad en la ocupación de las bahías, los clientes que olvidan sus mantenimientos, los retrasos en la cotización y disponibilidad de repuestos y los cruces o cancelaciones de citas de último momento. Respecto a una solución conectada a dispositivos telemáticos OBD2, considera que las alertas de fallas en tiempo real permitirían contactar anticipadamente a los clientes, ofrecer servicios y agendar citas automáticamente en los días de menor ocupación, optimizando el uso de las bahías y aumentando la facturación. Para adoptar una solución de este tipo, considera importante que la interfaz sea clara, intuitiva y fácil de aprender, que exista

un periodo de prueba y que las notificaciones sean compatibles con los smartphones utilizados por el equipo, como Samsung y Xiaomi. Entrevista #3 Nombre ... Apellidos ... Edad ... Distrito ... Aplicaciones Usadas ... Tecnologías ... Browsers ... Entrevistador ... Evidencia ... Link ... Duración ... Resumen ... **Segmento Objetivo 2: Conductores de Vehículos Particulares en Lima (B2C)**

Entrevista #1 Nombre ... Apellidos ... Edad ... Distrito ... Aplicaciones Usadas ... Tecnologías ... Browsers ... Entrevistador ... Evidencia ... Link ... Duración ... Resumen ... Entrevista #2 Nombre ... Apellidos ... Edad ... Distrito ... Aplicaciones Usadas ... Tecnologías ... Browsers ... Entrevistador ... Evidencia ... Link ... Duración ... Resumen ... Entrevista #3 Nombre ... Apellidos ... Edad ... Distrito ... Aplicaciones Usadas ... Tecnologías ... Browsers ... Entrevistador ... Evidencia ... Link ... Duración ... Resumen ...

2.2.3. Análisis de entrevistas

2.3. Needfinding

En esta sección se presentan los artefactos derivados del análisis de necesidades de los dos segmentos objetivo de Shiftlq: dueños o administradores de talleres automotrices independientes y conductores de vehículos particulares en Lima Metropolitana. El needfinding articula la investigación cualitativa con el diseño de una solución **móvil** (aplicaciones nativas y/o multiplataforma), de modo que las personas, tareas, journeys y mapas de empatía alimenten directamente los requerimientos de experiencia de usuario, conectividad, notificaciones y ética de datos que exige el curso de Aplicaciones para Dispositivos Móviles.

2.3.1. User Personas

Para desarrollar la propuesta de solución móvil se elabora un *User Persona* por cada segmento objetivo. Cada persona sintetiza motivaciones, frustraciones, hábitos digitales y contexto de uso del smartphone, de forma que el equipo de TuxLogic diseñe flujos nativos/cross-platform alineados a comportamientos reales y no a supuestos genéricos.

Segmento Objetivo 1: Dueños o administradores de talleres automotrices independientes en Lima

Figura 8

User persona — Felipe Hernández (dueño / administrador de taller)

NAME
Felipe Hernandez

TAMAÑO DEL MERCADO
45 %

TYPE
Dueño De Taller Pragmático Y Precavido

Goals

- Reducir los tiempos muertos y mantener una agenda de trabajo constante.
- Anticipar mantenimientos para evitar reparaciones de emergencia.
- Fidelizar clientes mediante un servicio preventivo y confiable.
- Organizar citas, vehículos, diagnósticos e historial desde un solo lugar.
- Recibir alertas tempranas sobre posibles fallas antes de que se conviertan en problemas graves.
- Mejorar la comunicación con sus clientes y generar mayor confianza.
- Incrementar los ingresos del taller sin depender únicamente de reparaciones urgentes.

Quote

«Si me avisan antes de que el auto se quede parado, yo llamo al cliente y lleno mi agenda.»

Background

Felipe Hernandez tiene 46 años y vive en San Martín de Porres, Lima. Es dueño y administrador de un taller automotriz multimarca independiente desde hace 12 años. Su experiencia le ha permitido conocer muy bien las necesidades de sus clientes y detectar problemas frecuentes en los vehículos. Sin embargo, gran parte de la gestión del taller todavía depende de herramientas tradicionales como cuadernos, llamadas y WhatsApp. Actualmente atiende entre 15 y 25 vehículos por semana, por lo que necesita organizar mejor sus citas, diagnósticos e historial de mantenimiento. Felipe quiere pasar de un modelo basado principalmente en reparaciones cuando ocurre una falla a uno más preventivo, donde pueda anticiparse a los problemas y contactar al cliente antes de que su vehículo presente una avería grave.

Motivations

- Mantener una relación cercana y confiable con sus clientes.
- Lograr que los clientes regresen para mantenimientos preventivos.
- Reducir la incertidumbre causada por una demanda irregular.
- Diferenciar su taller frente a la competencia mediante un servicio más profesional.
- Tener mayor control sobre citas, historial de vehículos y alertas.
- Evitar que los clientes lleguen únicamente cuando el vehículo ya presenta una falla grave.
- Aumentar la rentabilidad del taller mediante servicios planificados.

Frustrations

- Muchos clientes llegan al taller solo cuando el vehículo ya presenta una falla grave.
- La demanda de servicios es irregular, dificultando la planificación de ingresos y recursos.
- Depende de libretas, llamadas y WhatsApp para recordar citas y mantenimientos.
- Puede olvidar cuándo corresponde realizar el mantenimiento de determinados vehículos.
- Pierde oportunidades de fidelización por no tener un seguimiento constante de sus clientes.
- Le preocupa invertir en una tecnología difícil de usar o que no se adapte al ritmo del taller.
- La falta de información organizada dificulta saber qué vehículo necesita una reparación o mantenimiento.
- Los tiempos muertos afectan directamente los ingresos del taller.

Skills

Diagnóstico mecánico y automotriz

Atención y comunicación con clientes

Experiencia en gestión de taller

Organización de citas y servicios

Uso de software de gestión

Technology

Android, Windows, iOS

Browsers

Google Chrome

Channels

WhatsApp, Phone, Smartphone, Laptop, Facebook, Checklist

Brands and influencers

AUTEL ROBOTICS, TOYOTA, LAUNCH, HYUNDAI, BOSCH

Campo	Detalle
Nombre	Felipe Hernández
Tipo	Dueño de taller pragmático y precavido (≈ 45% del mercado objetivo)
Edad / Distrito	46 años · San Martín de Porres, Lima
Estado civil	Casado
Ocupación	Dueño y administrador de un taller multimarca independiente (12 años de operación)
Volumen	Atiende entre 15 y 25 vehículos por semana
Objetivo principal	Reducir tiempos muertos, fidelizar clientes y pasar de un modelo reactivo a uno preventivo sin perder el control diario del taller
Frustraciones	Clientes solo llegan con falla grave; demanda irregular; olvidos por notas manuales; temor a software complicado; tiempos muertos que afectan ingresos
Habilidades	Alta en diagnóstico mecánico y atención al cliente; media en organización de citas; baja en software de gestión
Uso de tecnología	Android + Windows; Chrome; WhatsApp, teléfono, laptop, Facebook
Marcas de referencia	Autel, Toyota, Launch, Hyundai, Bosch
Quote	«¡Si me avisan antes de que el auto se quede parado, yo llamo al cliente y lleno mi agenda!»

Segmento Objetivo 2: Conductores de vehículos de Lima

Figura 9

User persona — Raúl Jiménez (conductor particular)

NAME
Raúl Jiménez

MARKET SIZE
 **35 %**

TYPE
Conductor Particular Práctico Y Preventivo



Goals

- Mantener su vehículo seguro y operativo durante sus desplazamientos diarios.
- Evitar averías inesperadas mientras conduce.
- Recibir recordatorios de mantenimiento según el kilometraje.
- Entender fácilmente qué problema presenta su vehículo.
- Conocer el nivel de urgencia de una posible falla.
- Evitar gastos elevados por reparaciones de emergencia.
- Poder agendar una revisión con su taller de manera rápida.

Demographic

Male 35 years

Los Olivos, Lima, Perú

Married

Analista de operaciones

Utiliza su vehículo constantemente y busca mantenerlo operativo sin gastar dinero innecesariamente en reparaciones.

Quote

“ Si sé qué tiene el auto y qué tan urgente es, puedo solucionarlo antes de que se convierta en un problema mayor. ”

Skills

Conducción y experiencia vehicular

0 25 50 75 100

Uso de aplicaciones móviles

0 25 50 75 100

Conocimiento básico de mantenimiento

0 25 50 75 100

Identificación de problemas del vehículo

0 25 50 75 100

Gestión del mantenimiento del vehículo

0 25 50 75 100

Background

Raúl Jiménez tiene 34 años y vive en Los Olivos, Lima. Trabaja como analista de operaciones y utiliza su vehículo principalmente para trasladarse diariamente entre su hogar y el trabajo, además de realizar viajes durante los fines de semana.

Tiene conocimientos básicos sobre el funcionamiento de su vehículo, pero no domina los términos técnicos utilizados por los mecánicos. Cuando aparece una alerta en el tablero, normalmente busca información en Internet o consulta directamente con su taller de confianza.

Uno de sus principales problemas es que suele postergar los mantenimientos hasta que aparece una señal evidente de falla. También le preocupa no saber si una reparación realmente es necesaria o si el presupuesto recibido es demasiado elevado.

Por ello, busca una solución que le permita entender el estado de su vehículo en un lenguaje sencillo, recibir alertas oportunas y contactar a su taller sin tener que realizar llamadas prolongadas.

Motivations

- Mantener su vehículo en buenas condiciones.
- Evitar quedarse varado durante sus desplazamientos.
- Tener mayor seguridad al conducir.
- Ahorrar dinero evitando reparaciones de emergencia.
- Recibir información clara y fácil de entender.
- Saber cuándo realmente necesita llevar el vehículo al taller.
- Tener confianza en las recomendaciones y presupuestos del mecánico.
- Resolver sus necesidades de mantenimiento sin perder demasiado tiempo.

Frustrations

- No comprende fácilmente los términos técnicos utilizados por los mecánicos.
- Tiene dudas sobre si algunas reparaciones realmente son necesarias.
- Le preocupa recibir presupuestos elevados o cobros inesperados.
- Olvida cuándo corresponde realizar el mantenimiento del vehículo.
- Suele prestar atención al vehículo cuando aparece una alerta o problema evidente.
- No sabe qué tan urgente es una falla cuando aparece un indicador en el tablero.
- Las llamadas y coordinaciones con el taller pueden tomar demasiado tiempo.

Brands and influencers








Technology




Browsers



Channels





Campo	Detalle
Nombre	Raúl Jiménez
Tipo	Conductor particular práctico y preventivo (≈ 35% del mercado objetivo)
Edad / Distrito	35 años · Los Olivos, Lima
Estado civil	Casado
Ocupación	Analista de operaciones; usa el auto a diario (casa–trabajo) y fines de semana
Objetivo principal	Mantener el vehículo seguro y operativo, evitar averías inesperadas y entender urgencias sin jerga técnica
Frustraciones	No entiende términos del mecánico; duda de presupuestos; olvida mantenimientos; no sabe urgencia de luces del tablero; llamadas largas para coordinar
Habilidades	Alta en uso de apps móviles y manejo; baja–media en identificación de fallas y gestión de mantenimiento
Uso de tecnología	Smartphone Android; Chrome; WhatsApp, Plin, Yape, Waze, Google Maps
Quote	«Si sé qué tiene el auto y qué tan urgente es, puedo solucionarlo antes de que se convierta en un problema mayor.»

2.3.2. User Task Matrix

Se presenta la *User Task Matrix* de los segmentos objetivo. Cada tarea se evalúa según **importancia** y **frecuencia** para priorizar qué flujos deben resolverse primero en las aplicaciones móviles de Shiftlq (tablero del taller vs. app del conductor).

Tabla 8

User Task Matrix — Shiftlq

Tareas (Task) Felipe Hernández (Dueño / Admin. taller) Raúl Jiménez (Conductor) Importancia Frecuencia Importancia Frecuencia Revisar estado general / salud del vehículo Alta Alta Alta Media Recibir y priorizar alertas DTC / telemetría en el móvil Alta Alta Alta Media Buscar o seleccionar taller de confianza N/A N/A Alta Media Agendar cita de mantenimiento desde el smartphone Alta Alta Alta Media Evitar averías inesperadas durante traslados Media Baja Alta Media Interpretar información técnica automotriz (sin jerga) Alta Media Alta Baja* Gestionar órdenes de trabajo y agenda del taller Alta Alta N/A N/A Comunicación taller–cliente (estado,

presupuesto, entrega) Alta Alta Alta Alta Compra / control de repuestos e insumos Alta Media N/A N/A Seguimiento de historial de servicios del vehículo Alta Media Alta Baja Limpieza y mantenimiento básico del vehículo Media Media Media Alta Evaluar costos de combustible / operación del auto Baja Baja Alta Media *Hoy la frecuencia es baja porque el conductor evita o no entiende la información técnica; la app debe **eleva la frecuencia útil** al traducir DTC a lenguaje natural.

Implicancia para el diseño móvil: las tareas de alta importancia + alta frecuencia (alertas DTC, agenda, comunicación y órdenes) definen el MVP de las apps de Shiftlq y justifican una estrategia **multiplataforma (p. ej. Flutter)** o nativa, con notificaciones push, Material Design 3 / Human Interface Guidelines y persistencia local ante cortes de red.

2.3.3. User Journey Mapping

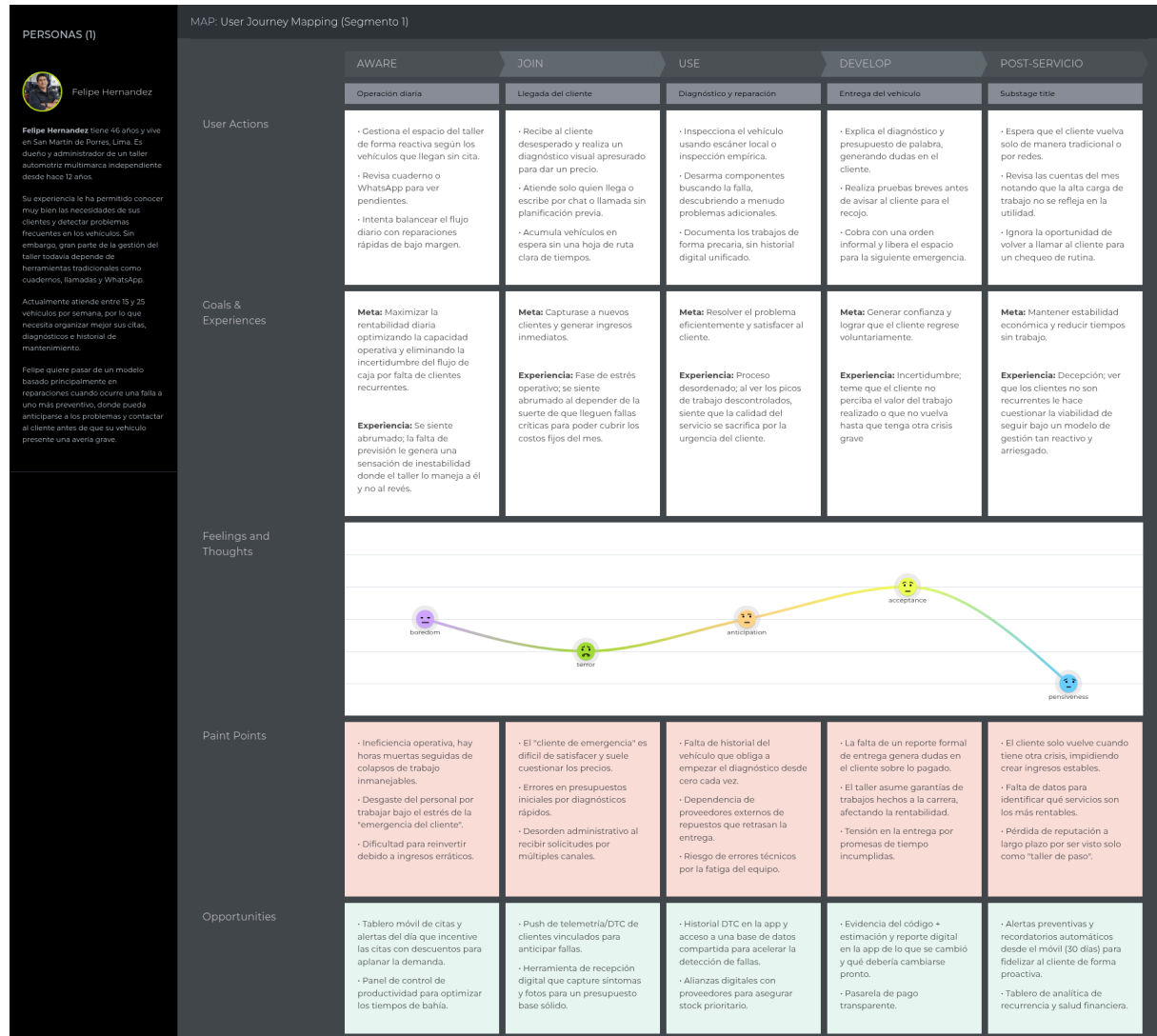
El *User Journey Mapping* visualiza el recorrido del usuario para alcanzar un objetivo, incluyendo acciones, touchpoints (sobre todo **móviles**), emociones y puntos de dolor. Sirve para detectar dónde Shiftlq debe intervenir con la app.

Segmento Objetivo 1: Dueños o administradores de talleres

Objetivo del journey: llenar la agenda con servicios preventivos y reducir la demanda solo-reactiva.

Figura 10

User journey mapping — Felipe Hernández (taller)



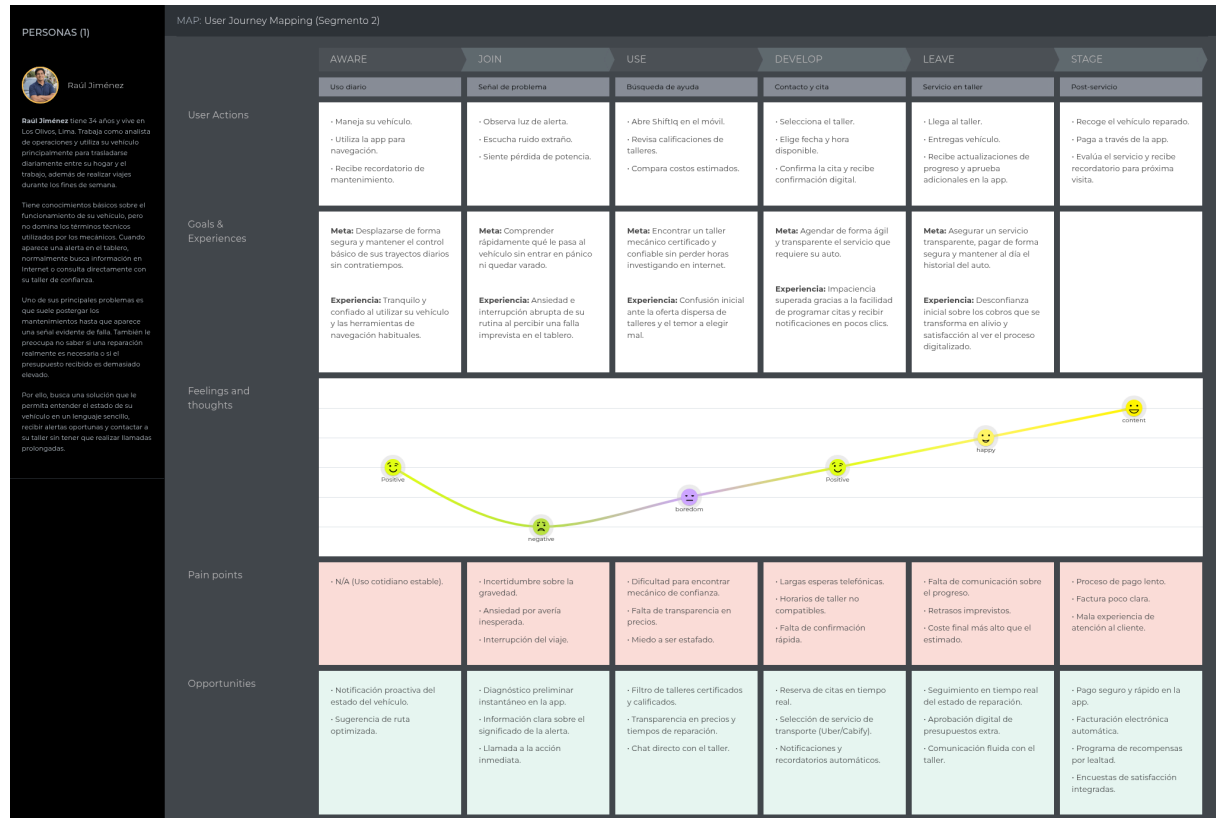
Etapa	Acciones actuales	Emoción	Pain points	Oportunidad Shiftlq (móvil)
1. AWARE — Operación diaria	Gestiona espacio y demanda de forma reactiva	Rutina / aburrimiento	Tiempos muertos e ineficiencia	Tablero móvil de citas + alertas para aplazar picos
2. JOIN — Llegada del cliente	Atiende walk-ins sin cita; recepción improvisada	Estrés / caos	Cotizaciones apresuradas y errores	Recepción digital: síntomas, fotos y presupuesto inicial
3. USE — Diagnóstico y reparación	Inspección empírica; poco historial digital	Anticipación	Falta historial centralizado del vehículo	Historial + telemetría/DTC en la app
4. DEVELOP — Entrega	Explica de palabra; cliente duda del trabajo	Aceptación	Desconfianza sobre lo reparado	Informe digital con evidencias y estimación
5. POST-SERVICE	Espera que el cliente vuelva solo	Decepción	Baja lealtad; imagen de “taller de paso”	Alertas preventivas cada 30 días + métricas de recurrencia

Segmento Objetivo 2: Conductores de vehículos

Objetivo del journey: prevenir una avería grave y agendar mantenimiento con confianza.

Figura 11

User journey mapping — Raúl Jiménez (conductor)



Etapa	Acciones actuales	Emoción	Pain points	Oportunidad Shiftlq (móvil)
1. AWARE — Uso diario	Conduce; usa navegación; recibe recordatorios	Estable / positivo	No monitorea salud del auto	Notificaciones proactivas de estado
2. JOIN — Señal de problema	Luz en tablero, ruido, pérdida de potencia	Ansiedad (punto más bajo)	No sabe la gravedad; interrumpe el viaje	Diagnóstico preliminar in-app + significado claro de alertas
3. USE — Búsqueda de ayuda	Abre Shiftlq; compara talleres, ratings y costos	Confusión	Miedo a estafas; talleres poco confiables	Filtro de talleres certificados + transparencia de precio/tiempo
4. DEVELOP — Contacto y cita	Elige taller, fecha y confirma	Mejora	Esperas telefónicas; confirmación lenta	Reserva en tiempo real + recordatorios
5. LEAVE — Servicio en taller	Deja el auto; aprueba extras	Alivio	Falta de avances; costos sorpresa	Tracking del servicio + aprobación digital de presupuestos
6. STAGE — Post-servicio	Recoge, paga y califica	Satisfacción	Pagos lentos; boletas poco claras	Pago in-app, comprobante digital y fidelización

2.3.4. Empathy Mapping

El *Empathy Mapping* profundiza en lo que cada persona **dice, piensa, hace y siente**, además de sus dolores (*pains*) y ganancias deseadas (*gains*). Orienta decisiones de UX móvil, tono de las notificaciones y ética de comunicación (transparencia, sin alarmismo engañoso).

Segmento Objetivo 1: Dueño / administrador de taller

Figura 12

Empathy Mapping — Felipe Hernández

Usuario: Dueño y administrador de un taller automotriz multimarca

- **Nombre:** Felipe Hernández
- **Edad:** 46 años
- **Ubicación:** San Martín de Porres, Lima, Perú

1.WHO are we empathizing with?

Felipe Hernández es un hombre de 46 años que vive en San Martín de Porres y administra un taller automotriz independiente. Se define como un profesional pragmático y precavido, con sólidas habilidades técnicas en diagnóstico y reparación mecánica (así como un gran manejo de herramientas tradicionales como Autel, Launch y Bosch). Sin embargo, gran parte de su gestión operativa diaria todavía depende de cuadernos, llamadas y WhatsApp, atendiendo entre 15 y 25 vehículos por semana en un modelo mayoritariamente reactivo.

7.What do they THINK and FEEL?

“What thoughts and feelings might motivate their behavior?”



2.What do they need to DO?

Felipe necesita transicionar de un modelo basado en reparaciones correctivas de emergencia a uno más preventivo. Debe organizar eficientemente sus citas, el historial de los vehículos y los diagnósticos en un solo lugar centralizado, logrando anticiparse a las fallas mecánicas de sus clientes para aplanar la demanda, reducir sus horas muertas y asegurar ingresos constantes para su taller.

6.What do they HEAR?

- A sus clientes recurrentes quejarse de sorpresas en las reparaciones cuando el daño ya avanzó demasiado.
- Comentarios de colegas del sector automotriz sobre la pérdida de dinero debido a una mala coordinación de repuestos y citas.
- Voces del mercado que exigen digitalización, aunque muchas veces se ofertan sistemas muy lejanos a la dinámica diaria de un taller familiar.

3.What do they SEE?

- Un flujo de trabajo irregular donde los vehículos solo llegan cuando el problema ya es grave.
- Una alta dependencia de métodos rústicos de anotación (libretas físicas y chats dispersos).
- El crecimiento constante del parque automotor multimarca en Lima Norte y la presión de mantenerse competitivo frente a otros talleres.
- Su entorno de trabajo físico rodeado de herramientas de diagnóstico y marcas con las que labora cotidianamente (Toyota, Hyundai, Bosch, Autel).

5.What do they DO?

- **Pensamientos:**
 - «Necesito reducir los tiempos muertos y mantener una agenda de trabajo constante».
 - «Quiero anticiparme a los mantenimientos para evitar que los clientes lleguen solo por reparaciones de emergencia».
 - «Tengo la experiencia técnica, pero la gestión administrativa informal me resta rentabilidad».
- **Sentimientos:**
 - *Prudencia y desconfianza:* Recelo inicial hacia softwares o tecnologías de gestión que perciba demasiado complejos o ajenos a su realidad de taller independiente.
 - *Orgullo y compromiso:* Alta satisfacción por la calidad de su trabajo técnico y la relación cercana que cultiva con su clientela de confianza.
 - *Estrés operativo:* Preocupación constante por la irregularidad de los ingresos y la falta de una herramienta unificada de control.

PAINS

- Muchos clientes llegan al taller solo cuando el vehículo ya presenta una falla grave.
- La demanda de servicios es irregular, dificultando la planificación de ingresos y recursos.
- Depende de libretas, llamadas y WhatsApp para recordar citas, historiales y mantenimientos.
- Puede olvidar cuándo corresponde realizar el servicio de determinados vehículos, perdiendo oportunidades de fidelización.
- Le preocupa invertir en tecnología difícil de usar que no se adapte al ritmo operativo de su taller.
- Los tiempos muertos y la falta de información organizada afectan directamente la rentabilidad del negocio.

GAINS

- Mantener una agenda de trabajo constante y reducir los tiempos muertos del taller.
- Anticipar mantenimientos preventivos mediante alertas tempranas para evitar reparaciones de emergencia.
- Fidelizar a los conductores a través de un servicio transparente, organizado y confiable.
- Centralizar la gestión de citas, vehículos, diagnósticos e historiales desde una sola plataforma digital intuitiva.
- Incrementar los ingresos generales del negocio sin depender únicamente de fallas imprevistas.

4.What do they SAY?

- “
- «Si me avisan antes de que el auto se quede parado, yo llamo al cliente y lleno mi agenda».
 - «WhatsApp se me desordena con tantos mensajes de cotizaciones y consultas sueltas».
 - «El escáner solo me sirve cuando el auto ya está físicamente aquí en el taller».
 - «Quiero modernizarme, pero temo que una aplicación sea muy complicada de usar o me haga perder tiempo valioso».
- ”

Cuadrante	Insights
Quién	Dueño de taller multimarca en SMP; pragmático, hábil en diagnóstico (Autel, Launch, Bosch), pero administra con cuaderno y WhatsApp
Necesita hacer	Pasar de reactivo a preventivo; centralizar citas, historial y diagnósticos; anticipar fallas para reducir tiempos muertos
Ve	Autos que solo llegan en emergencia; métodos rústicos; presión competitiva en Lima Norte
Dice	«Si me avisan antes de que el auto se quede parado, lleno mi agenda»; «WhatsApp se me desordena»; «El escáner solo sirve cuando el auto ya está aquí»; «Quiero modernizarme, pero temo una app complicada»
Oye	Quejas por “sorpresas” de precio; colegas que pierden dinero por mala coordinación; presión a digitalizarse
Piensa / siente	Quiere reducir tiempos muertos; prudencia ante software complejo; orgullo técnico; estrés por ingresos irregulares
Pains	Demanda irregular; dependencia de papel/ WhatsApp; olvido de mantenimientos; miedo a invertir mal en tech
Gains	Agenda constante; alertas tempranas; fidelización; plataforma intuitiva centralizada; más ingresos planificados

Segmento Objetivo 2: Conductor

Figura 13

Empathy Mapping — Raúl Jiménez

PERSONA: Empathy Mapping (Segmento 1)

Usuario: Conductor Particular Práctico y Preventivo

- **Nombre:** Raúl Jiménez
- **Edad:** 35 años
- **Ubicación:** Los Olivos, Lima, Perú

1.WHO are we empathizing with?

Raúl Jiménez es un hombre de 35 años que vive en Los Olivos y trabaja como analista de operaciones. Utiliza su vehículo de forma diaria para trasladarse entre su hogar y su centro de labores, además de realizar viajes los fines de semana. Cuenta con conocimientos básicos sobre el funcionamiento de su auto y un buen manejo de aplicaciones móviles cotidianas (como Waze, WhatsApp, Yape y Plin). Es un conductor de perfil práctico y preventivo que busca mantener su vehículo operativo sin gastar dinero innecesariamente en reparaciones imprevistas.

7.What do they THINK and FEEL?

“What thoughts and feelings might motivate their behavior?”



2.What do they need to DO?

Raúl necesita monitorear y mantener su vehículo seguro y operativo en sus desplazamientos diarios, evitando averías inesperadas. Debe ser capaz de entender de forma sencilla qué problema presenta su vehículo al encenderse una alerta en el tablero, conocer el nivel de urgencia real y poder agendar revisiones o mantenimientos preventivos con su taller de confianza de manera rápida y sin llamadas telefónicas prolongadas.

6.What do they HEAR?

- Comentarios de amigos y compañeros de trabajo sobre malas experiencias o cobros excesivos en talleres mecánicos informales.
- Advertencias de familiares sobre la importancia de hacer mantenimientos preventivos a tiempo para cuidar la economía del hogar.
- Recomendaciones sobre el uso de aplicaciones móviles y canales digitales para agilizar servicios cotidianos (como pagos por Yape/Plin o coordinaciones por WhatsApp).

3.What do they SEE?

- Indicadores y alertas luminosas repentinas en el tablero de su vehículo durante sus trayectos diarios.
- Una oferta dispersa y poco clara de talleres mecánicos tradicionales donde teme ser estafado o recibir cobros excesivos.
- La necesidad constante de optimizar sus tiempos de traslado utilizando herramientas digitales (como Waze y Google Maps) en su smartphone.
- Su entorno de movilidad urbana en Lima Norte, donde una avería imprevista interrumpe abruptamente su rutina laboral o personal.

5.What do they DO?

- **Pensamientos:**
 - «Debo mantener mi auto en buenas condiciones para evitar quedarme varado camino al trabajo».
 - «Me preocupa recibir presupuestos inflados o cobros excesivos por reparaciones que quizás no necesitaba todavía».
 - «Necesito recordatorios claros según el kilometraje para no olvidar las fechas clave de mantenimiento».
- **Sentimientos:**
 - *Ansiedad e incertidumbre:* Al aparecer una falla inesperada en el tablero y no saber qué tan grave o costosa será.
 - *Desconfianza:* Recelo hacia los talleres tradicionales por la falta de transparencia en los diagnósticos y costos.
 - *Tranquilidad:* Sensación de alivio y control cuando cuenta con información clara, digital y accesible desde su teléfono.

PAINS

- No comprende fácilmente los términos técnicos utilizados por los mecánicos.
- Tiene dudas constantes sobre si algunas reparaciones mecánicas realmente son necesarias.
- Le preocupa recibir presupuestos elevados o cobros inesperados por parte del taller.
- Olvida con facilidad cuándo corresponde realizar el próximo mantenimiento de su vehículo.
- Suele prestar atención al auto únicamente cuando aparece una alerta o un problema evidente en el camino.
- Desconoce el nivel de urgencia exacto de una falla al encenderse un indicador en el tablero.

GAINS

- Mantener su vehículo en óptimas condiciones, seguro y operativo para sus desplazamientos diarios.
- Evitar averías imprevistas y quedarse varado durante sus rutas de trabajo o fines de semana.
- Recibir recordatorios automáticos de mantenimiento basados en el kilometraje real de su auto.
- Obtener información clara, transparente y fácil de entender sobre el estado de su vehículo sin enredos técnicos.
- Saber con precisión cuándo debe llevar el auto al taller y qué nivel de prioridad tiene la falla.
- Tener total confianza en las recomendaciones y presupuestos transparentes emitidos por el mecánico.

4.What do they SAY?

- “
- «Si sé qué tiene el auto y qué tan urgente es, puedo solucionarlo antes de que se convierta en un problema mayor».
 - «No entiendo los términos técnicos complejos que usan los mecánicos cuando revisan el motor».
 - «Quiero saber si las reparaciones que me proponen son realmente necesarias antes de aprobar un presupuesto elevado».
 - «Prefiero usar el celular para coordinar o pagar rápido en lugar de perder tiempo llamando al taller».
- ”

Cuadrante	Insights
Quién	Analista de operaciones en Los Olivos; usa Waze, WhatsApp, Yape y Plin; busca evitar gastos innecesarios
Necesita hacer	Monitorear el auto, entender alertas del tablero, saber urgencia real y agendar sin llamadas largas
Ve	Luces de advertencia inesperadas; mercado fragmentado de talleres; riesgo de quedarse en vía en Lima Norte
Dice	«Si sé qué tiene y qué tan urgente es, lo soluciono a tiempo»; «No entiendo la jerga del mecánico»; «Quiero saber si la reparación es necesaria»; «Prefiero coordinar y pagar por el celular»
Oye	Historias de cobros excesivos; consejos familiares de preventivo; recomendaciones de apps digitales
Piensa / siente	Ansiedad ante luces del tablero; desconfianza por falta de transparencia; alivio cuando tiene info clara en el móvil
Pains	Jerga técnica; duda de necesidad de reparaciones; costos ocultos; olvido de km; no conoce urgencia exacta
Gains	Auto seguro; menos emergencias; recordatorios por kilometraje; info no técnica; confianza en presupuestos

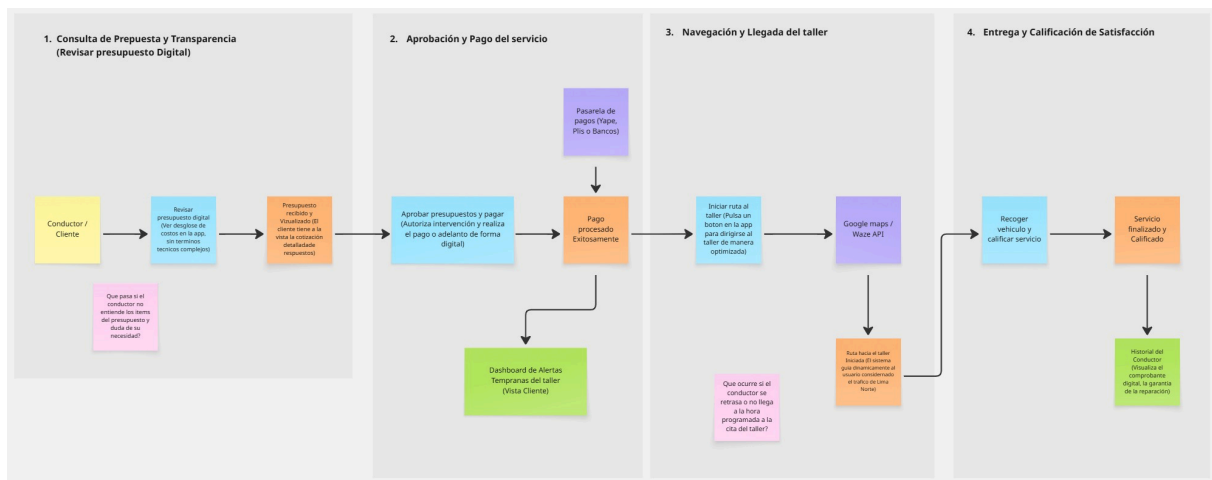
Consideración ética (curso): las alertas móviles deben informar con precisión, evitar presión comercial engañosa, respetar privacidad de telemetría/ubicación y exponer términos claros de uso y tratamiento de datos en la app.

2.3.5. Big Picture EventStorming

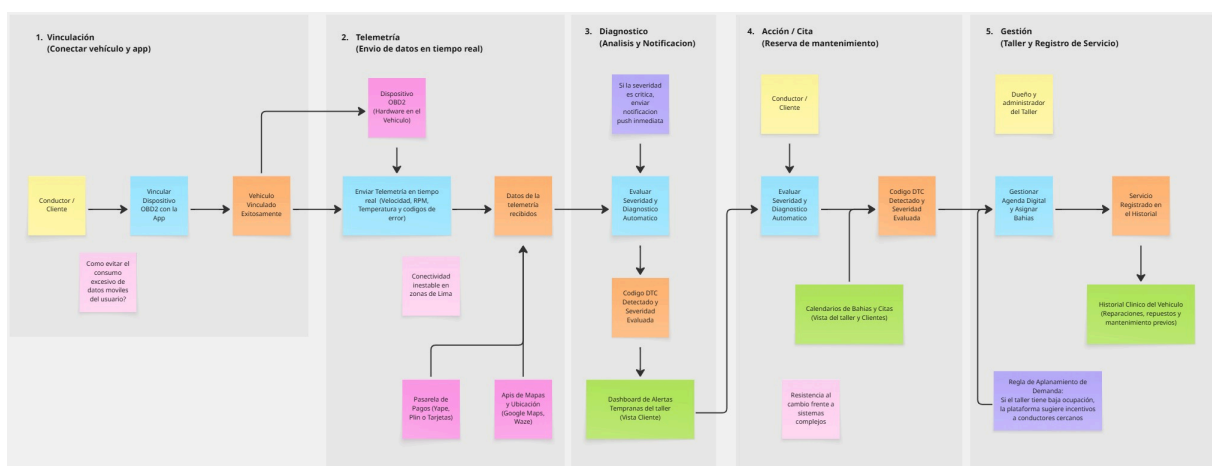
El *Big Picture EventStorming* es una técnica colaborativa que ordena en una línea de tiempo los **eventos de dominio** (hechos en pasado) del negocio automotriz digitalizado por Shiftlq. Permite ver el flujo completo —desde la telemetría hasta la orden de trabajo— e identificar *hotspots* (fricciones, dudas o riesgos) antes de especificar requerimientos móviles.

Figura 14

Big Picture EventStorming — Segmento Objetivo 1 (flujo de servicio: presupuesto, pago, navegación y calificación)


Figura 15

Big Picture EventStorming — Segmento Objetivo 2 (flujo núcleo: vinculación OBD2, telemetría, diagnóstico, cita y gestión del taller)



Flujo macro de eventos de dominio (síntesis)

Tablero 1 — servicio y cierre (Figura 14)

1. **PresupuestoRecibido** — el conductor revisa en la app el desglose de costos sin jerga técnica.
2. **PagoProcesado** — aprueba el presupuesto y paga o adelanta vía pasarela (Yape, Plin o bancos).
3. **RutaAlTallerIniciada** — inicia navegación optimizada con Google Maps / Waze.
4. **ServicioFinalizadoYCalificado** — recoge el vehículo, califica el servicio y consulta historial/comprobante digital.

Tablero 2 — monitoreo y operación (Figura 15)

5. **VehiculoVinculado** — el conductor empareja el dongle OBD2 con la app móvil.
6. **TelemetriaRecibida** — el dispositivo envía velocidad, RPM, temperatura y códigos de error.

7. **CodigoDTCDetectado / SeveridadEvaluada** — diagnóstico automático; si es crítico, push inmediato.
8. **CitaReservada** — el conductor agenda mantenimiento en el calendario de bahías.
9. **AgendaGestionada** — el dueño/administrador del taller asigna bahías y registra el servicio.
10. **HistorialClinicoActualizado** — queda el historial de reparaciones, repuestos y mantenimientos.

Hotspots identificados

- Consumo excesivo de datos móviles al mantener telemetría continua.
- Conectividad inestable en zonas de Lima → necesidad de **cola offline** y sync diferido.
- Resistencia al cambio frente a sistemas de gestión complejos en el taller.
- Duda del conductor si no entiende ítems del presupuesto digital.
- Retrasos o inasistencia a la cita programada.
- Regla de aplanamiento de demanda: incentivos cuando el taller tiene baja ocupación.

2.3.6. Ubiquitous Language

Se define el glosario de términos del dominio Shiftlq / TuxLogic. Este *Ubiquitous Language* evita ambigüedades entre negocio, UX móvil y desarrollo (nativo o Flutter), y debe usarse de forma consistente en código, pantallas, historias de usuario y documentación.

Tabla 9

Ubiquitous Language — ecosistema Shiftlq

Término (EN)	Equivalente (ES)	Definición	Segmentos
Workshop Owner	Dueño del taller	Responsable de decisiones administrativas y operativas del taller independiente.	Taller
Workshop Admin	Administrador de taller	Usuario que gestiona agenda, órdenes y alertas	Taller

Término (EN)	Equivalente (ES)	Definición	Segmentos
		desde la app/web del taller.	
Mechanic	Mecánico	Técnico que ejecuta el servicio físico sobre el vehículo.	Taller / Operaciones
Driver	Conductor	Propietario o usuario habitual del vehículo que recibe alertas y agenda citas en la app móvil.	Conductor
OBD2 Dongle / Scanner	Dispositivo / escáner OBD2	Hardware IoT conectado al puerto de diagnóstico del vehículo para capturar telemetría.	Telemetría
Telemetry	Telemetría	Datos técnicos transmitidos desde el vehículo (sensores, esta-	App móvil / Backend

Término (EN)	Equivalente (ES)	Definición	Segmentos
		do, códigos).	
DTC	Código de falla (DTC)	Código alfanumérico estandarizado que identifica una falla detectada por la ECU.	Diagnóstico
Preventive Alert	Alerta preventiva	Notificación push proactiva basada en telemetría/ DTC, expresada en lenguaje natural.	Conductor / Taller
Severity Level	Nivel de severidad	Clasificación Bajo / Medio / Crítico asociada a una alerta.	UX móvil
Work Order	Orden de trabajo	Registro digital del servicio: hallazgos, repuestos, estado y responsables.	Taller

Término (EN)	Equivalente (ES)	Definición	Segmentos
Appointment	Cita / agendamiento	Reserva de franja horaria para mantenimiento o reparación, creada desde la app.	Ambos
Vehicle Health	Salud del vehículo	Representación simplificada del estado general mostrada al conductor en la app.	Conductor
Predictive Maintenance	Mantenimiento predictivo	Intervención anticipada basada en datos antes de una avería crítica.	Dominio
Service History	Historial de servicio	Línea de tiempo de intervenciones y lecturas asociadas a un vehículo.	Ambos
Offline Sync	Sincronización offline	Persistencia lo-	App móvil

Término (EN)	Equivalente (ES)	Definición	Segmentos
		cal en el dispositivo móvil y envío diferido al restablecer conectividad.	
Consent / Privacy	Consentimiento / privacidad	Autorización explícita del usuario para recolectar y tratar telemetría y datos personales.	Ética / Términos

Este lenguaje ubicuo alimenta las siguientes etapas del informe (especificación de requerimientos, DDD estratégico/táctico y diseño de la experiencia móvil), asegurando que taller y conductor hablen el mismo vocabulario que el equipo de ingeniería de TuxLogic.

2.4. Requirements Specification

2.4.1. User Stories

2.4.2. Impact Mapping

2.4.3. Product Backlog

2.5. Strategic-Level Domain-Driven Design

2.5.1. EventStorming

Para el diseño estratégico de ShiftIQ se empleó **EventStorming** como técnica para explorar y estructurar el dominio del sistema a partir de los acontecimientos relevantes del negocio. Esta

actividad permitió pasar de una visión general del dominio hacia una identificación progresiva de eventos, causas, procesos y límites de responsabilidad.

El proceso se desarrolló tomando como punto de partida el **Big Picture EventStorming** realizado anteriormente en la etapa de Needfinding. En dicho análisis ya se habían identificado acontecimientos relacionados con la vinculación del vehículo, recepción de telemetría, detección de códigos DTC, evaluación de severidad, reserva de citas, gestión del taller y actualización del historial de servicio.

Para profundizar este análisis en el nivel estratégico de DDD, el EventStorming se organizó en tres actividades: **Collect Domain Events**, **Refine Domain** y **Track Causes (Process Modeling)**. Los resultados de estas actividades fueron utilizados posteriormente para identificar los **Candidate Bounded Contexts**, establecer sus interacciones mediante **Domain Message Flows** y formalizar cada contexto mediante **Bounded Context Canvases**.

Collect Domain Events

En la primera actividad se recopilaron los principales **Domain Events** asociados con las actividades del dominio automotriz de Shiftlq. El objetivo fue obtener una visión amplia de los hechos relevantes que ocurren durante los diferentes procesos del sistema, sin establecer inicialmente límites entre contextos.



Figura 1: Collect Domain Events — EventStorming parte 1

Figura 16. Collect Domain Events — Identificación de eventos de dominio de Shiftlq.

La actividad permitió reunir acontecimientos relacionados con identidad, gestión del taller, vehículos, telemetría, diagnóstico, citas, órdenes de trabajo, inventario, cotizaciones y pagos. Esta recopilación constituye la materia prima para el posterior refinamiento del dominio.

Refine Domain

A continuación, los eventos recopilados fueron revisados y refinados para distinguir los acontecimientos relevantes del negocio de acciones, datos o elementos que no representan hechos de dominio. También se consolidaron eventos relacionados y se estableció una nomenclatura orientada al lenguaje del dominio.

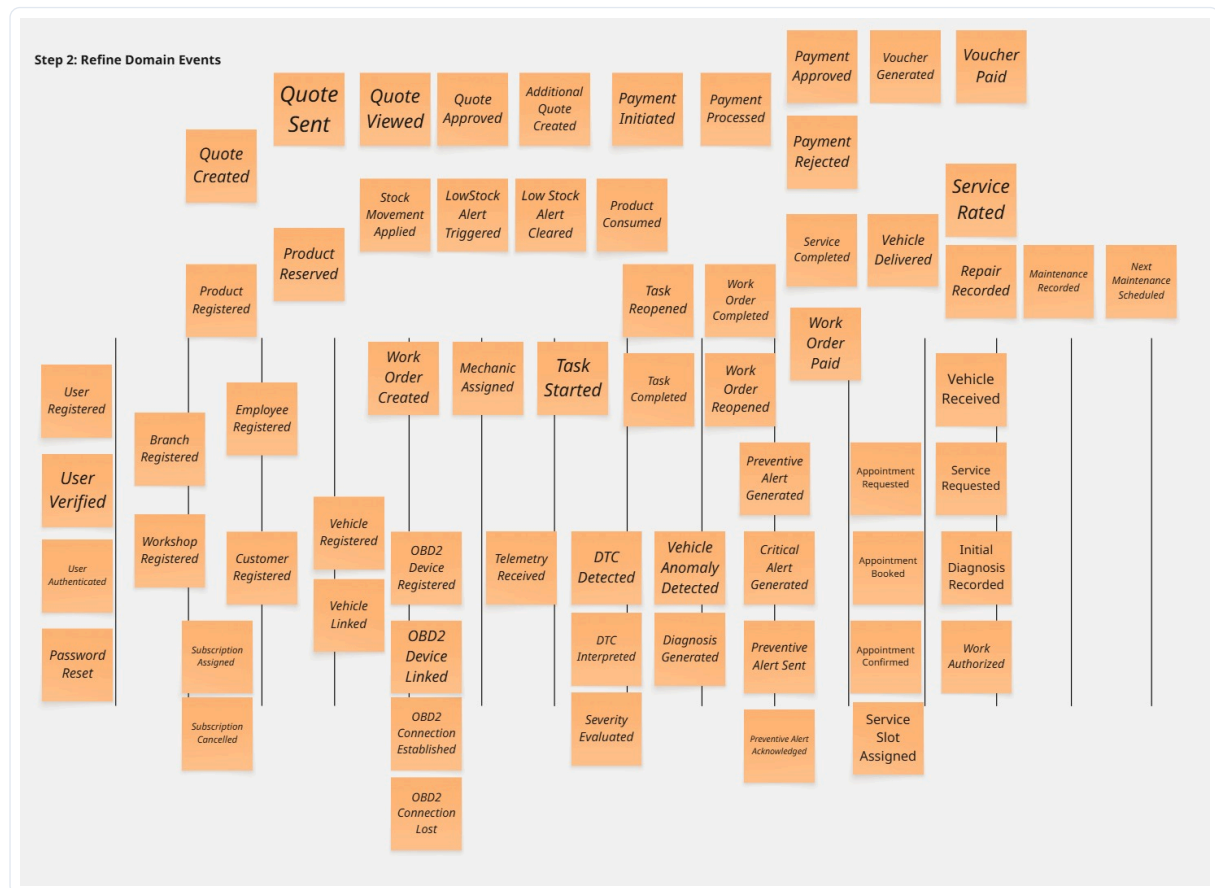


Figura 2: Refine Domain — EventStorming parte 2

Figura 17. Refine Domain — Refinamiento y organización de los eventos del dominio.

El refinamiento permitió obtener un conjunto de eventos más consistente y representativo de los procesos principales de ShiftIq. Esta etapa es importante porque proporciona una base común para posteriormente identificar agrupaciones de responsabilidades.

Track Causes (Process Modelling)

Finalmente, se realizó el seguimiento de las causas y consecuencias de los eventos mediante **Process Modelling**. En esta actividad se relacionaron los elementos del EventStorming, principalmente **Actor**, **Command**, **Event**, **Business Process/Policy**, **Aggregate**, **External System** y **Hotspot**.



Figura 3: Track Causes (Process Modelling) — EventStorming parte 3

Figura 18. Track Causes (Process Modelling) — Relación causal de comandos, eventos y procesos.

El modelado permitió comprender cómo una acción ejecutada por un actor genera un comando que modifica el estado de un agregado y produce un evento. Asimismo, permitió identificar interacciones con sistemas externos y puntos de incertidumbre o riesgo (*hotspots*).

Entre los principales problemas identificados previamente se encuentran la conectividad inestable, el consumo de datos de telemetría, los retrasos o inasistencias a las citas, la incertidumbre respecto a los presupuestos y la necesidad de simplificar la gestión del taller.

El resultado de estas tres actividades constituyó la base para la siguiente etapa: **Candidate Context Discovery**.

2.5.1.1. Candidate Context Discovery

El **Candidate Context Discovery** tuvo como objetivo identificar agrupaciones coherentes de responsabilidades dentro del dominio. Para ello se analizaron conjuntamente los eventos refinados, comandos, actores, agregados, modelos de lectura, políticas y relaciones identificadas durante el EventStorming.

Es importante señalar que los **Bounded Contexts candidatos no fueron definidos únicamente a partir de los nombres de los paquetes del backend**. Primero se identificaron las agrupaciones funcionales que emergieron del comportamiento del dominio y posteriormente se contrastaron con la estructura existente del sistema.

Como resultado se reconocieron **siete Candidate Bounded Contexts**:

1. **Identity & Access**
2. **Workshop Management**
3. **Fleet & Appointments**
4. **Vehicle Intelligence & Diagnostics**
5. **Service Operations**
6. **Inventory Management**
7. **Billing & Payments**

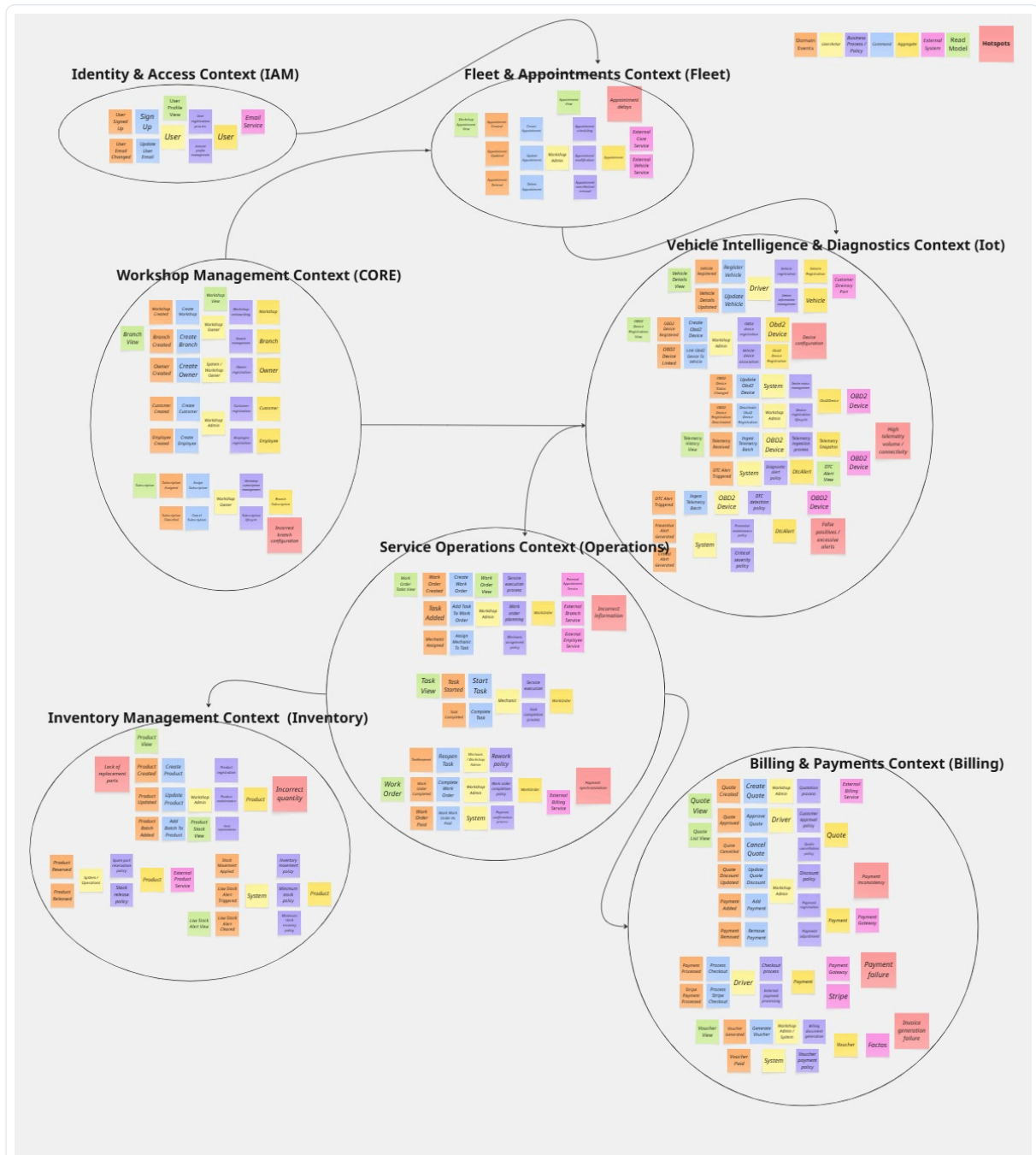


Figura 4: Candidate Context Discovery — Reconocimiento de los Candidate Bounded Contexts

Figura 19. Candidate Context Discovery — Reconocimiento de los Candidate Bounded Contexts de ShiftIQ.

A continuación, se describe la responsabilidad de cada contexto identificado.

1. Identity & Access

Este contexto concentra las responsabilidades relacionadas con la identidad y el acceso de los usuarios. Incluye procesos de registro, autenticación, recuperación de contraseña y actualización de credenciales.

Su correspondencia con el backend se evidencia en el módulo `iam`, donde se encuentra el agregado `User`, comandos como `SignUpCommand`, `SignInCommand`, `ResetPasswordCommand` y comandos para actualizar las credenciales. También existen servicios relacionados con correo, hashing y tokens.

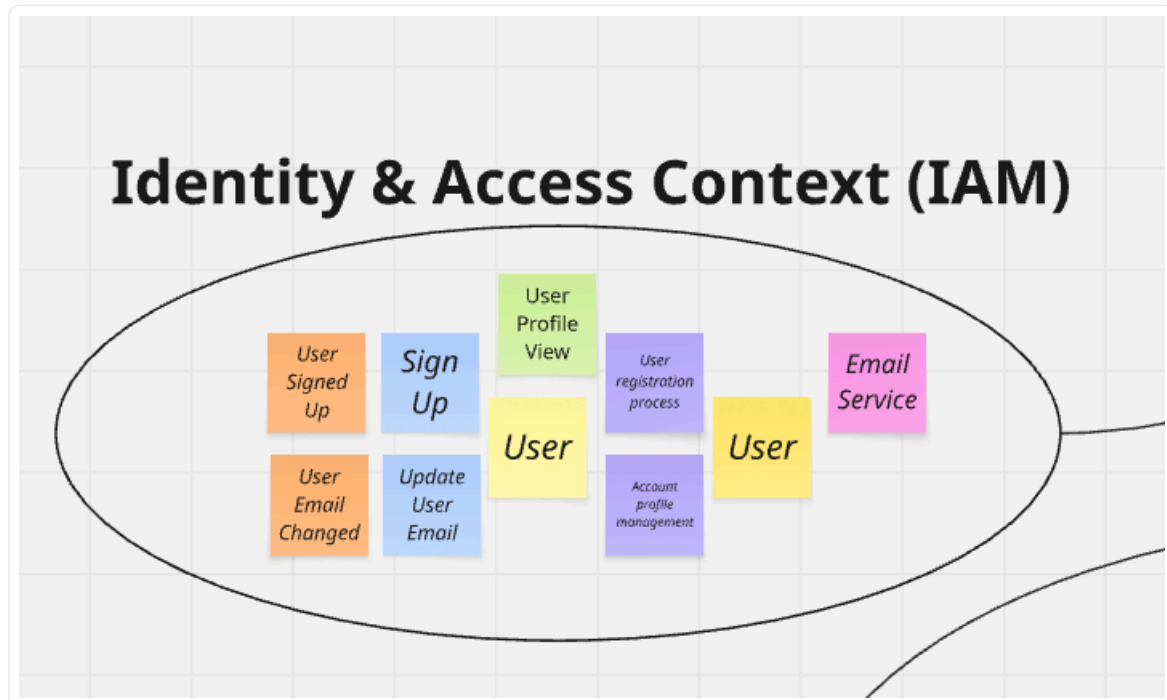


Figura 5: Candidate Bounded Context — Identity & Access

Figura 20. Candidate Bounded Context — Identity & Access.

2. Workshop Management

Este contexto gestiona la información organizacional del taller, incluyendo talleres, sucursales, propietarios, clientes y empleados.

La correspondencia con el backend se encuentra en el módulo `core`, donde existen eventos como `WorkshopCreatedEvent`, `BranchCreatedEvent`, `CustomerCreatedEvent`, `EmployeeCreatedEvent` y `OwnerCreatedEvent`, además de sus respectivas consultas.

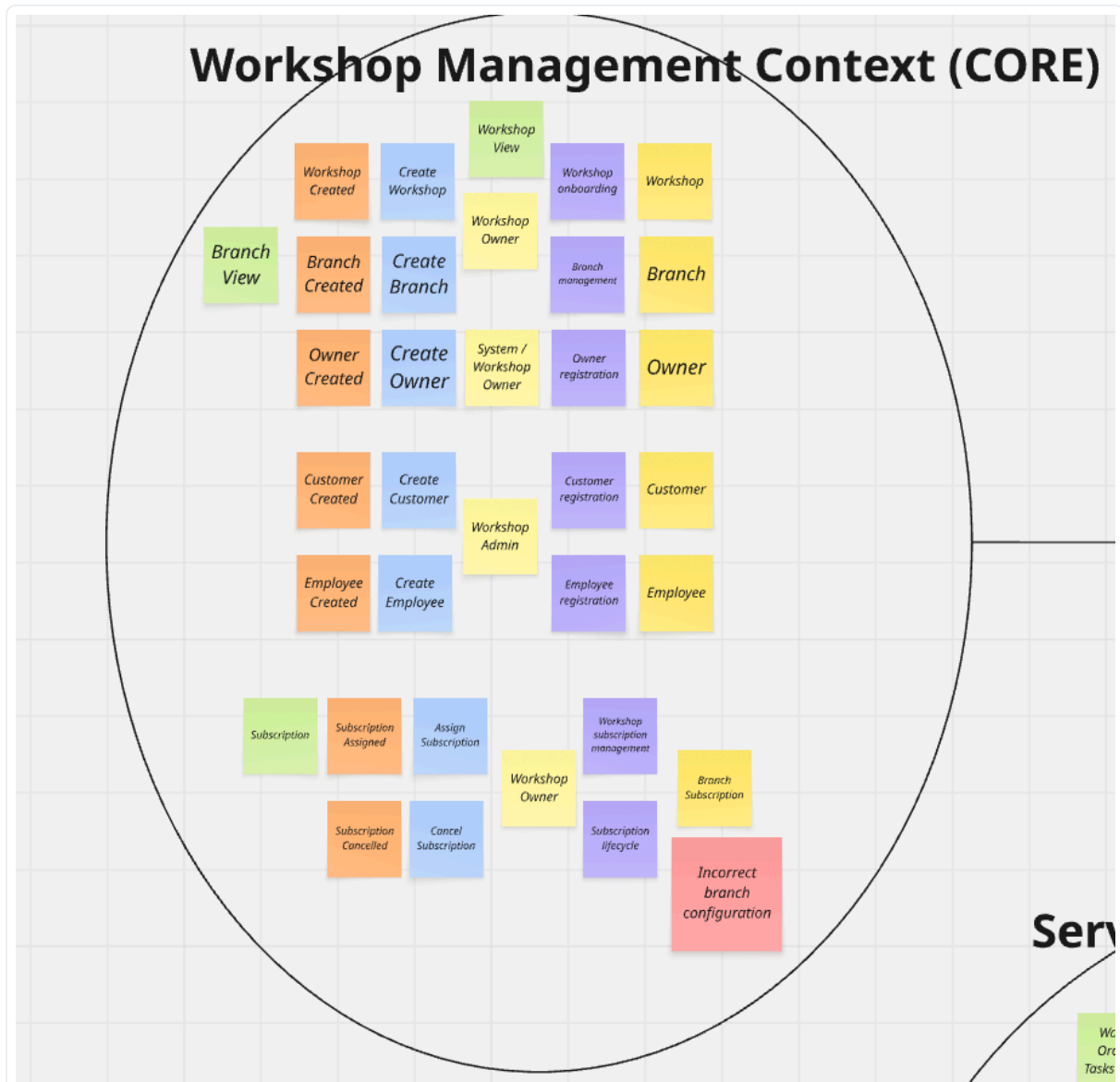


Figura 6: Candidate Bounded Context — Workshop Management

Figura 21. Candidate Bounded Context — Workshop Management.

3. Fleet & Appointments

Este contexto concentra la gestión de citas y registros asociados a clientes y empleados, funcionando como punto de coordinación entre la planificación del servicio y las operaciones del taller.

El backend contiene Appointment y comandos relacionados con su creación, actualización y eliminación, además de AppointmentCreatedEvent. También dispone de recursos y servicios de consulta asociados.

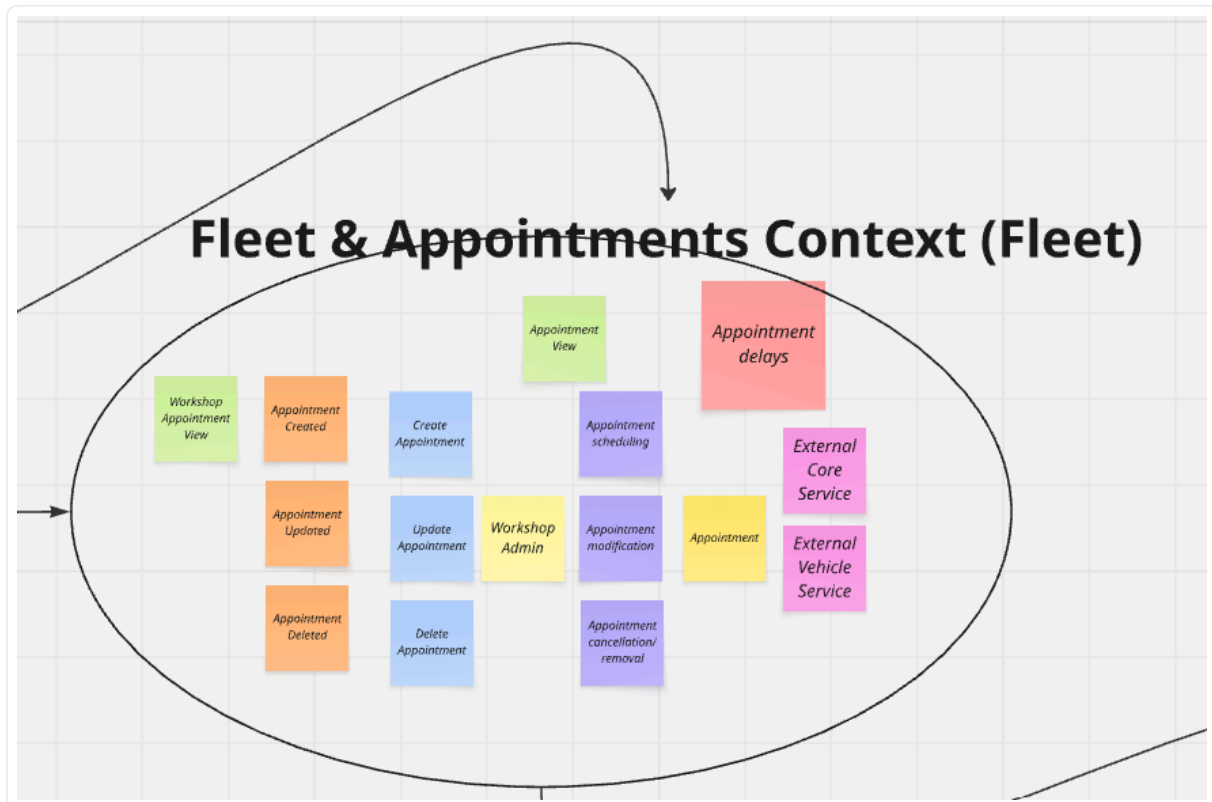


Figura 7: Candidate Bounded Context — Fleet & Appointments

Figura 22. Candidate Bounded Context — Fleet & Appointments.

4. Vehicle Intelligence & Diagnostics

Este contexto concentra las capacidades relacionadas con el vehículo, dispositivos OBD2, telemetría y diagnóstico. Su propósito dentro del dominio es transformar los datos técnicos del vehículo en información útil para el seguimiento de su condición.

Esta agrupación se relaciona con el flujo identificado previamente de vehículo vinculado, telemetría recibida, detección de DTC y evaluación de severidad.

Además, el lenguaje ubicuo del proyecto incorpora explícitamente términos como **Vehicle**, **OBD2 Dongle**, **Telemetry**, **DTC**, **Preventive Alert** y **Severity Level**.

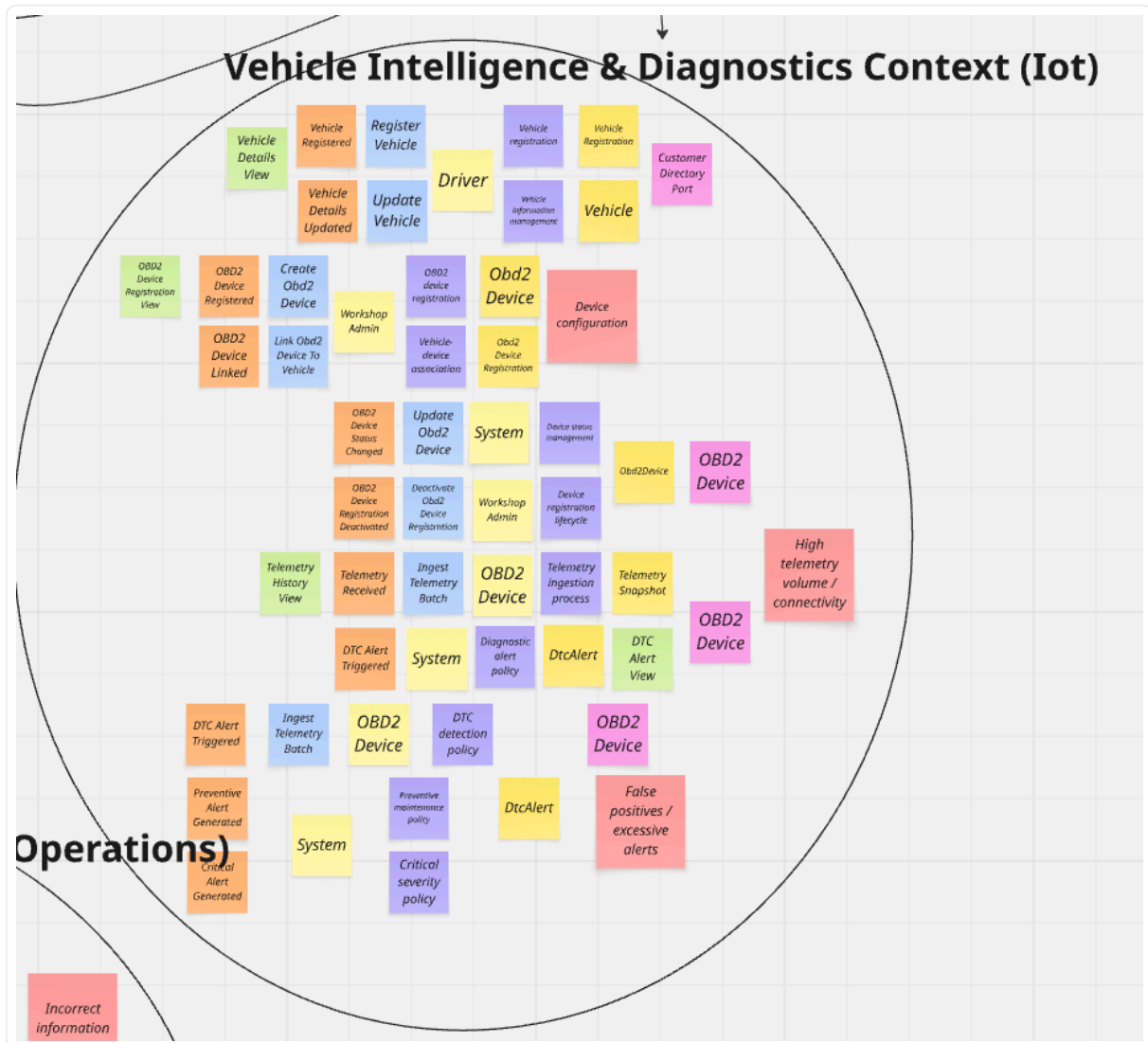


Figura 8: Candidate Bounded Context — Vehicle Intelligence & Diagnostics

Figura 23. Candidate Bounded Context — Vehicle Intelligence & Diagnostics.

5. Service Operations

Este contexto gestiona la ejecución de los servicios mediante órdenes de trabajo, tareas y asignación de mecánicos.

La estructura del backend contiene comandos para crear órdenes de trabajo, agregar tareas, asignar mecánicos, iniciar y completar tareas, así como eventos como TaskStartedEvent, TaskCompletedEvent, TaskReopenedEvent y WorkOrderCompletedEvent.



Figura 9: Candidate Bounded Context — Service Operations

Figura 24. Candidate Bounded Context — Service Operations.

6. Inventory Management

Este contexto administra los productos y existencias necesarias para la ejecución de los servicios. Incluye productos, lotes, reservas, liberaciones, movimientos de stock y evaluación de niveles mínimos.

La estructura contiene el agregado Product, la entidad ProductBatch, comandos de productos y eventos como ProductCreatedEvent, StockReservedEvent, StockReleasedEvent, StockMovementAppliedEvent y eventos relacionados con bajo stock.

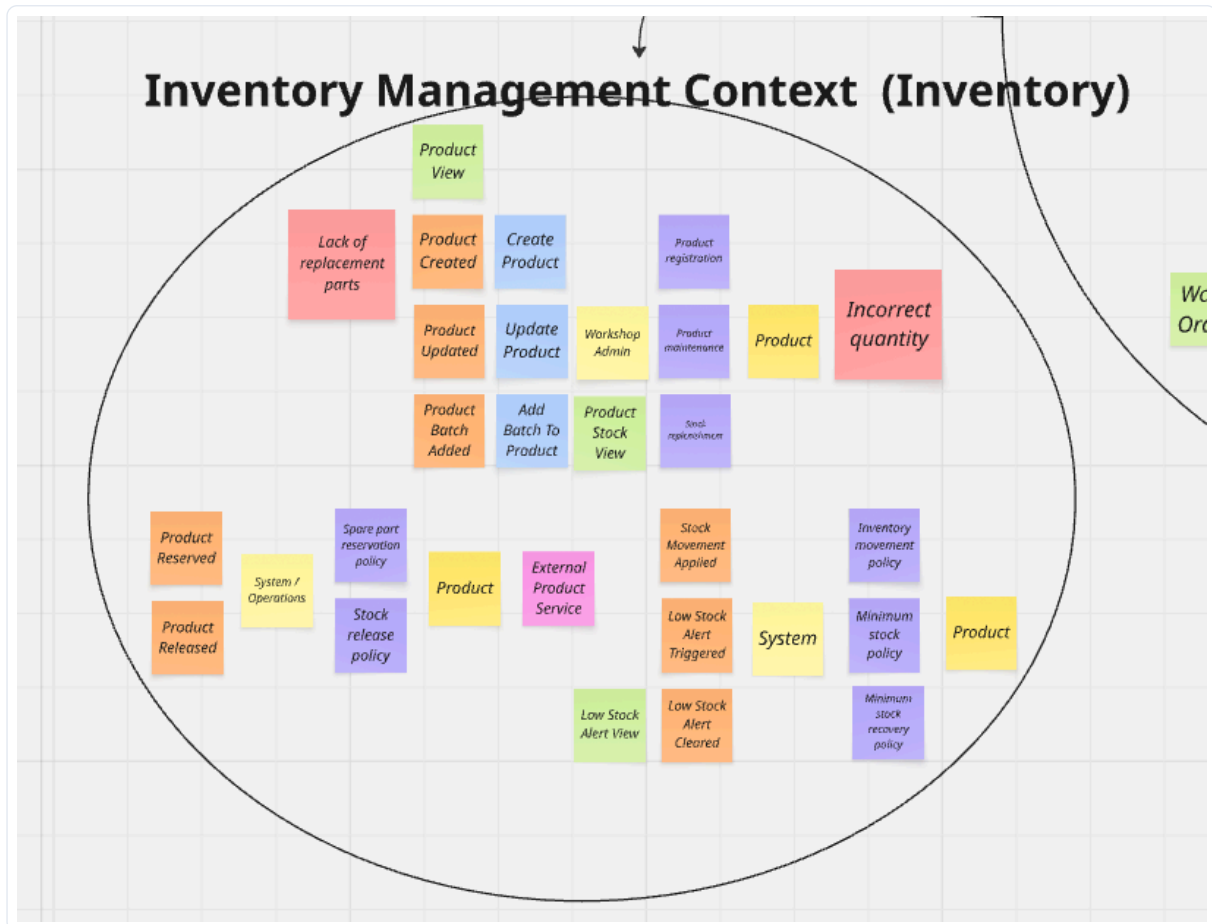


Figura 10: Candidate Bounded Context — Inventory Management

Figura 25. Candidate Bounded Context — Inventory Management.

7. Billing & Payments

Este contexto concentra las responsabilidades relacionadas con cotizaciones, pagos y comprobantes asociados al servicio.

La estructura del backend contiene elementos de cotización y facturación, incluyendo Quote, Voucher, Payment y comandos relacionados con la creación y aprobación de cotizaciones, checkout y procesamiento de pagos.

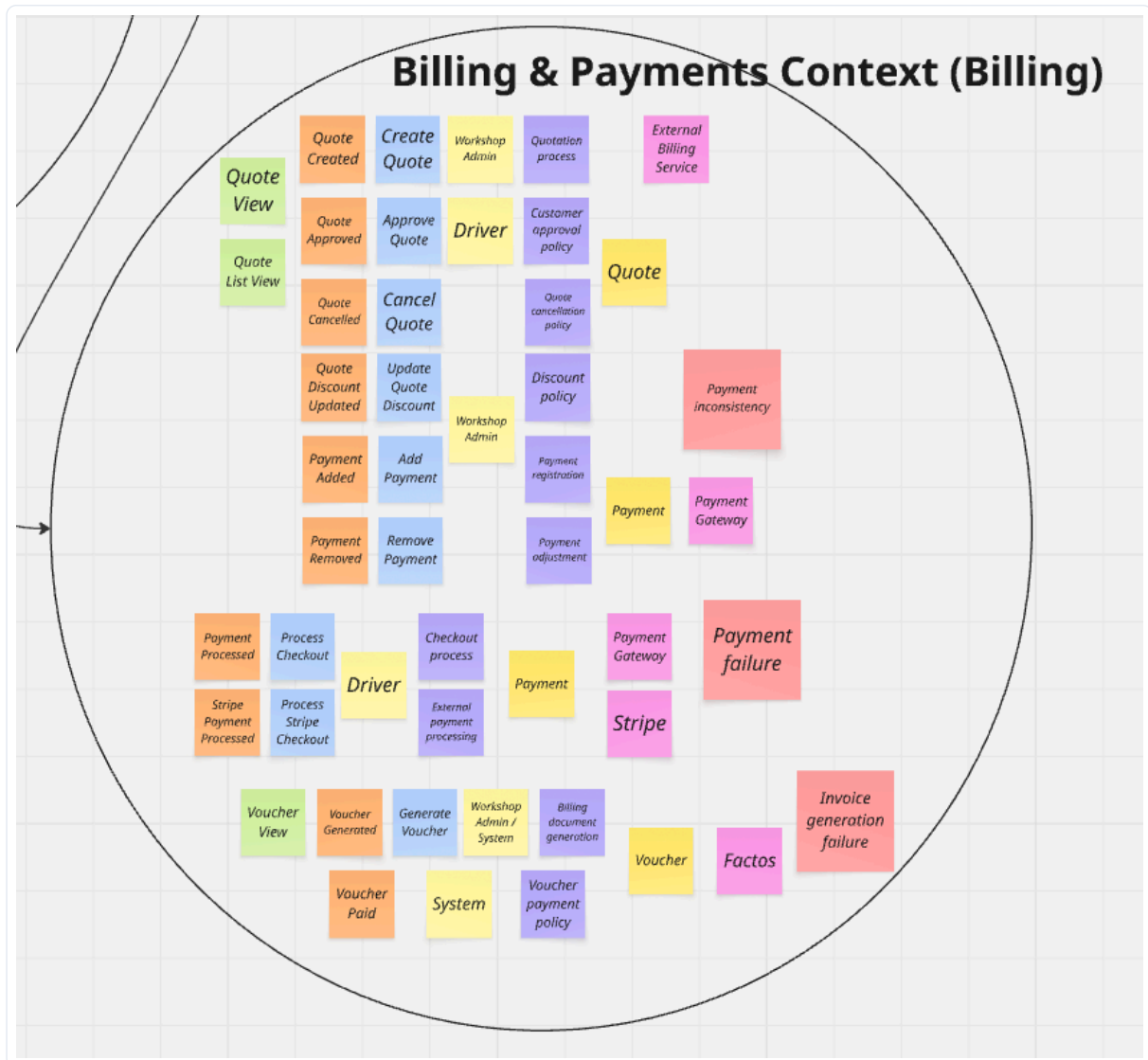


Figura 11: Candidate Bounded Context — Billing & Payments

Figura 26. Candidate Bounded Context — Billing & Payments.

Resultado del Candidate Context Discovery

Como resultado, se obtuvieron **siete agrupaciones de responsabilidad**, que posteriormente fueron utilizadas para construir los flujos de interacción y los Bounded Context Canvases.

Por tanto, esta actividad establece **los límites candidatos del dominio**, pero no constituye todavía la definición definitiva de las relaciones entre ellos. Estas relaciones se desarrollan en el siguiente apartado mediante el **Domain Message Flows Modeling**.

2.5.1.2. Domain Message Flows Modeling

2.5.1.2. Domain Message Flows Modeling

Una vez reconocidos los siete Candidate Bounded Contexts, se realizó el **Domain Message Flows Modeling** con el propósito de representar cómo estos contextos colaboran para ejecutar los principales procesos del dominio.

Los flujos se construyeron utilizando los elementos identificados durante el EventStorming, principalmente **Commands, Events, Actors, Aggregates, Read Models y External Systems**.

Se definieron seis flujos principales.

Flow 1 — Vehicle Intelligence & Diagnostics

Representa el proceso de registro y monitoreo del vehículo, incluyendo la vinculación del dispositivo OBD2, recepción de telemetría y generación de información de diagnóstico.

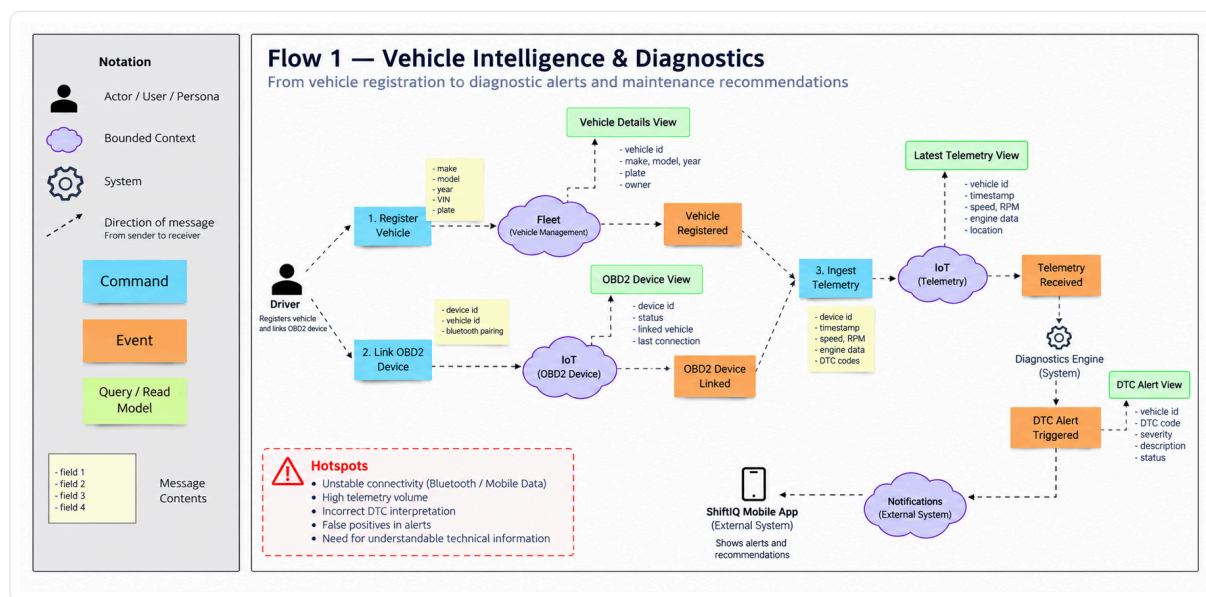


Figura 12: Domain Message Flow — Vehicle Intelligence & Diagnostics

Figura 27. Domain Message Flow — Vehicle Intelligence & Diagnostics.

Este flujo representa una de las capacidades centrales de Shiftlq, ya que conecta la información obtenida del vehículo con el proceso de diagnóstico y seguimiento de su condición.

Flow 2 — Appointment & Service

Representa la transición desde la planificación de una atención hasta la recepción del vehículo para iniciar el servicio.

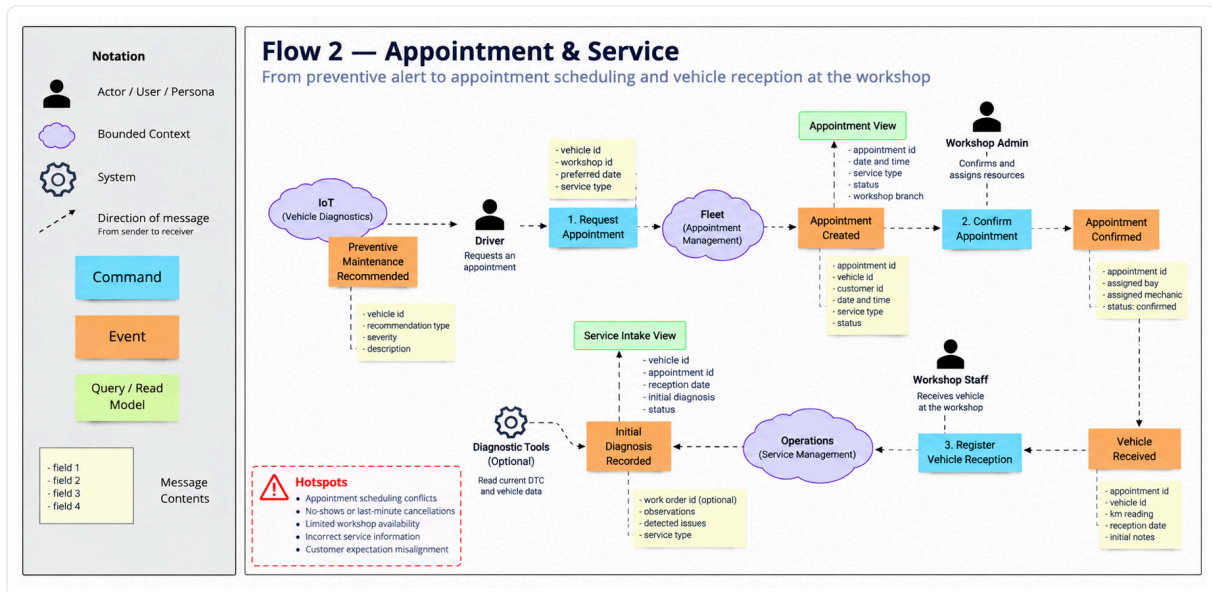


Figura 13: Domain Message Flow — Appointment & Service

Figura 28. Domain Message Flow — Appointment & Service.

Este flujo muestra principalmente la interacción entre **Fleet & Appointments** y **Service Operations**, evidenciando que la cita constituye un punto de conexión entre la planificación y la ejecución del servicio.

Flow 3 — Work Order & Inventory

Representa la ejecución de una orden de trabajo y la utilización de productos necesarios para realizar las tareas del servicio.

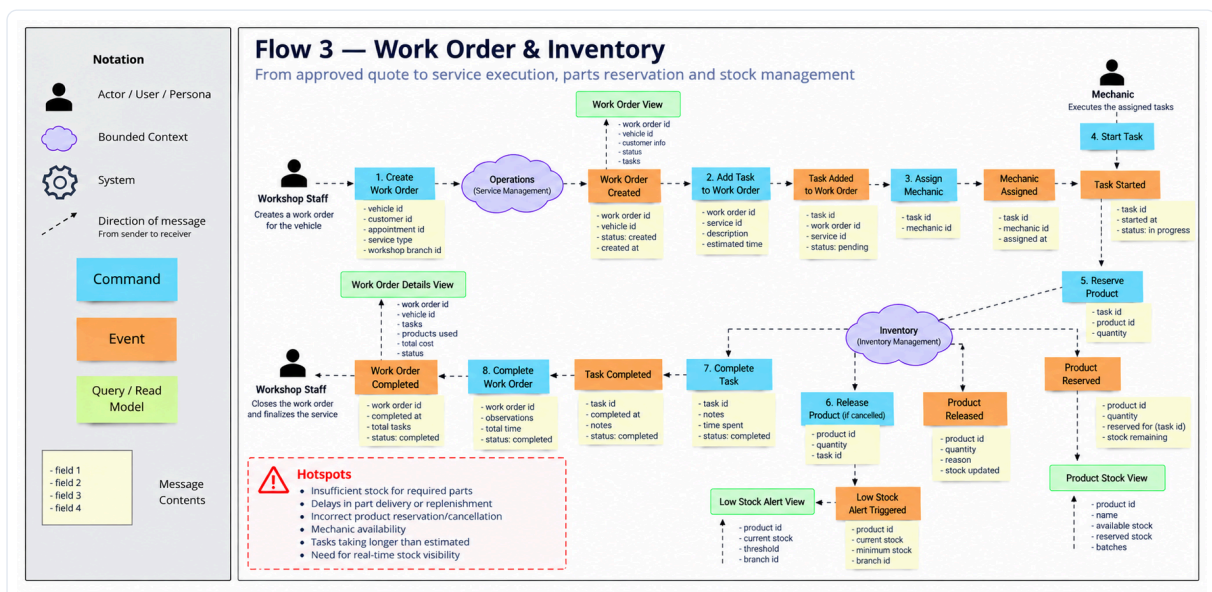


Figura 14: Domain Message Flow — Work Order & Inventory

Figura 29. Domain Message Flow — Work Order & Inventory.

El flujo evidencia la colaboración entre **Service Operations** e **Inventory Management**, particularmente cuando una tarea requiere reservar o utilizar productos.

Flow 4 — Quotation & Payment

Representa el proceso que comienza con la elaboración de una cotización y continúa con su aprobación y procesamiento del pago.

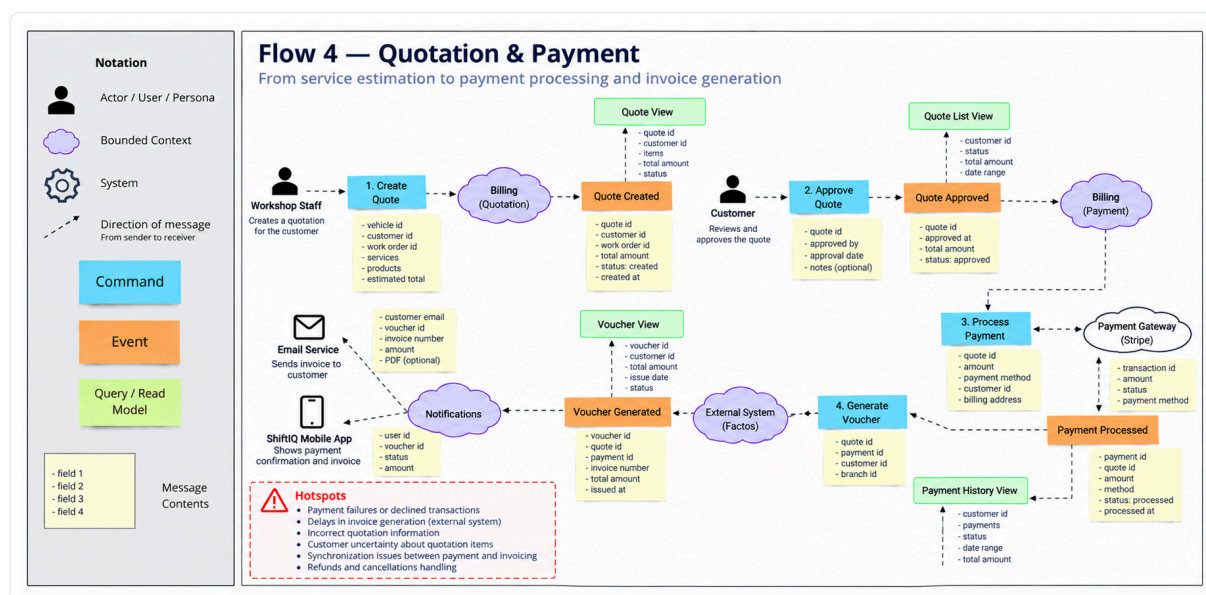


Figura 15: Domain Message Flow — Quotation & Payment

Figura 30. Domain Message Flow — Quotation & Payment.

Este flujo permite representar la colaboración entre el proceso operativo del servicio y **Billing & Payments**, incluyendo la intervención de los mecanismos externos de pago cuando corresponde.

Flow 5 — Workshop Management

Representa las operaciones relacionadas con la configuración y administración de la información del taller, sus sucursales, clientes y empleados.

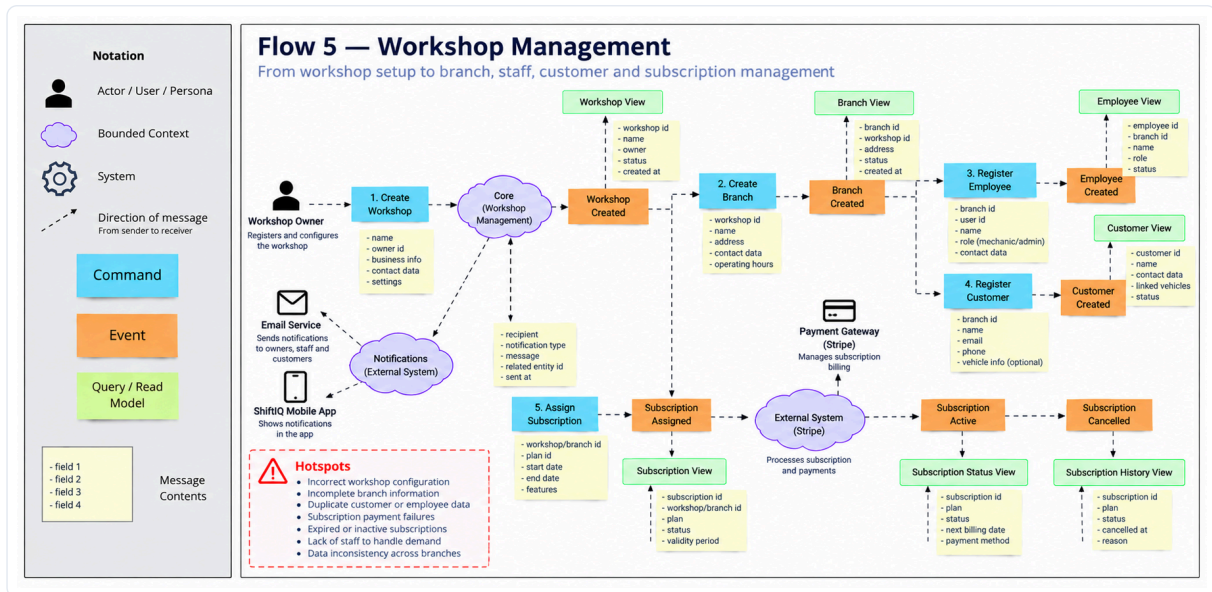


Figura 16: Domain Message Flow — Workshop Management

Figura 31. Domain Message Flow — Workshop Management.

El flujo representa las principales operaciones administrativas que permiten mantener disponible la información necesaria para los demás procesos del sistema.

Flow 6 — Identity & Access

Representa el ciclo de acceso de los usuarios a la plataforma, desde el registro hasta la autenticación y gestión de sus credenciales.

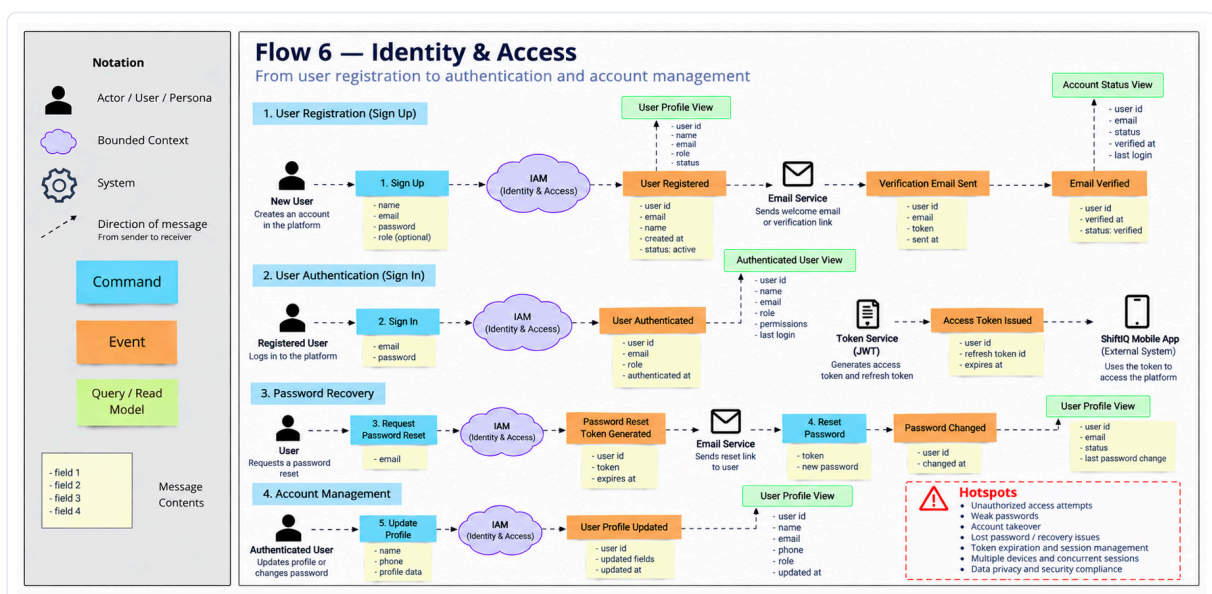


Figura 17: Domain Message Flow — Identity & Access

Figura 32. Domain Message Flow — Identity & Access.

Este flujo delimita las responsabilidades de **Identity & Access** frente a los demás contextos que requieren información sobre el usuario autenticado.

Resultado del Domain Message Flows Modeling

Los seis flujos permiten observar que los Candidate Bounded Contexts **colaboran mediante mensajes**, en lugar de funcionar como componentes completamente aislados.

De esta forma, los flujos muestran las principales interacciones del dominio y permiten comprobar que un mismo proceso de negocio puede atravesar más de un contexto. Por ejemplo, la ejecución de un servicio puede involucrar **Fleet & Appointments, Service Operations, Inventory Management y Billing & Payments**.

Los Domain Message Flows constituyen así el vínculo entre el descubrimiento de los contextos y su formalización mediante los Canvas.

2.5.1.3. Bounded Context Canvases

A partir de los siete Candidate Bounded Contexts identificados y de los seis Domain Message Flows modelados, se elaboraron los **Bounded Context Canvases**.

El Canvas permite describir cada contexto desde una perspectiva estratégica, especificando su propósito, clasificación estratégica, roles de dominio, comunicaciones de entrada y salida, lenguaje ubicuo, decisiones de negocio, supuestos, métricas de verificación y preguntas abiertas.

Los siete Canvas desarrollados son los siguientes:

Canvas 1 — Identity & Access

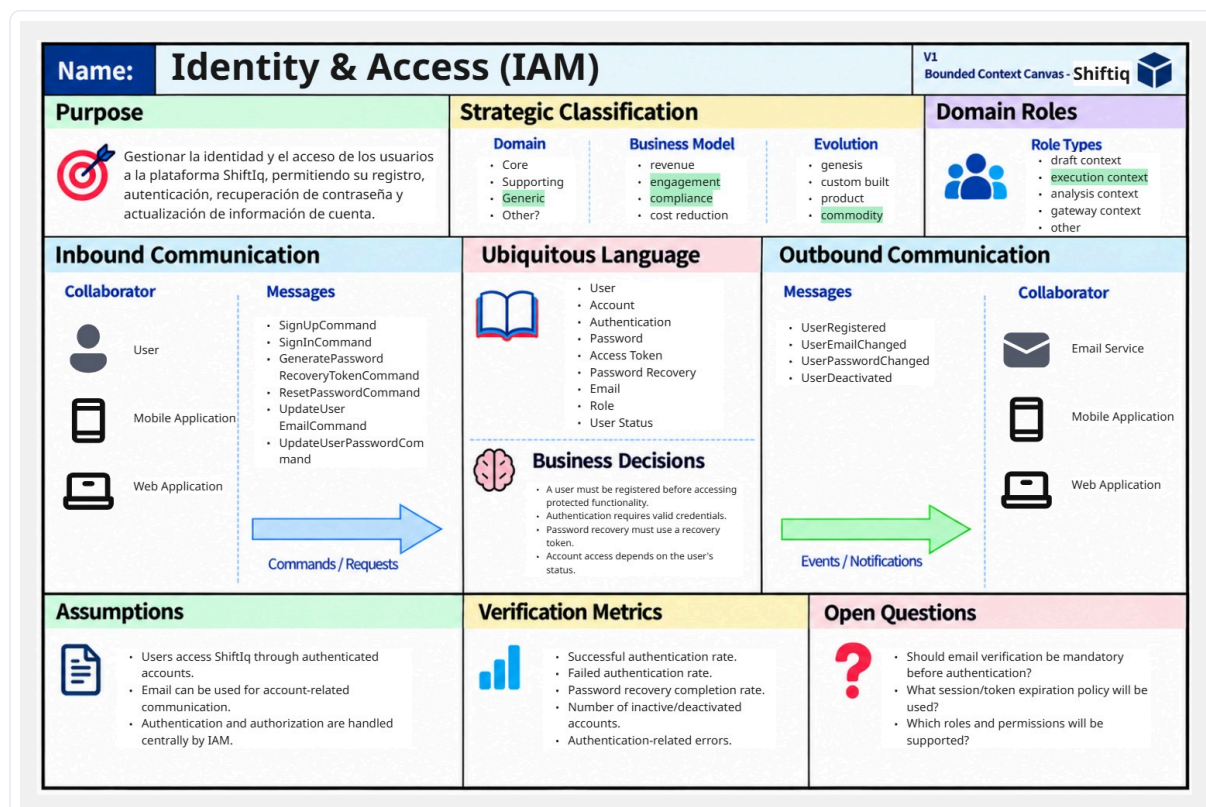


Figura 18: Bounded Context Canvas — Identity & Access

Figura 33. Bounded Context Canvas — Identity & Access.

El Canvas formaliza las responsabilidades del contexto relacionadas con usuarios, autenticación, recuperación de credenciales y acceso a la plataforma.

Canvas 2 — Workshop Management

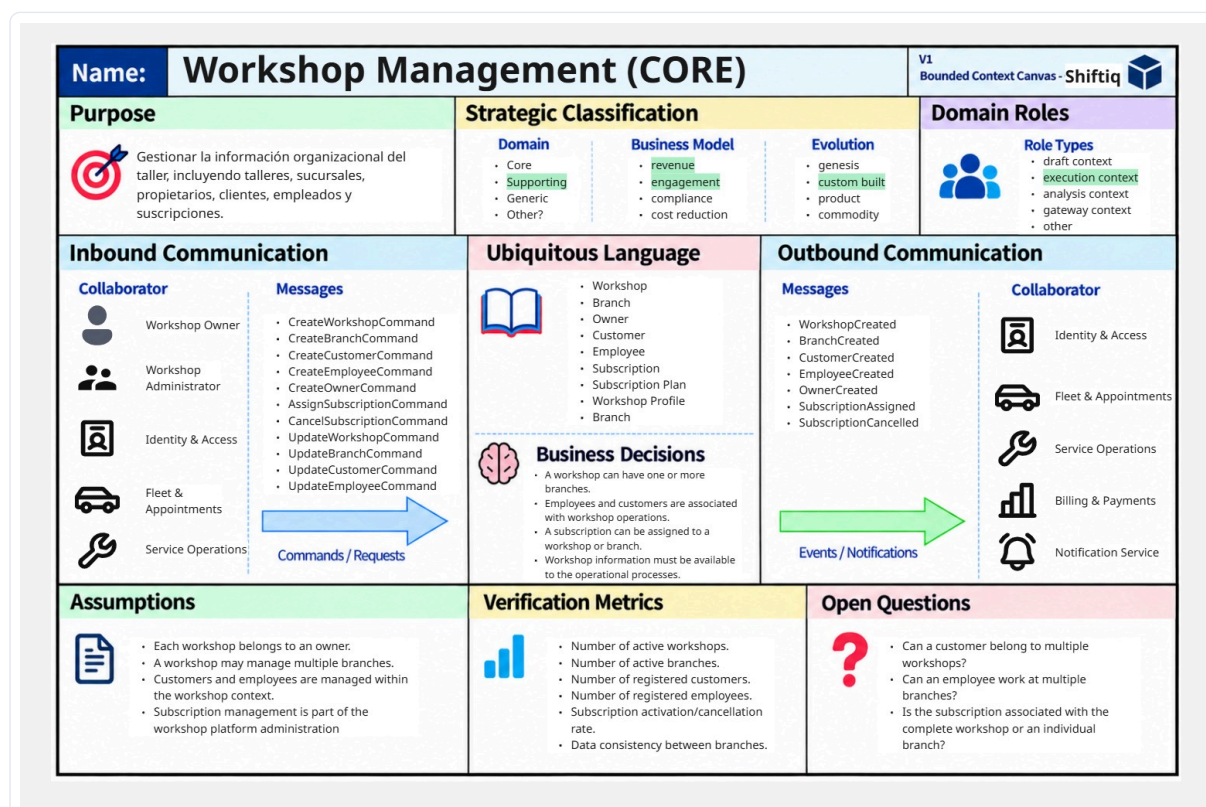


Figura 19: Bounded Context Canvas — Workshop Management

Figura 34. Bounded Context Canvas — Workshop Management.

El Canvas delimita las responsabilidades relacionadas con Workshop, Branch, Owner, Customer y Employee.

Canvas 3 — Fleet & Appointments

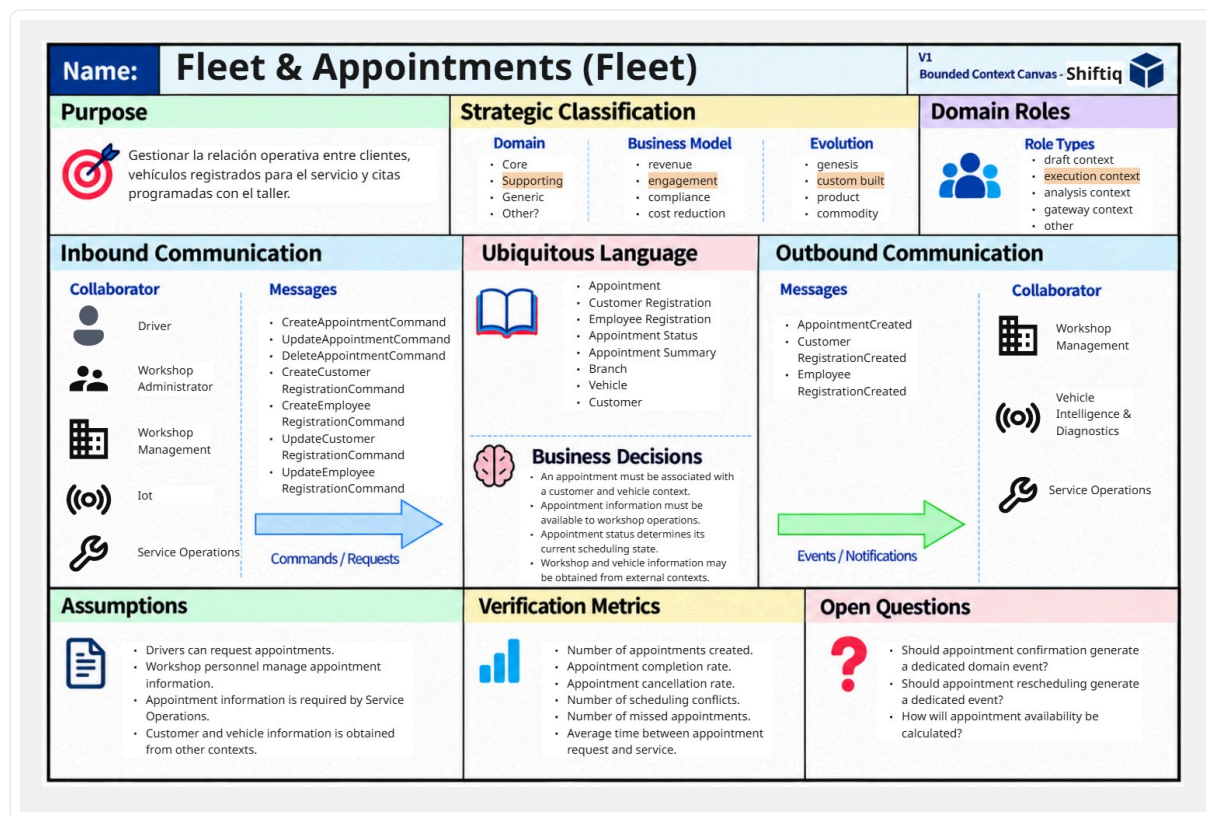


Figura 20: Bounded Context Canvas — Fleet & Appointments

Figura 35. Bounded Context Canvas — Fleet & Appointments.

El Canvas formaliza la responsabilidad de gestionar citas y los registros necesarios para coordinar la atención del vehículo.

Canvas 4 — Vehicle Intelligence & Diagnostics

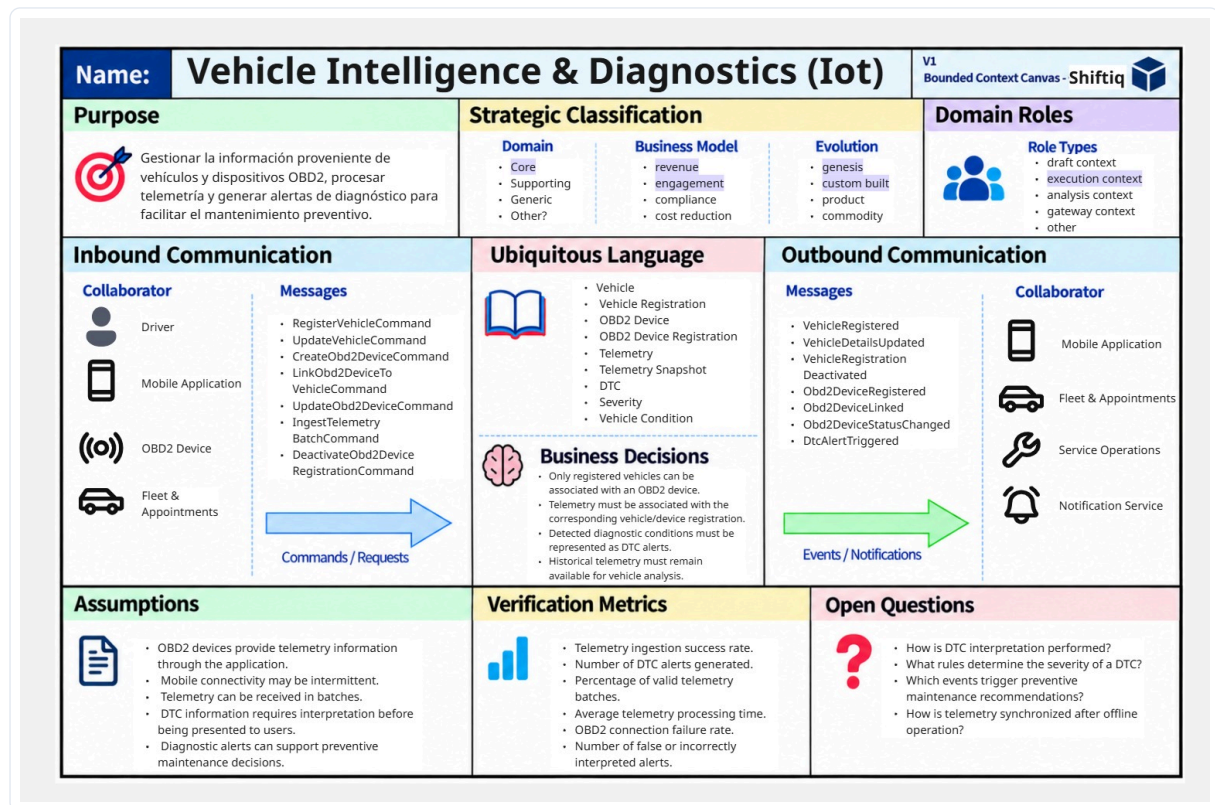


Figura 21: Bounded Context Canvas — Vehicle Intelligence & Diagnostics

Figura 36. Bounded Context Canvas — Vehicle Intelligence & Diagnostics.

El Canvas formaliza las responsabilidades relacionadas con Vehicle, OBD2 Device, Telemetry, DTC y diagnóstico, elementos que forman parte del lenguaje ubicuo del proyecto.

Canvas 5 — Service Operations

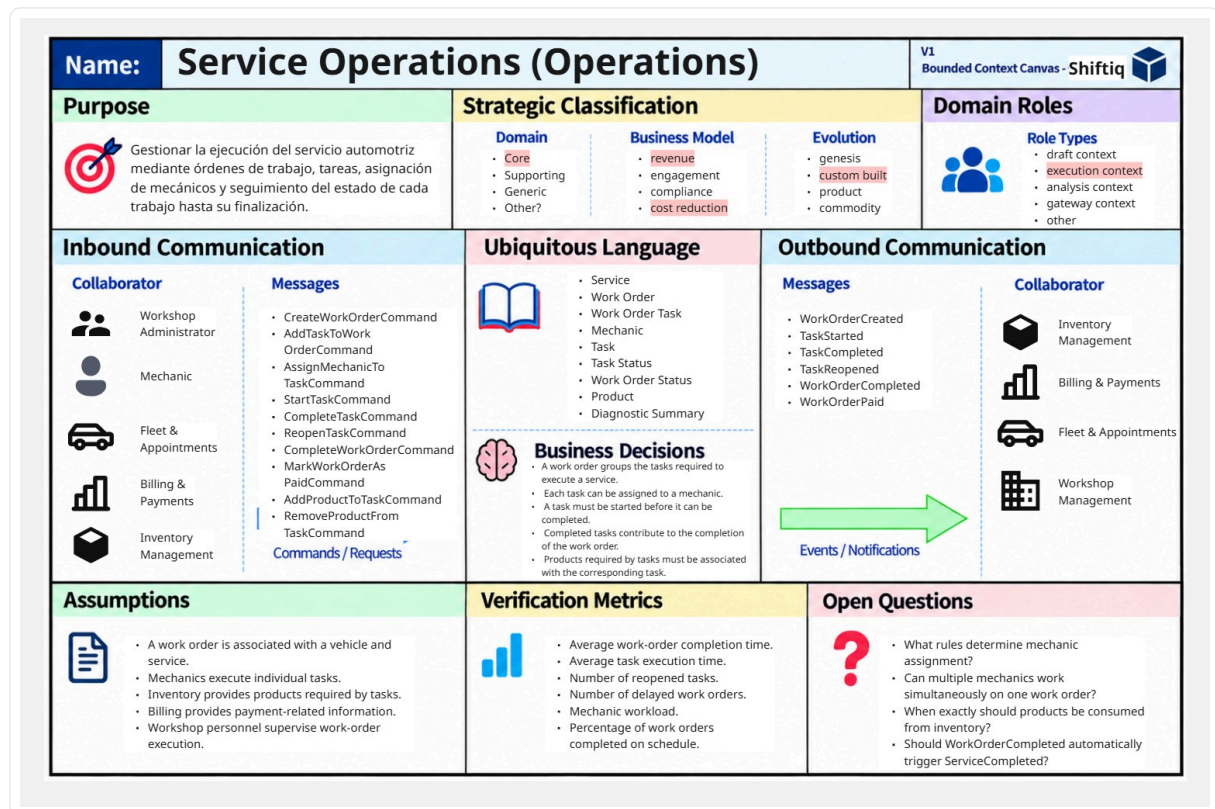


Figura 22: Bounded Context Canvas — Service Operations

Figura 37. Bounded Context Canvas — Service Operations.

El Canvas formaliza las responsabilidades relacionadas con Service, Work Order, Task y Mechanic, incluyendo las reglas asociadas con la ejecución del servicio.

Canvas 6 — Inventory Management

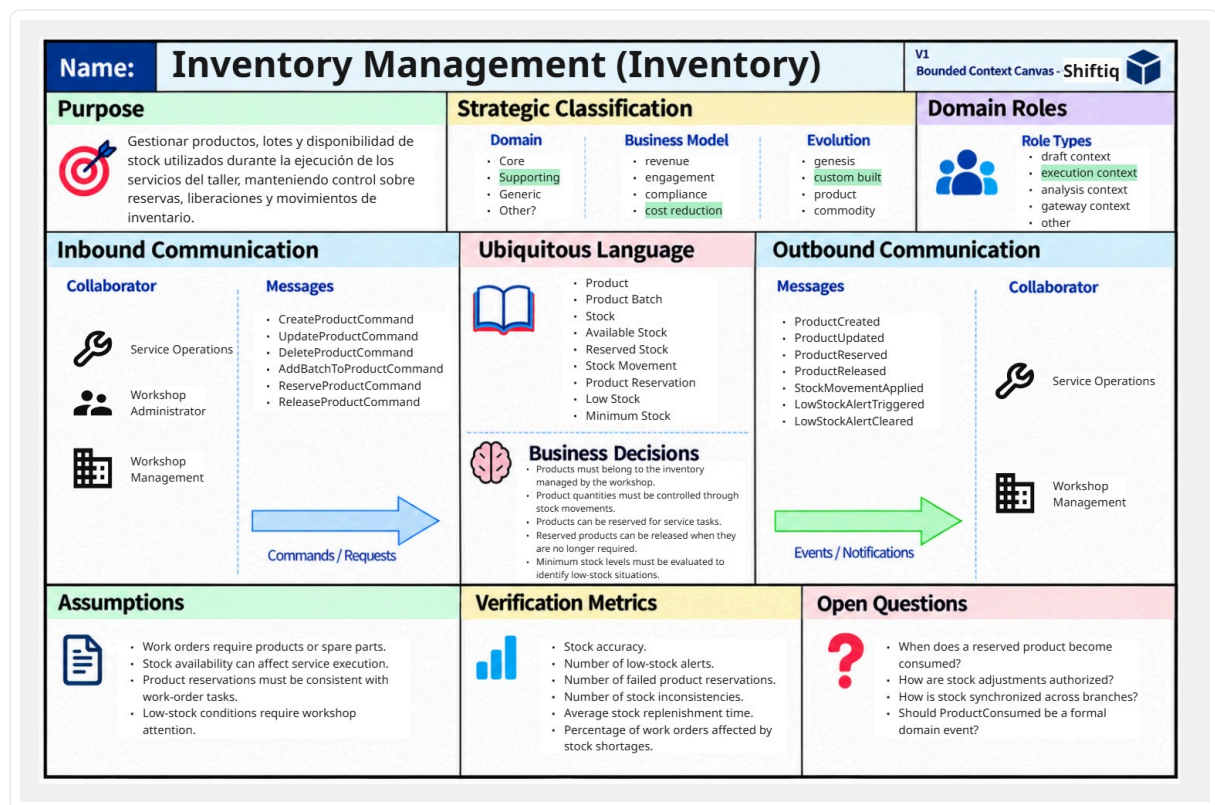


Figura 23: Bounded Context Canvas — Inventory Management

Figura 38. Bounded Context Canvas — Inventory Management.

El Canvas formaliza las responsabilidades relacionadas con Product, Product Batch, Stock, Reservation y Low Stock.

Canvas 7 — Billing & Payments

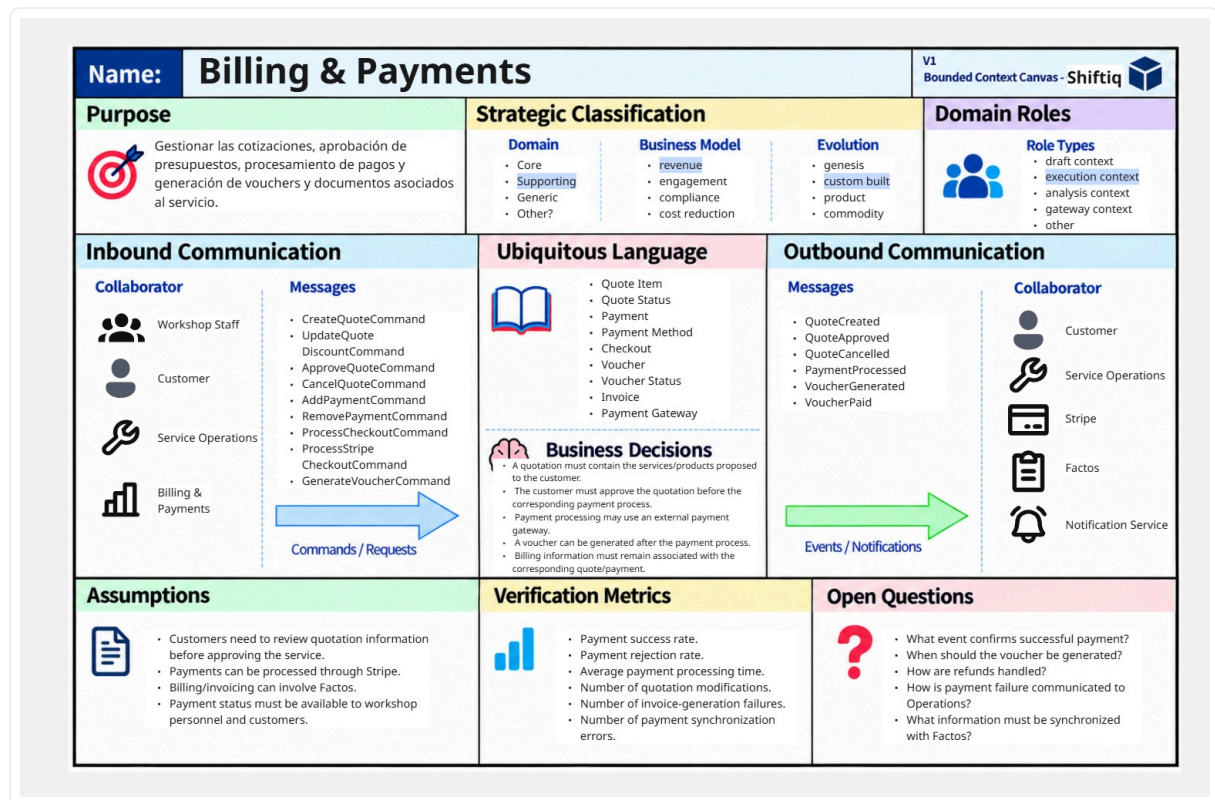


Figura 24: Bounded Context Canvas — Billing & Payments

Figura 39. Bounded Context Canvas — Billing & Payments.

El Canvas formaliza las responsabilidades relacionadas con Quote, Payment, Checkout y Voucher, incluyendo las interacciones necesarias con servicios externos de pago y facturación.

2.5.2. Context Mapping

Una vez identificados los siete **Candidate Bounded Contexts** mediante EventStorming y formalizadas sus responsabilidades mediante los **Bounded Context Canvases**, se realizó el **Context Mapping** con el propósito de analizar las relaciones existentes entre dichos contextos.

El Context Mapping permite pasar de la identificación individual de los contextos a una visión de conjunto, determinando **quién consume información de quién, qué contexto proporciona determinados servicios o eventos y qué sistemas externos participan en cada interacción.**

Para Shiftlq, el análisis considera los siguientes siete contextos:

1. **Identity & Access**
2. **Workshop Management**
3. **Fleet & Appointments**
4. **Vehicle Intelligence & Diagnostics**
5. **Service Operations**
6. **Inventory Management**

7. Billing & Payments

La separación de estos contextos tiene correspondencia con la organización del backend, donde existen módulos independientes para iam, core, fleet, iot, operations, inventory y billing.

Identificación de relaciones entre Bounded Contexts

El análisis de Context Mapping se realizó tomando como referencia los **Domain Message Flows** desarrollados anteriormente. Estos flujos permitieron identificar qué contextos requieren información o servicios proporcionados por otros contextos.

Las principales relaciones identificadas son:

Contexto consumidor	Contexto proveedor	Información / servicio relacionado
Fleet & Appointments	Workshop Management	Workshop, Branch, Customer, Employee
Fleet & Appointments	Vehicle Intelligence & Diagnostics	Vehicle
Service Operations	Fleet & Appointments	Appointment
Service Operations	Workshop Management	Branch, Employee
Service Operations	Inventory Management	Product, Stock, Reservation
Service Operations	Billing & Payments	Billing / Payment
Billing & Payments	Service Operations	Service / Work Order information
Identity & Access	Workshop Management	User identity / profile association
Workshop Management	Identity & Access	User identity

Estas relaciones se encuentran respaldadas por la estructura del backend. Por ejemplo, fleet dispone de servicios externos hacia core y hacia el contexto de vehículos, mientras que operations posee servicios externos para Appointment, Billing, Branch, Employee y Product.

Context Map de Shiftlq

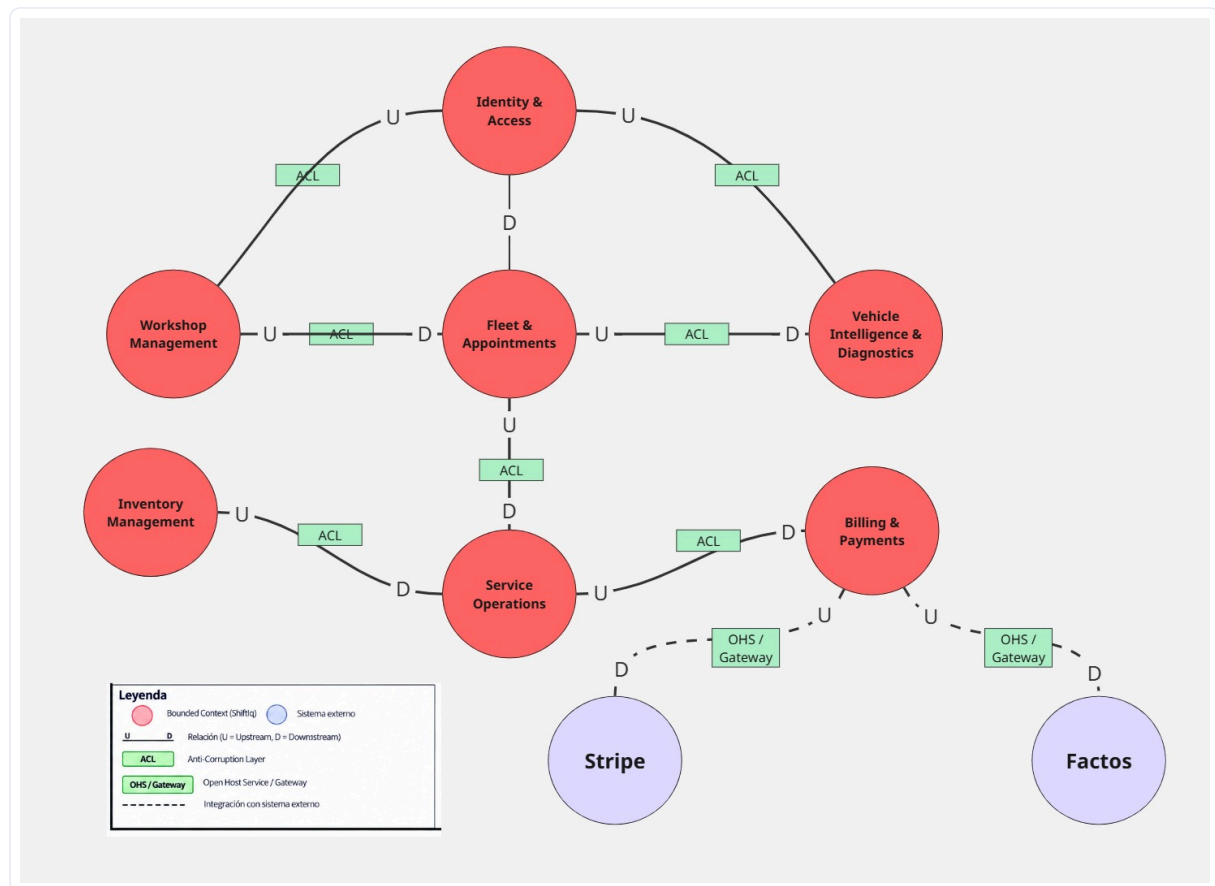
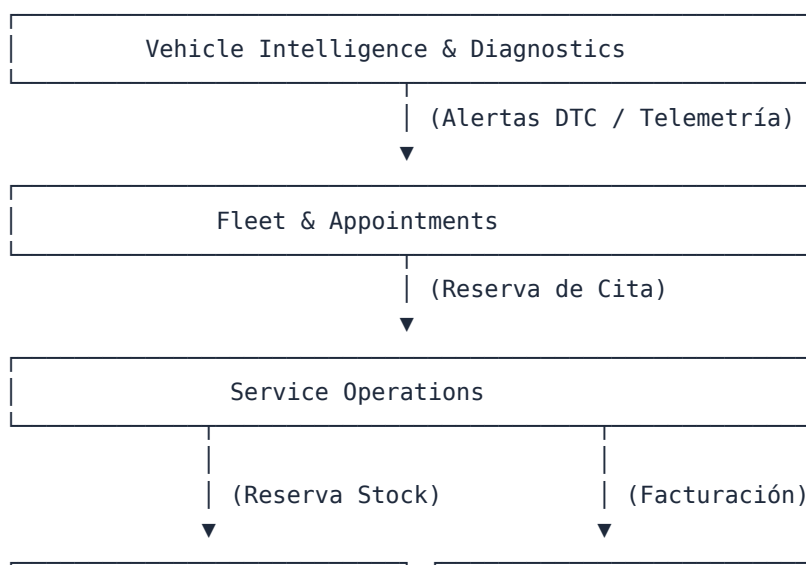


Figura 25: Context Map — Relaciones entre los Bounded Contexts de Shiftlq

Figura 40. Context Map — Relaciones entre los Bounded Contexts de Shiftlq.

El mapa representa las principales relaciones entre los siete contextos identificados. Las conexiones muestran que los contextos mantienen responsabilidades independientes, pero requieren colaborar para completar determinados procesos de negocio.

La interacción más relevante se presenta alrededor del proceso de servicio automotriz:



Inventory Management	Billing & Payments
----------------------	--------------------

2.5.3. Software Architecture

2.5.3.1. Software Architecture Context Level Diagrams

Según el Modelo C4 de Simon Brown, el **Diagrama de Contexto del Sistema** ofrece la vista de mayor nivel de abstracción, representando el sistema como una caja negra. En ShiftIQ, este nivel muestra la plataforma como sistema central y sus interacciones con los usuarios principales, el dispositivo IoT del vehículo y los servicios externos utilizados para mensajería, pagos y facturación electrónica.

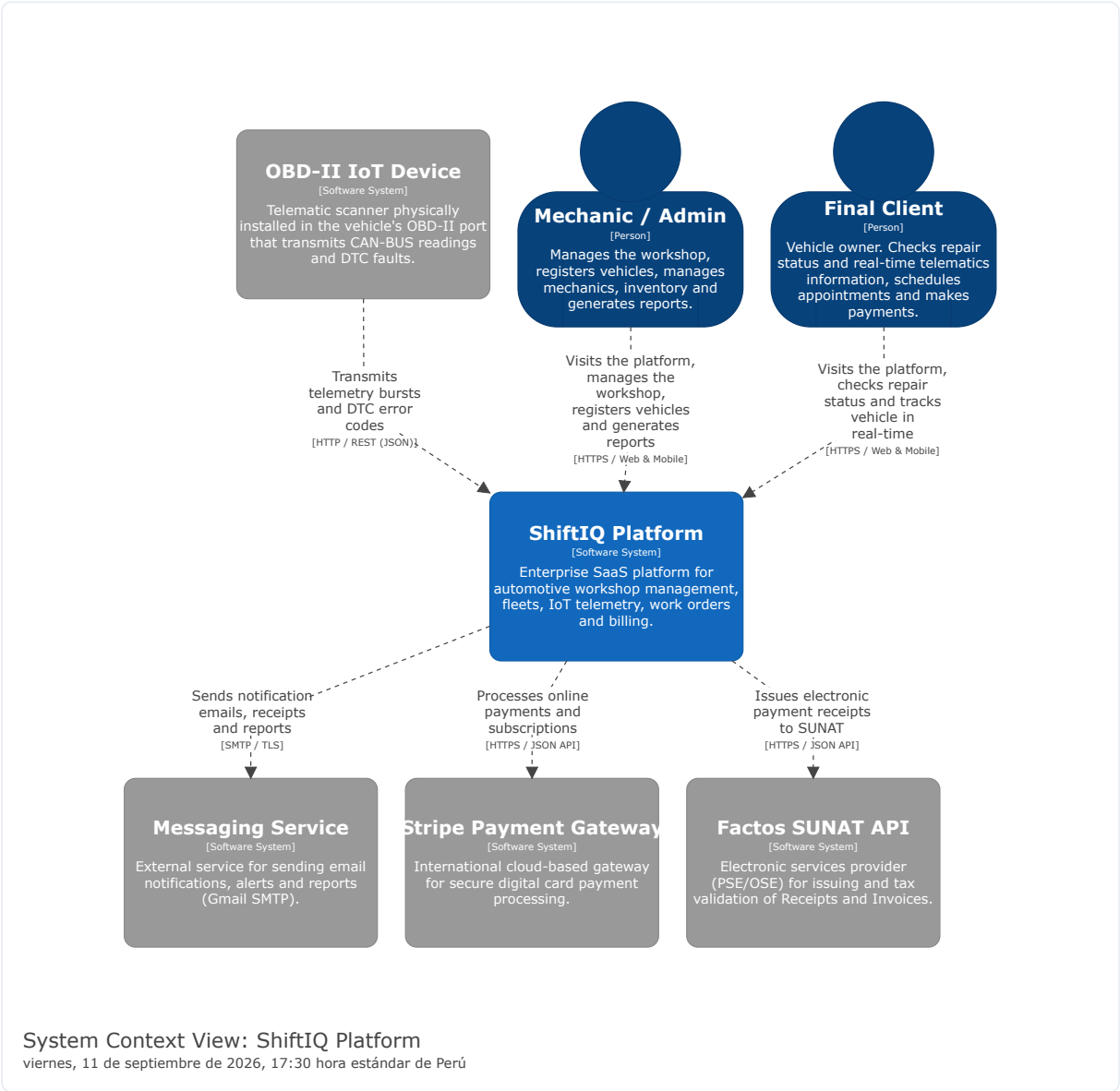


Figura 26: Diagrama de Contexto del Sistema — ShiftIQ Platform

Figura 41. Diagrama de Contexto del Sistema — ShiftIQ Platform.

Elementos y Relaciones

- **Mechanic / Admin:** Usuario administrativo y técnico que gestiona talleres, sucursales, vehículos y genera reportes operativos.
 - **Final Client:** Cliente dueño del vehículo que monitorea el estado del mantenimiento en tiempo real y gestiona sus citas.
 - **Dispositivo OBD-II IoT:** Componente de hardware que transmite parámetros del vehículo y códigos de diagnóstico DTC.
 - **ShiftIQ Platform:** Sistema central que integra la gestión de talleres, vehículos, telemetría, órdenes de trabajo y facturación.
 - **Messaging Service:** Servicio externo utilizado para el envío de notificaciones y comprobantes por correo electrónico.
 - **Stripe Payment Gateway:** Pasarela externa utilizada para el procesamiento de pagos digitales.
 - **Factos SUNAT API:** Servicio externo utilizado para la emisión y validación de comprobantes electrónicos.
 - **Relaciones principales:** Los usuarios interactúan con ShiftIQ mediante interfaces web y móviles, el dispositivo OBD-II transmite información telemática hacia la plataforma y ShiftIQ se comunica con los servicios externos para mensajería, pagos y facturación.
-

2.5.3.2. Software Architecture Container Level Diagrams

El **Diagrama de Contenedores** abre la caja negra del sistema para mostrar sus principales unidades ejecutables y responsabilidades tecnológicas. En ShiftIQ se representan la Landing Page, la aplicación web SPA, la aplicación móvil, la API RESTful y la base de datos PostgreSQL, junto con sus principales mecanismos de comunicación y las integraciones con servicios externos.

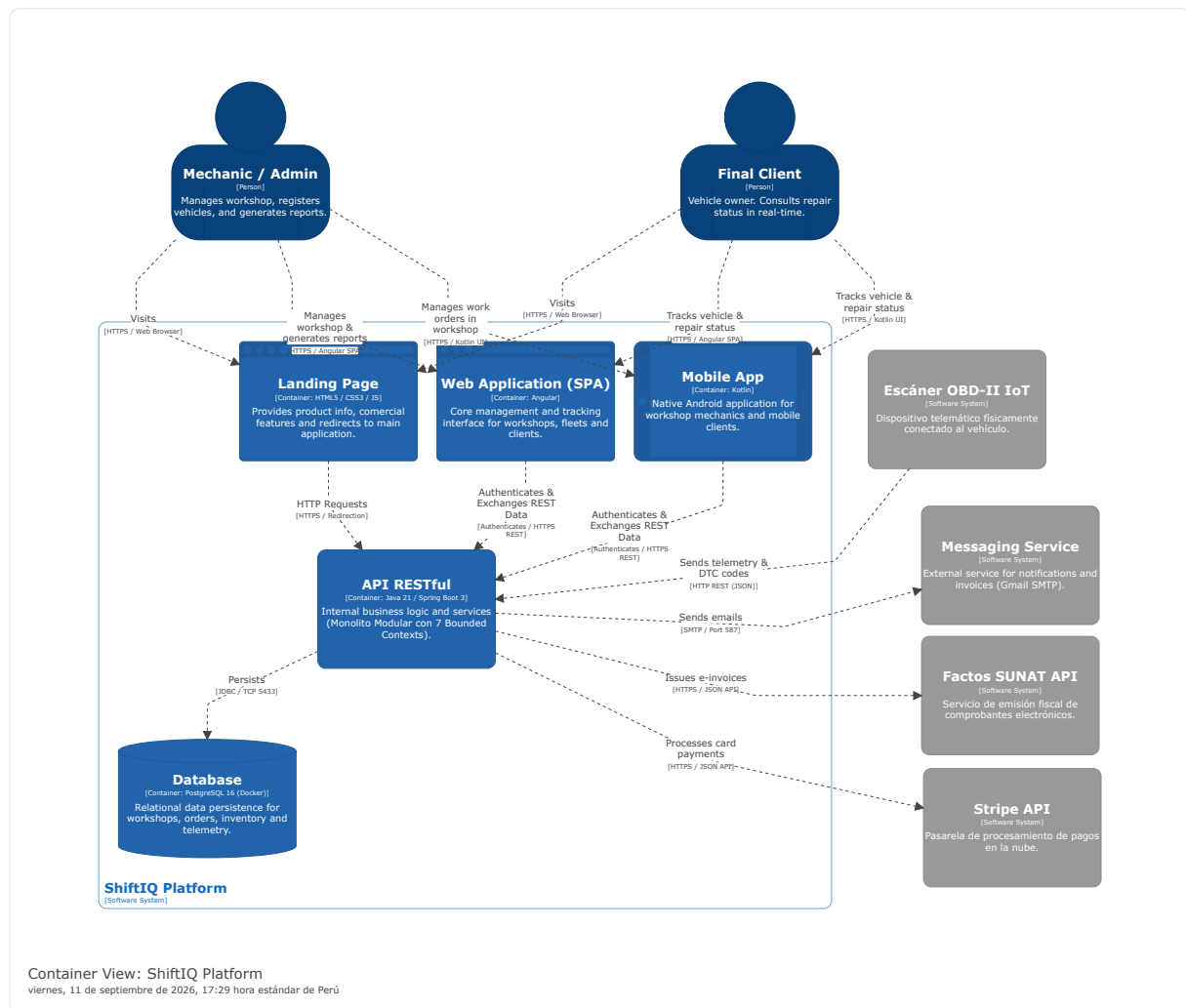


Figura 27: Diagrama de Contenedores — ShiftIQ Platform

Figura 42. Diagrama de Contenedores — ShiftIQ Platform.

Elementos y Relaciones

- **Landing Page:** Sitio web de presentación del producto que proporciona información comercial y redirige hacia la aplicación principal.
- **Web Application (SPA):** Interfaz web desarrollada en Angular para la gestión del taller, flota y seguimiento de los vehículos.
- **Mobile App:** Aplicación nativa Android desarrollada en Kotlin para mecánicos y clientes.
- **API RESTful:** Backend desarrollado con Java 21 y Spring Boot 3 que contiene la lógica de negocio y los siete Bounded Contexts.
- **Database:** Base de datos PostgreSQL utilizada para la persistencia de la información del sistema.
- **Messaging Service:** Servicio externo utilizado para el envío de notificaciones y facturas por correo electrónico.
- **Relaciones principales:** Las aplicaciones web y móvil consumen los servicios de la API RESTful mediante HTTPS. El dispositivo OBD-II transmite telemetría y códigos DTC al backend. La API persiste información en PostgreSQL y se comunica con los servicios externos de mensajería, pagos y facturación.

2.5.3.3. Software Architecture Deployment Diagrams

El **Diagrama de Despliegue** representa la distribución física y tecnológica de ShiftIQ en su infraestructura. Esta vista muestra cómo los componentes definidos en los niveles anteriores se despliegan sobre dispositivos de usuario, servidores web, servidores de aplicación, contenedores Docker y servicios externos en la nube.

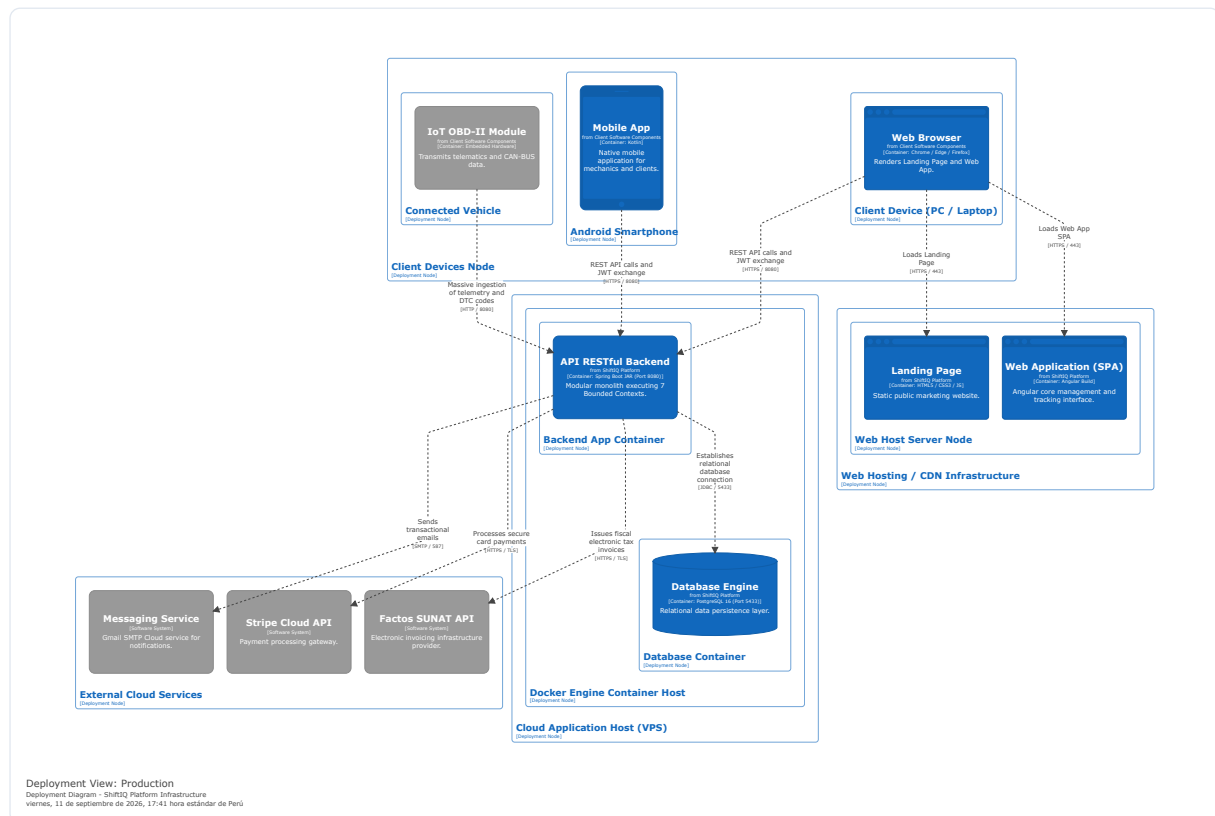


Figura 28: Diagrama de Despliegue — ShiftIQ Platform Infrastructure

Figura 43. Diagrama de Despliegue — ShiftIQ Platform Infrastructure.

Elementos y Relaciones

- **Web Browser:** Dispositivo utilizado para acceder a la Landing Page y a la Web Application.
- **Smartphone Android:** Dispositivo donde se ejecuta la aplicación móvil desarrollada en Kotlin.
- **Módulo IoT OBD-II:** Hardware instalado en el vehículo que transmite información telemática.
- **Servidor Web / CDN Hosting:** Infraestructura encargada de distribuir los archivos estáticos de la Landing Page y la aplicación web.
- **Cloud Application Host:** Servidor que aloja el entorno de ejecución de la aplicación backend.
- **Docker Engine:** Entorno de contenedores donde se ejecutan el backend y la base de datos.
- **API RESTful Backend:** Aplicación Spring Boot que ejecuta los siete Bounded Contexts.
- **Database:** Contenedor PostgreSQL encargado de la persistencia de los datos.

- **Servicios externos:** Servicios cloud utilizados para mensajería, procesamiento de pagos y facturación electrónica.
- **Relaciones principales:** Los navegadores acceden al servidor web mediante HTTPS, mientras que la aplicación web y móvil consumen la API RESTful. El dispositivo OBD-II transmite información al backend. La API se conecta con PostgreSQL mediante JDBC y con los servicios externos mediante HTTPS y SMTP.

2.6. Tactical-Level Domain-Driven Design

2.6.1. Bounded Context: Shared Kernel

El **Shared Kernel** (shared) provee la infraestructura transversal, las abstracciones de dominio compartidas, los Value Objects reutilizables, la gestión de eventos de dominio cross-context, el patrón funcional de manejo de errores (Result<T, E>), la seguridad multi-tenant por sucursal (MultiTenancySecurityService), el mapeo relacional base auditado (AuditableAbstractPersistenceEntity) y el manejo centralizado de excepciones REST en la plataforma **ShiftIQ**.

2.6.1.1. Domain Layer

La Capa de Dominio del Shared Kernel encapsula los tipos de valor reutilizables entre múltiples Bounded Contexts, la abstracción base para raíces de agregado (AbstractDomainAggregateRoot), y los eventos de dominio de integración que comunican el flujo entre módulos sin acoplamiento directo de infraestructura.

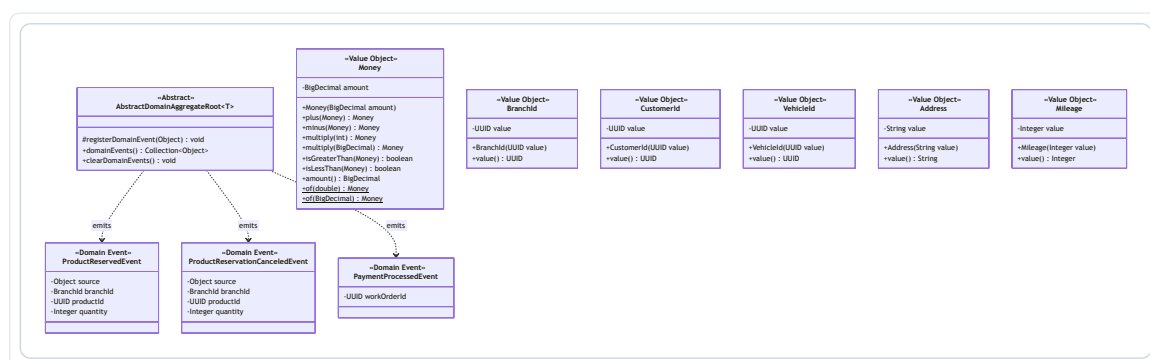


Figura 29: Diagrama de la Capa de Dominio – Shared Kernel

2.6.1.1.1. Base Aggregates & Abstract Entities

AbstractDomainAggregateRoot<T> extends AbstractDomainAggregateRoot<T>>

Tipo: Clase Abstracta (extends AbstractAggregateRoot<T>)

Propósito: Provee soporte inmutable para registro y despacho de Eventos de Dominio sin acoplar el modelo a JPA ni a frameworks de persistencia.

Métodos:

- `#registerDomainEvent(Object event)`: Registra un evento de dominio para ser publicado tras persistir el agregado.
- `+domainEvents()`: Retorna la colección no modificable de eventos registrados.
- `+clearDomainEvents()`: Limpia la lista de eventos tras su publicación exitosa por los adaptadores de repositorio.

2.6.1.1.2. Shared Value Objects & Records

Record: Money(BigDecimal amount)

Propósito: Value Object inmutable para montos monetarios.

Invariantes & Validaciones:

- No puede ser nulo (`operations.error.money.required`).
- No puede ser negativo (`operations.error.money.cannotBeNegative`).
- Redondeo automático a 2 decimales (`HALF_UP`).

Operaciones: `plus(Money)`, `minus(Money)`, `multiply(int)`, `multiply(BigDecimal)`, `isGreaterThan(Money)`, `isLessThan(Money)`.

Constantes: `ZERO (BigDecimal.ZERO)`.

Record: BranchId(UUID value)

Propósito: Identificador fuertemente tipado para sucursales.

Validación: No nulo (`shared.error.branchId.required`).

Record: CustomerId(UUID value)

Propósito: Identificador fuertemente tipado para clientes.

Validación: No nulo (`shared.error.customerId.required`).

Record: VehicleId(UUID value)

Propósito: Identificador fuertemente tipado para vehículos.

Validación: No nulo (`shared.error.vehicleId.required`).

Record: Mileage(Integer value)

Propósito: Kilometraje de vehículos.

Validación: No nulo y no negativo.

Record: Address(String value)

Propósito: Dirección física formateada.

Validación: No vacía ni nula (`operations.error.address.notBlank`) y longitud máxima de 100 caracteres.

2.6.1.1.3. Cross-Context Domain Events

- `ProductReservedEvent(Object source, BranchId branchId, UUID productId, Integer quantity)`: Notifica la reserva temporal de repuestos emitida desde `Operations` hacia `Inventory`.

- **ProductReservationCanceledEvent(Object source, BranchId branchId, UUID productId, Integer quantity)**: Notifica la liberación de reservas de stock al modificar o cancelar tareas de órdenes de trabajo.
- **PaymentProcessedEvent(UUID workOrderId)**: Notifica el procesamiento exitoso de pago de una orden de trabajo desde Billing.

2.6.1.2. Interface Layer

Manejo global de excepciones (@RestControllerAdvice), ensambladores universales de respuestas HTTP e internacionalización (MessageSource).

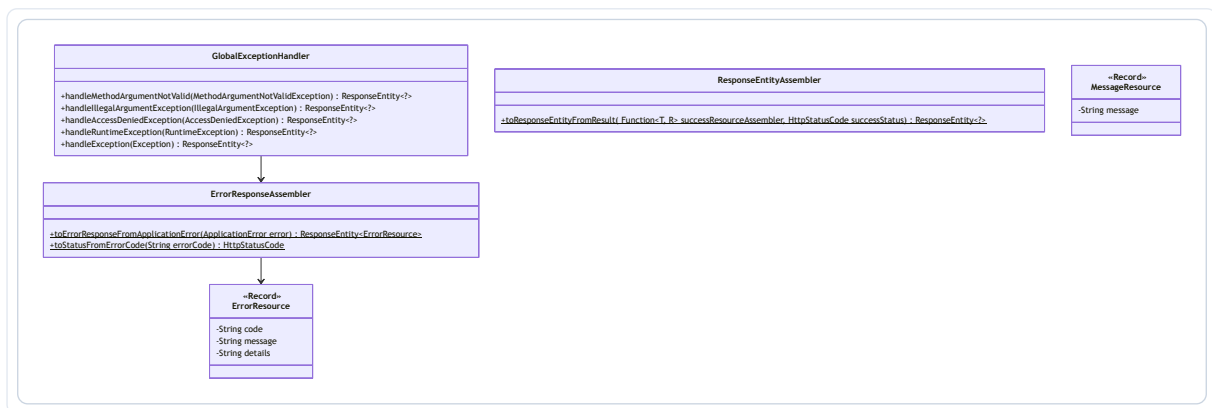


Figura 30: Diagrama de la Capa de Interfaz – Shared Kernel

2.6.1.2.1. Infrastructure REST Utilities & Cross-Cutting Exception Handlers

GlobalExceptionHandler (@RestControllerAdvice)

Centraliza las excepciones no capturadas a nivel REST. Traduce @Valid binding errors (MethodArgumentNotValidException), IllegalArgumentException, AccessDeniedException y RuntimeException a respuestas ErrorResponse internacionalizadas mediante messages.properties.

ErrorResponseAssembler & ResponseEntityAssembler

Mapea códigos de error (VALIDATION_ERROR, NOT_FOUND, CONFLICT, ACCESS_DENIED) a los códigos de estado HTTP correspondientes (400, 404, 409, 403, 500).

2.6.1.3. Application Layer

La Capa de Aplicación del Shared Kernel provee la estructura funcional Result<T, E> para manejo de errores sin excepciones de control de flujo, y el modelo canónico de errores de aplicación ApplicationError.

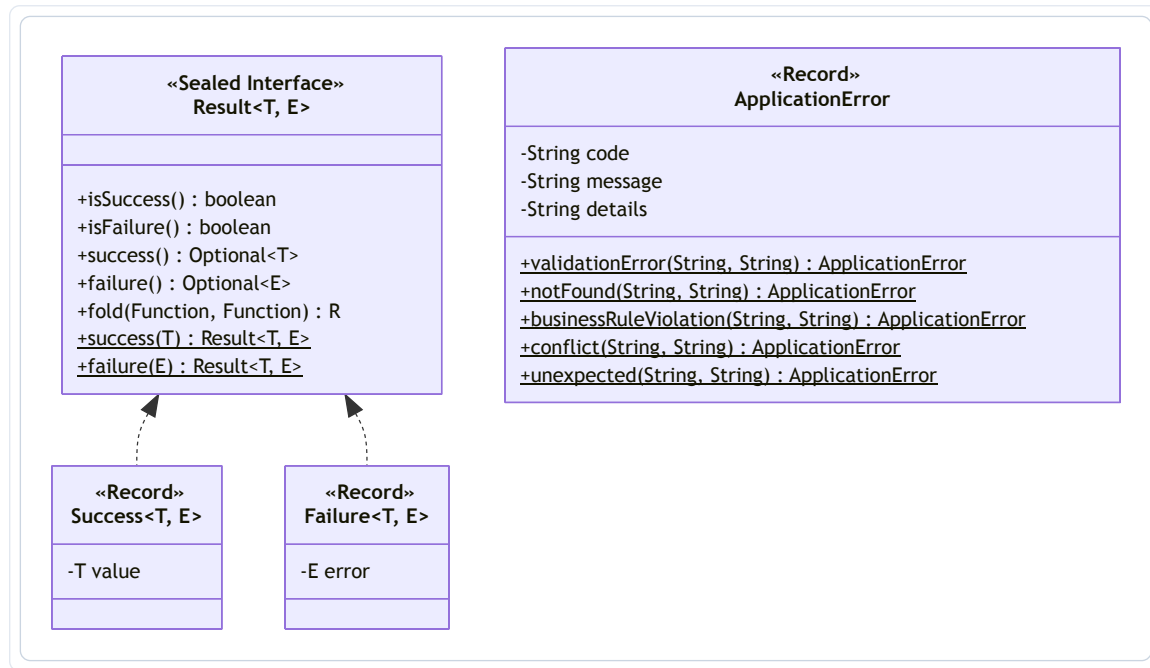


Figura 31: Diagrama de la Capa de Aplicación – Shared Kernel

2.6.1.3.1. Functional Result Pattern & Error Specification

Sealed Interface: Result<T, E>

Permite: Result.Success<T, E>, Result.Failure<T, E>.

Métodos Principales:

- static success(T value) / static failure(E error): Métodos de fábrica.
- fold(onSuccess, onFailure): Evaluación funcional pattern-matching.
- isSuccess(), isFailure(), success(), failure().

Record: ApplicationError(code, message, details)

Métodos Estáticos de Fábrica:

- validationError(field, reason)
- notFound(resourceType, identifier)
- businessRuleViolation(rule, reason)
- conflict(resource, reason)
- unexpected(context, reason)

2.6.1.4. Infrastructure Layer

Clase base relacional auditada JPA (AuditableAbstractPersistenceEntity), conversores de atributos (AttributeConverter), seguridad multi-tenant por sucursal y configuraciones transversales.

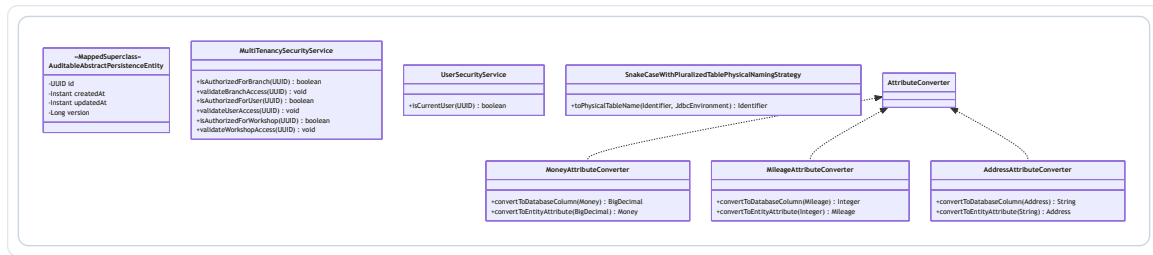


Figura 32: Diagrama de la Capa de Infraestructura – Shared Kernel

2.6.1.4.1. JPA MappedSuperclass & Persistence Base

@MappedSuperclass: AuditableAbstractPersistenceEntity

Anotaciones: @EntityListeners(AuditingEntityListener.class)

Atributos Heredados:

- @Id @GeneratedValue(strategy = GenerationType.UUID) UUID id
- @CreateDate Instant createdAt
- @LastModifiedDate Instant updatedAt
- @Version Long version

2.6.1.4.2. JPA Custom Attribute Converters

- **MoneyAttributeConverter:** Mapea Money ↔ DECIMAL(12,2).
- **MileageAttributeConverter:** Mapea Mileage ↔ INTEGER.
- **AddressAttributeConverter:** Mapea Address ↔ VARCHAR(100).

2.6.1.4.3. Multi-Tenancy Security & Auditing

- **MultiTenancySecurityService:** Bean @Service("multiTenancySecurityService") expuesto para expresiones SpEL (@PreAuthorize) que valida si el usuario autenticado posee permisos sobre el branchId, userId o workshopId de la petición.
- **UserSecurityService:** Bean @Service("userSecurityService") para verificación de identidad propia en SpEL (prevención de IDOR).
- **SnakeCaseWithPluralizedTablePhysicalNamingStrategy:** Convierte los nombres de entidades de CamelCase a snake_case pluralizado para PostgreSQL (ej. WorkOrder → work_orders).

2.6.1.5. Bounded Context Software Architecture Component Level Diagrams

Descomposición del Container REST API resaltando los componentes del **Shared Kernel** que prestan servicio transversal a todos los Bounded Contexts.

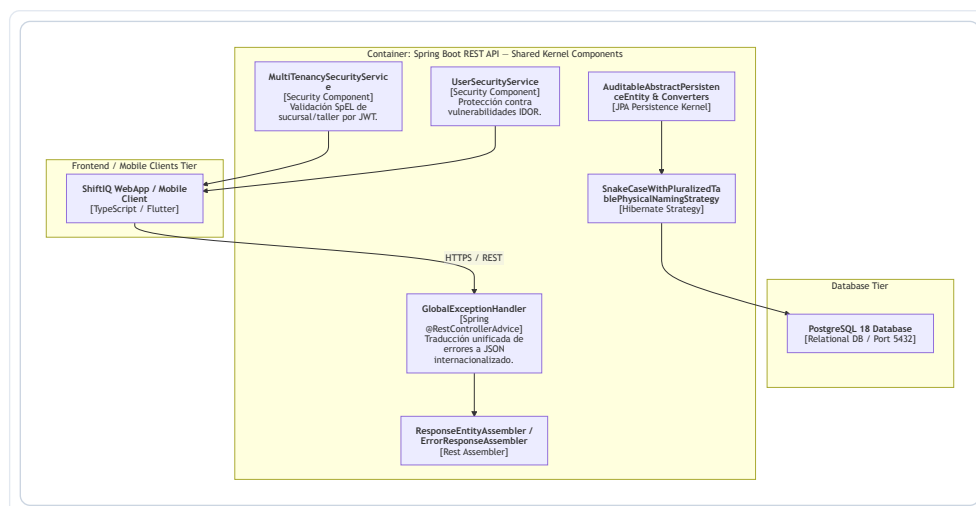


Figura 33: Component Diagram (C4 Level 3) – Shared Kernel

2.6.1.6. Bounded Context Software Architecture Code Level Diagrams

2.6.1.6.1. Bounded Context Domain Layer Class Diagrams

Representación detallada de clases del módulo Shared Kernel en formato UML, abarcando las clases abstractas, Value Objects, Records, Eventos de Dominio, interfaces selladas y utilitarios transversales.



Figura 34: Diagrama de Clases del Dominio UML – Shared Kernel

2.6.1.6.2. Bounded Context Database Design Diagram

Estructura de la tabla relacional base heredada por las entidades persistentes mediante la estrategia @MappedSuperclass de JPA en PostgreSQL:

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
version	BIGINT	NOT NULL, DEFAULT 0 (Control de concurrencia optimista)

Este documento presenta la especificación exhaustiva, formal y técnica de los **Bounded Contexts** implementados en el ecosistema **ShiftIQ Platform**. Cada contexto se estructura rigurosamente bajo los lineamientos del **Domain-Driven Design (DDD) Táctico** y los princi-

pios de la **Arquitectura Limpia / Hexagonal**, alineado al 100% con la base de código real. Incluye diagramas de arquitectura en **C4 Model (Nivel 3: Componentes)**, diagramas a nivel de código (**UML Class Diagrams** y **Database ER Diagrams**), y el **Diccionario de Clases por Capas** con atributos tipados, firmas completas de métodos, visibilidad, reglas de negocio, invariantes y relaciones estructurales entre componentes a través de las cuatro capas fundamentales: **Domain Layer**, **Application Layer**, **Interface Layer** e **Infrastructure Layer**.

Este documento presenta la especificación exhaustiva, formal y técnica de los **Bounded Contexts** implementados en el ecosistema **ShiftIQ Platform**. Cada contexto se estructura rigurosamente bajo los lineamientos del **Domain-Driven Design (DDD) Táctico** y los principios de la **Arquitectura Limpia / Hexagonal**, alineado al 100% con la base de código real. Incluye diagramas de arquitectura en **C4 Model (Nivel 3: Componentes)**, diagramas a nivel de código (**UML Class Diagrams** y **Database ER Diagrams**), y el **Diccionario de Clases por Capas** con atributos tipados, firmas completas de métodos, visibilidad, reglas de negocio, invariantes y relaciones estructurales entre componentes a través de las cuatro capas fundamentales: **Domain Layer**, **Application Layer**, **Interface Layer** e **Infrastructure Layer**.

2.6.2. Bounded Context: Identity & Access Management (IAM)

El Bounded Context de **Identity & Access Management (IAM)** constituye la piedra angular de seguridad, identidad y control de acceso de la plataforma **ShiftIQ**. Su responsabilidad primordial radica en centralizar el ciclo de vida de las identidades de usuario, garantizando:

- La confidencialidad y almacenamiento seguro de credenciales mediante hashing criptográfico BCrypt.
- La provisión de mecanismos de autenticación local (vía email y contraseña) y federada (mediante Google Identity Services / OAuth 2.0).
- La emisión, firma criptográfica y verificación de tokens de autorización sin estado (**JSON Web Tokens - JWT**).
- La asignación y verificación de roles de seguridad (Role-Based Access Control - RBAC) con soporte de autorización por propiedad de recurso mediante UserSecurityService.
- La orquestación del restablecimiento seguro de contraseñas olvidadas mediante tokens efímeros hasheados con SHA-256 (TTL por defecto de 60 minutos) y notificados vía correo electrónico (SMTP).

2.6.2.1. Domain Layer (Capa de Dominio)

La Capa de Dominio encierra la lógica de negocio pura, las invariantes operativas y las reglas del sistema de identidad, manteniéndose completamente agnóstica de frameworks web, motores de bases de datos o librerías de persistencia. En esta capa se definen Agregados, Entidades, Value Objects, Servicios de Dominio, Fábricas e Interfaces de Repositorio.

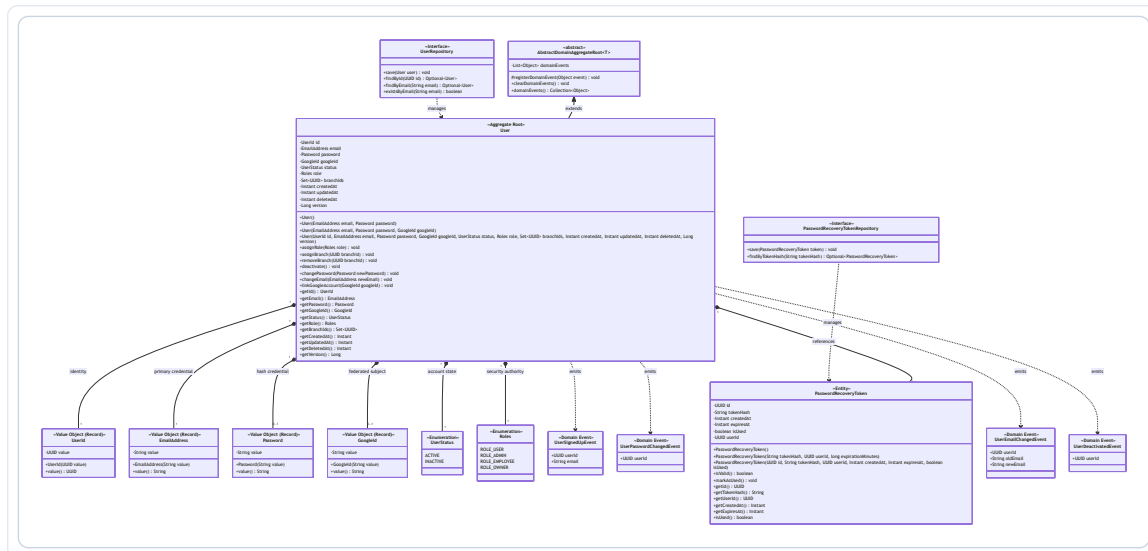


Figura 35: Diagrama de la Capa de Dominio – IAM

2.6.2.1.1. Aggregates & Entities

User (Aggregate Root) Tipo: Aggregate Root (extends `AbstractDomainAggregateRoot<User>`) **Propósito:** Raíz de consistencia del agregado de usuario en IAM. Encapsula las credenciales, el estado vital de la cuenta, el rol de autorización RBAC, las asociaciones a sucursales de taller y las transiciones de estado, garantizando la publicación atómica de eventos de dominio ante cambios relevantes.

Atributos del Agregado:

- `id: UserId` (No Nulo) – Identificador único tipado del usuario encapsulado en un Value Object.
- `email: EmailAddress` (No Nulo) – Dirección de correo electrónico normalizada y validada.
- `password: Password` (No Nulo para auth local) – Hash de la contraseña encriptada con BCrypt.
- `googleId: GoogleId` (Opcional) – Sujeto único emitido por Google OAuth (sub).
- `status: UserStatus` (No Nulo) – Estado de la cuenta (ACTIVE o INACTIVE).
- `role: Roles` (No Nulo) – Rol de autorización (ROLE_USER, ROLE_ADMIN, ROLE_EMPLOYEE, ROLE_OWNER).
- `branchIds: Set<UUID>` (No Nulo) – Colección de identificadores de sedes (BranchId) asociadas al usuario.
- `createdAt, updatedAt, deletedAt: Instant` – Auditoría temporal UTC.
- `version: Long` – Control de concurrencia optimista.

Métodos y Comportamientos de Dominio:

- `assignRole(Roles role)`: Actualiza el rol de autorización RBAC.
- `assignBranch(UUID branchId) / removeBranch(UUID branchId)`: Asocia o desvincula sedes operativas.

- `deactivate()`: Desactiva la cuenta (`status = INACTIVE`) y registra `UserDeactivatedEvent`.
- `changePassword(Password newPassword)`: Actualiza la clave y registra `UserPasswordChangeEvent`.
- `changeEmail(EmailAddress newEmail)`: Cambia el correo y registra `UserEmailChangeEvent`.
- `linkGoogleAccount(GoogleId googleId)`: Vincula la identidad federada de Google.

PasswordRecoveryToken (Entity) Tipo: Entidad de Dominio **Propósito:** Representa un token efímero de recuperación de contraseña asociado a una cuenta de usuario.

Atributos:

- `id`: UUID – Identificador único de la entidad.
- `tokenHash`: String – Digest SHA-256 del token enviado al usuario.
- `createdAt`: Instant – Fecha y hora de generación.
- `expiresAt`: Instant – Expiración calculada (TTL de 60 minutos por defecto).
- `isUsed`: boolean – Bandera de consumo (inicialmente `false`).
- `userId`: UUID – Identificador del usuario propietario.

Reglas de Negocio e Invariantes:

- `isValid()`: Retorna `true` si `!isUsed` y `Instant.now().isBefore(expiresAt)`.
- `markAsUsed()`: Marca la bandera `isUsed = true`.

2.6.2.1.2. Value Objects & Records

Record: UserId(UUID value) Propósito: Identificador fuertemente tipado del usuario. **Validación:** Invariante no nula (`iam.error.userId.required`).

Record: EmailAddress(String value) Propósito: Correo electrónico normalizado. **Validación:** Regex RFC 5322 (`iam.error.email.invalidFormat`).

Record: Password(String value) Propósito: Credencial o hash criptográfico. **Validación:** No nula ni vacía (`iam.error.password.required`).

Record: GoogleId(String value) Propósito: Sujeto federado de Google OAuth (sub). **Validación:** Identificador de sujeto no nulo.

2.6.2.1.3. Enumerations

Enum: UserStatus Propósito: Estado vital de la cuenta. - `ACTIVE`: Habilitada para autenticación.

- `INACTIVE`: Dada de baja lógica.

Enum: Roles Propósito: Roles RBAC de seguridad. - `ROLE_USER`: Usuario cliente básico.

- `ROLE_ADMIN`: Administrador de plataforma.
- `ROLE_EMPLOYEE`: Empleado/Técnico.

- **ROLE_OWNER:** Propietario de taller.

2.6.2.1.4. Domain Repositories (Interfaces)

Interface: UserRepository **Métodos:** save(User user), findById(UUID id), findByEmail(String email), existsByEmail(String email).

Interface: PasswordRecoveryTokenRepository **Métodos:** save(PasswordRecoveryToken token), findByTokenHash(String tokenHash).

2.6.2.1.5. Domain Events

- UserSignedUpEvent(Object source, UUID userId, String email): Publicado al crearse una nueva cuenta.
- UserPasswordChangedEvent(Object source, UUID userId): Publicado al modificar la contraseña.
- UserEmailChangedEvent(Object source, UUID userId, String oldEmail, String newEmail): Publicado tras actualizar el correo.
- UserDeactivatedEvent(Object source, UUID userId): Publicado al ejecutar la baja lógica de la cuenta.

2.6.2.2. Application Layer (Capa de Aplicación)

La Capa de Aplicación orquesta los casos de uso del Bounded Context. Recibe comandos y consultas, coordina los agregados del dominio, ejecuta validaciones de unicidad y delega en servicios de infraestructura.

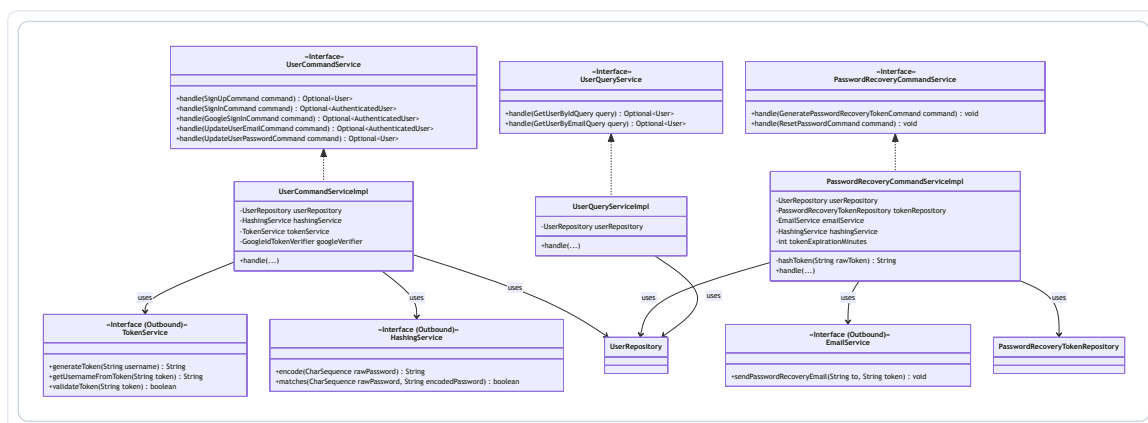


Figura 36: Diagrama de la Capa de Aplicación – IAM

2.6.2.2.1. Commands & Queries (DTOs de Aplicación)

Comandos de Escritura (CQRS Commands): - SignUpCommand(EmailAddress email, Password password): Alta de nuevo usuario.

- `SignInCommand(EmailAddress email, Password password)`: Autenticación local con credenciales.
- `GoogleSignInCommand(String idToken)`: Autenticación federada con Google Identity Services.
- `GeneratePasswordRecoveryTokenCommand(EmailAddress email)`: Solicitud de token de recuperación.
- `ResetPasswordCommand(String token, Password newPassword)`: Restablecimiento de contraseña.
- `UpdateUserEmailCommand(UserId userId, EmailAddress newEmail)`: Actualización de correo electrónico.
- `UpdateUserPasswordCommand(UserId userId, Password currentPassword, Password newPassword)`: Cambio de contraseña.

Consultas y DTOs de Resultado (CQRS Queries): - `GetUserByIdQuery(UserId userId)`: Consulta inmutable de perfil por ID.

- `GetUserByEmailQuery(EmailAddress email)`: Consulta inmutable por correo electrónico.
- `AuthenticatedUser(User user, String token)`: DTO de respuesta con agregados de usuario y JWT emitido.

2.6.2.2.2. Application Services

UserCommandServiceImpl (@Service, @Transactional) Orquesta el alta de usuarios (validando unicidad de correo), autenticación local (verificando hash BCrypt) y autenticación federada (validando token Google con `GoogleIdTokenVerifier`). Emite JWTs vía `TokenService`.

PasswordRecoveryCommandServiceImpl (@Service, @Transactional) Gestiona la emisión de tokens efímeros de recuperación (TTL 60 min), calcula su hash SHA-256 para almacenamiento y despacha el correo vía `EmailService`.

UserQueryServiceImpl (@Service, @Transactional(readOnly = true)) Atiende consultas de lectura de perfiles de usuario por ID o correo.

2.6.2.2.3. Outbound Port Interfaces

- `HashingService`: `encode(CharSequence raw) / matches(CharSequence raw, String encoded)`.
- `TokenService`: `generateToken(String username) / validateToken(String token) / getUsernameFromToken(String token)`.
- `EmailService`: `sendPasswordRecoveryEmail(String to, String token)`.

2.6.2.3. Interface Layer (Capa de Interfaces)

La Capa de Interfaces expone los controladores REST HTTP bajo la convención de URLs del sistema. Transforma peticiones JSON entrantes en comandos/queries y mapea los resultados a Resources DTOs.

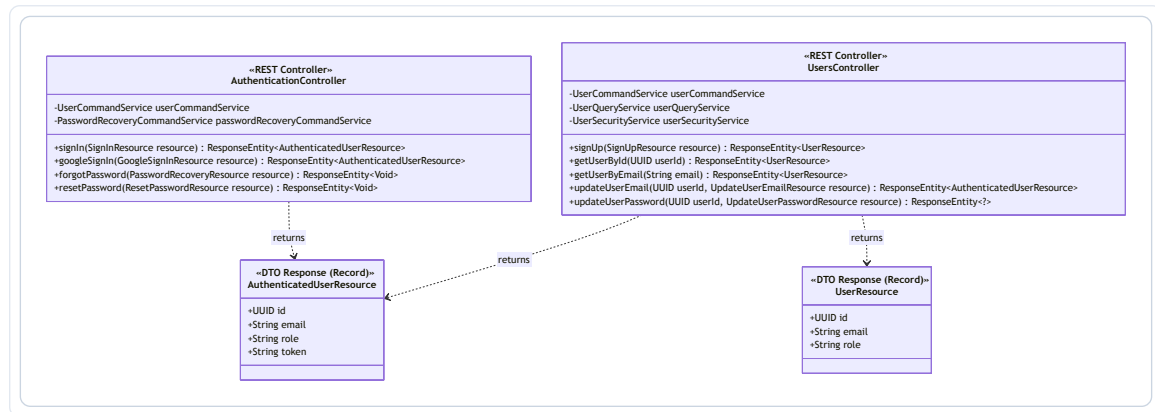


Figura 37: Diagrama de la Capa de Interfaces – IAM

2.6.2.3.1. REST Controllers & DTO Resources

AuthenticationController (/api/v1/authentication) - POST /sessions: Autenticación local con email y clave.

- POST /sessions/google: Autenticación federada con token OAuth de Google.
- POST /password-recoveries: Solicitud de envío de correo de recuperación.
- POST /password-resets: Restablecimiento de contraseña con token plano.

UsersController (/api/v1/users) - POST: Alta de nuevo usuario (SignUpResource).

- GET /{userId}: Consulta de usuario por ID.
- GET ?email={email}: Consulta por correo electrónico.
- PUT /{userId}/email: Actualización de correo electrónico.
- PUT /{userId}/password: Cambio de contraseña verificando clave actual.

2.6.2.4. Infrastructure Layer (Capa de Infraestructura)

La Capa de Infraestructura implementa la persistencia física en PostgreSQL 18 con Spring Data JPA, el filtrado de seguridad con Spring Security, el hashing BCrypt e integración externa con SMTP y Google Identity.

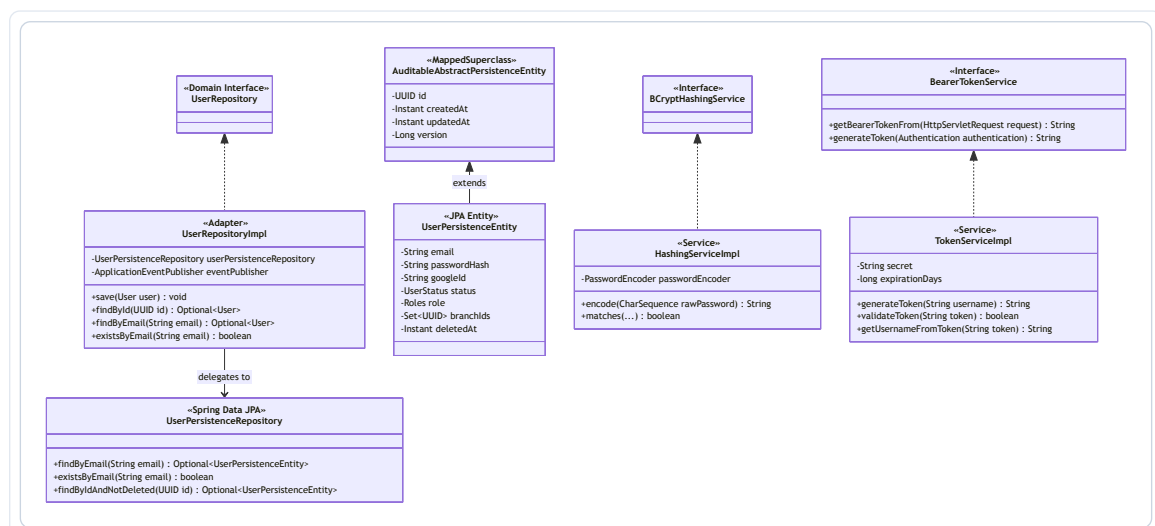


Figura 38: Diagrama de la Capa de Infraestructura – IAM

2.6.2.4.1. Persistence & Security Adapters

UserPersistenceEntity & UserRepositoryImpl Entidad JPA mapeada a la tabla users (extiende de AuditableAbstractPersistenceEntity). Adaptador UserRepositoryImpl que delega en UserPersistenceRepository y despacha eventos de dominio con ApplicationEventPublisher.

BearerAuthorizationRequestFilter & UserSecurityService Filtro OncePerRequestFilter de Spring Security que extrae y valida tokens Bearer JWT en cada petición HTTP. UserSecurityService evalúa expresiones SpEL (@PreAuthorize) para verificar propiedad sobre recursos.

2.6.2.5. C4 Model Component Diagram (Container: Spring Boot REST API – IAM)

Descomposición estructural del container REST API para el Bounded Context de IAM:

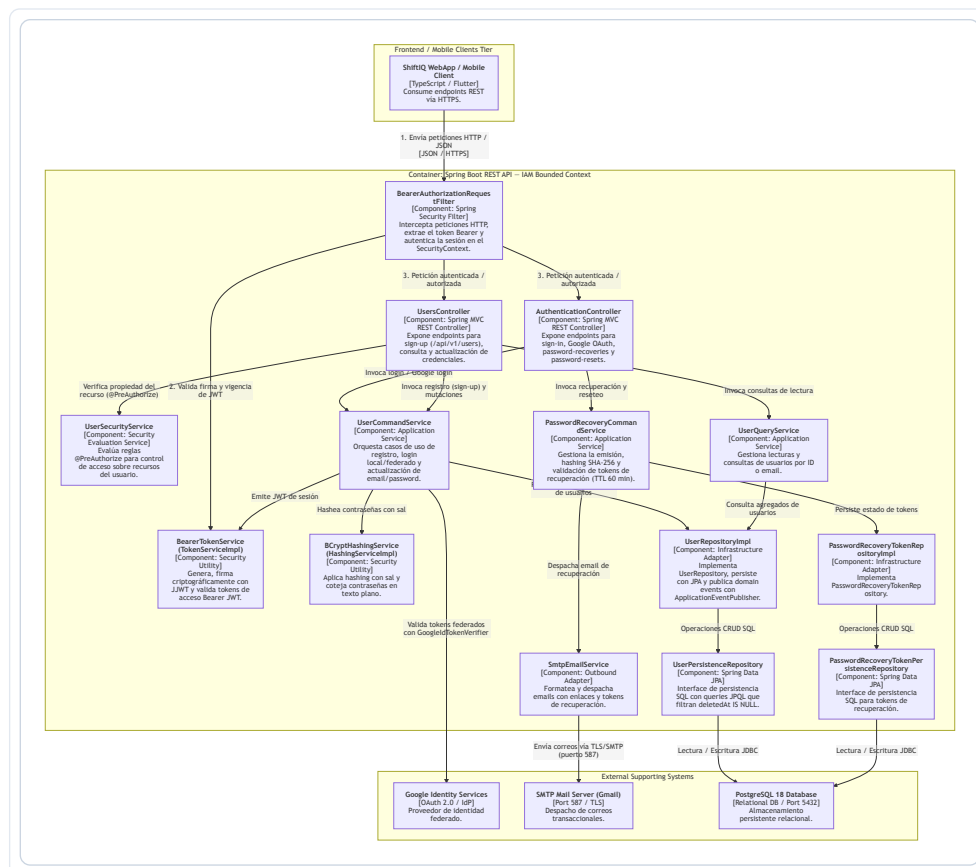


Figura 39: Diagrama de Componentes C4 Nivel 3 – IAM

2.6.2.6. Bounded Context Software Architecture Code Level Diagrams

2.6.2.6.1. Domain Layer Class Diagram

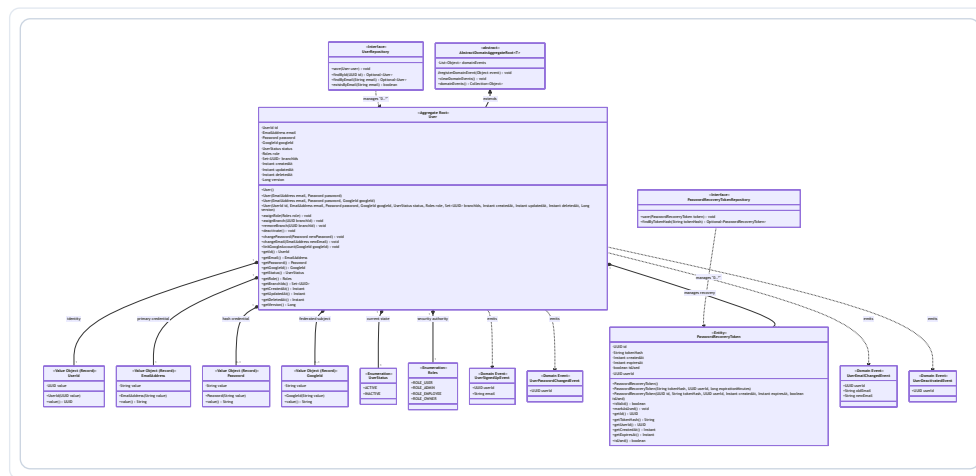


Figura 40: Diagrama de Clases del Dominio UML – IAM

2.6.2.6.2. Database Design Diagram (PostgreSQL 18)

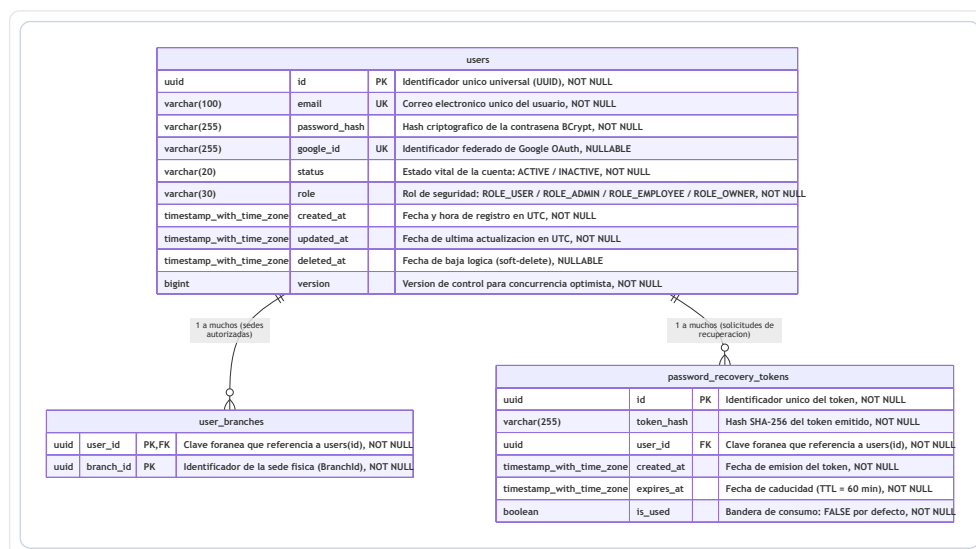


Figura 41: Diagrama de Base de Datos Relacional ER – IAM

Especificación de Tablas Relacionales (PostgreSQL 18):

Tabla: users

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
email	VARCHAR(100)	NOT NULL, UNIQUE INDEX
password_hash	VARCHAR(255)	NOT NULL (BCrypt hash)
google_id	VARCHAR(255)	NULLABLE, UNIQUE INDEX

Columna	Tipo de Dato	Constraints / Descripción
status	VARCHAR(20)	NOT NULL (ACTIVE / INACTIVE)
role	VARCHAR(30)	NOT NULL (ROLE_USER/ADMIN/EMPLOYEE/OWNER)
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
deleted_at	TIMESTAMP	NULLABLE (Soft-delete timestamp)
version	BIGINT	NOT NULL, DEFAULT 0 (Concurrencia optimista)

Tabla: user_branches

Columna	Tipo de Dato	Constraints / Descripción
user_id	UUID	PRIMARY KEY, FOREIGN KEY -> users(id)
branch_id	UUID	PRIMARY KEY (Sede física autorizada)

Tabla: password_recovery_tokens

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
token_hash	VARCHAR(255)	NOT NULL (SHA-256 digest)
user_id	UUID	FOREIGN KEY -> users(id)
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
expires_at	TIMESTAMP	NOT NULL (TTL 60 minutos)
is_used	BOOLEAN	NOT NULL, DEFAULT FALSE

2.6.3. Bounded Context: Operations (Work Orders & Services)

El Bounded Context de **Operations** constituye el motor operativo principal de la plataforma **ShiftIQ**. Gestiona el flujo de trabajo completo del taller automotriz: desde la definición del catálogo de servicios ofreciendo precios y mantenimiento (Service), la emisión y control del ciclo de vida de Órdenes de Trabajo (WorkOrder), la orquestación de tareas asignadas a mecánicos (WorkOrderTask), hasta el consumo y reserva de repuestos/productos de inventario (WorkOrderTaskProduct).

2.6.3.1. Domain Layer (Capa de Dominio)

La Capa de Dominio encapsula el modelo de negocio inmutable, asegurando transiciones estrictas de estado para las órdenes de trabajo y tareas mediante métodos **Factory**, reglas de negocio encapsuladas en el Agregado WorkOrder, el cálculo dinámico de costos y la emisión de eventos de dominio.

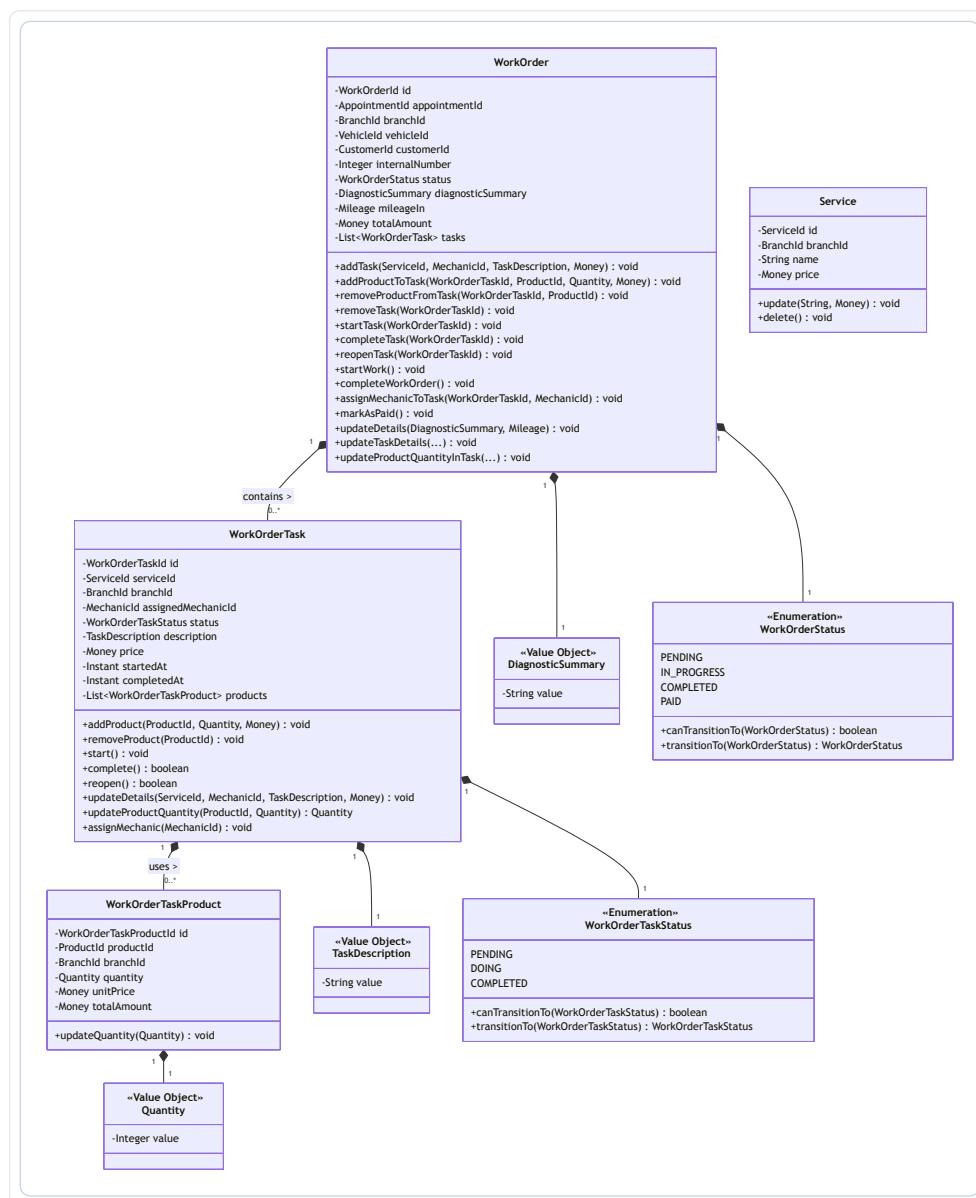


Figura 42: Diagrama de la Capa de Dominio – Operations

2.6.3.1.1. Aggregates & Entities

WorkOrder (Aggregate Root) Tipo: Aggregate Root (extends

AbstractDomainAggregateRoot<WorkOrder>) **Propósito:** Raíz de consistencia del flujo operativo de taller. Encapsula las tareas planificadas, diagnóstico, millaje de ingreso, relaciones con vehículo, cliente y sede, calculando dinámicamente el monto total y despachando eventos de dominio ante cambios de estado.

Atributos del Agregado:

- id: WorkOrderId – Identificador único del agregado.
- appointmentId: AppointmentId – Cita de origen asociada.
- branchId: BranchId – Sede física donde se ejecuta el trabajo.
- vehicleId: VehicleId – Vehículo en mantenimiento.

- `customerId`: `CustomerId` – Cliente propietario.
- `internalNumber`: `Integer` – Correlativo operativo visible.
- `status`: `WorkOrderStatus` – Estado (DRAFT, IN_PROGRESS, COMPLETED, PAID, CANCELED).
- `diagnosticSummary`: `DiagnosticSummary` – Resumen de diagnóstico técnico.
- `mileageIn`: `Mileage` – Kilometraje de ingreso.
- `totalAmount`: `Money` – Monto acumulado de tareas y repuestos.
- `tasks`: `List<WorkOrderTask>` – Lista interna de tareas operativas.

Métodos y Comportamientos de Dominio:

- `addTask(ServiceId, MechanicId, TaskDescription, Money)`: Adiciona una tarea e incrementa el costo total.
- `addProductToTask(WorkOrderTaskId, ProductId, Quantity, Money)`: Registra el uso de un repuesto y despacha `WorkOrderTaskProductAddedEvent`.
- `startWork()` / `completeWorkOrder()` / `markAsPaid()`: Controlan el ciclo de vida y emiten eventos atómicos.

Service (Aggregate Root) Tipo: Aggregate Root de Catálogo **Propósito:** Define los servicios ofrecidos por el taller (ej. Cambio de Aceite, Alineación) con precio base y estado de vigencia.

Atributos: `id` (`ServiceId`), `name` (`ServiceName`), `description`, `price` (`Money`), `isActive` (`boolean`).

WorkOrderTask & WorkOrderTaskProduct (Entities) - `WorkOrderTask`: Entidad interna que representa un servicio ejecutado por un mecánico con estado (PENDING, IN_PROGRESS, COMPLETED).

- `WorkOrderTaskProduct`: Entidad de repuesto asociado a una tarea con cantidad y subtotal.

2.6.3.1.2. Value Objects & Records

Record: WorkOrderId(UUID value) **Propósito:** Identificador de orden de trabajo.

Record: ServiceId(UUID value) **Propósito:** Identificador del servicio de catálogo.

Record: TaskDescription(String value) **Propósito:** Detalle operativo de la tarea.

Record: Quantity(Integer value) **Propósito:** Unidades de repuestos consumidos.

2.6.3.1.3. Enumerations

Enum: WorkOrderStatus - DRAFT: Borrador en preparación.

- IN_PROGRESS: En ejecución en taller.

Enum: WorkOrderTaskStatus - PENDING: Tarea pendiente de asignación.

- COMPLETED: Trabajo finalizado.
- PAID: Facturado y cancelado.
- IN_PROGRESS: Mecánico ejecutando la labor.
- COMPLETED: Tarea concluida.

2.6.3.1.4. Domain Repositories (Interfaces)

- WorkOrderRepository: save(WorkOrder order), findById(WorkOrderId id), findAllByBranchId(BranchId branchId).
- ServiceRepository: save(Service service), findById(ServiceId id), findAllActive().

2.6.3.1.5. Domain Events

- WorkOrderCreatedEvent: Notifica la creación de una orden de trabajo.
- WorkOrderStartedEvent / WorkOrderCompletedEvent: Notifican inicio y cierre de trabajos.
- WorkOrderTaskProductAddedEvent: Emite la reserva de repuestos consumidos hacia Inventory.
- ServiceCreatedEvent / ServiceUpdatedEvent: Notifican modificaciones en el catálogo de servicios.

2.6.3.2. Application Layer (Capa de Aplicación)

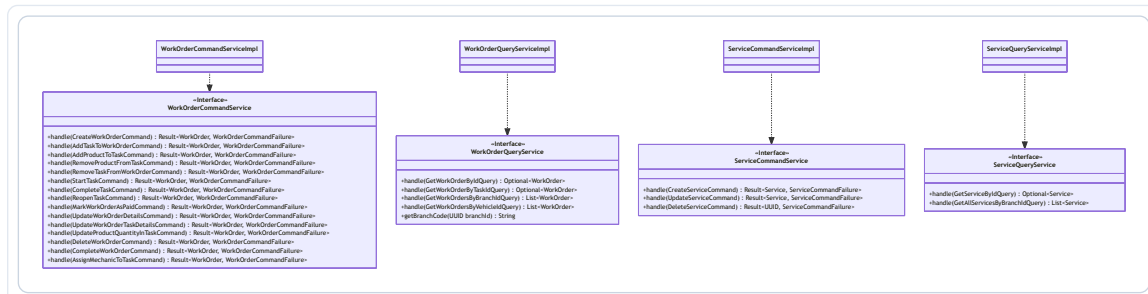


Figura 43: Diagrama de la Capa de Aplicación – Operations

2.6.3.2.1. Commands & Queries (DTOs de Aplicación)

Comandos de Escritura (CQRS Commands): - CreateWorkOrderCommand: Emisión de orden con cita y vehículo.

- AddTaskToWorkOrderCommand: Asignación de tarea a mecánico.
- AddProductToTaskCommand: Adición de repuesto consumido.
- StartWorkOrderCommand / CompleteWorkOrderCommand: Mutaciones de estado de orden.
- CreateServiceCommand / UpdateServiceCommand: Gestión de catálogo de servicios.

Consultas y DTOs de Resultado (CQRS Queries): - GetWorkOrderByIdQuery: Consulta por ID de orden.

- GetWorkOrdersByBranchIdQuery: Consulta de órdenes por sede.

- `GetServiceByIdQuery` / `GetAllActiveServicesQuery`: Consultas de servicios.

2.6.3.2.2. Application Services

WorkOrderCommandServiceImpl & ServiceCommandServiceImpl Servicios transaccionales (`@Service`, `@Transactional`) que coordinan las mutaciones del agregado `WorkOrder` y `Service`, persisten en repositorio y publican eventos de dominio.

2.6.3.3. Interface Layer (Capa de Interfaces)

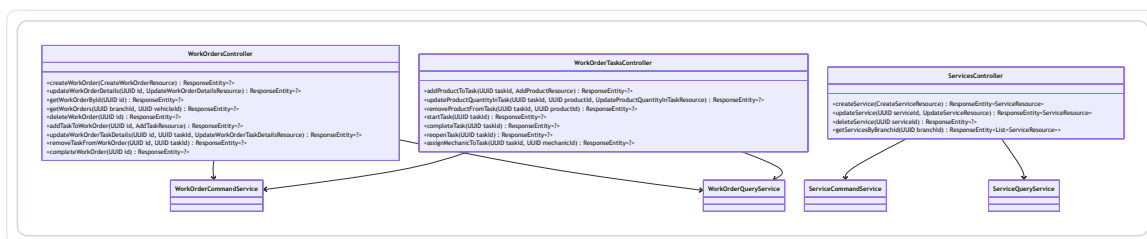


Figura 44: Diagrama de la Capa de Interfaces – Operations

2.6.3.3.1. REST Controllers & DTO Resources

- `WorkOrdersController` (`/api/v1/work-orders`): Endpoints REST para creación, flujo de estado, tareas y repuestos.
- `ServicesController` (`/api/v1/services`): Endpoints para administración del catálogo de servicios.

2.6.3.4. Infrastructure Layer (Capa de Infraestructura)

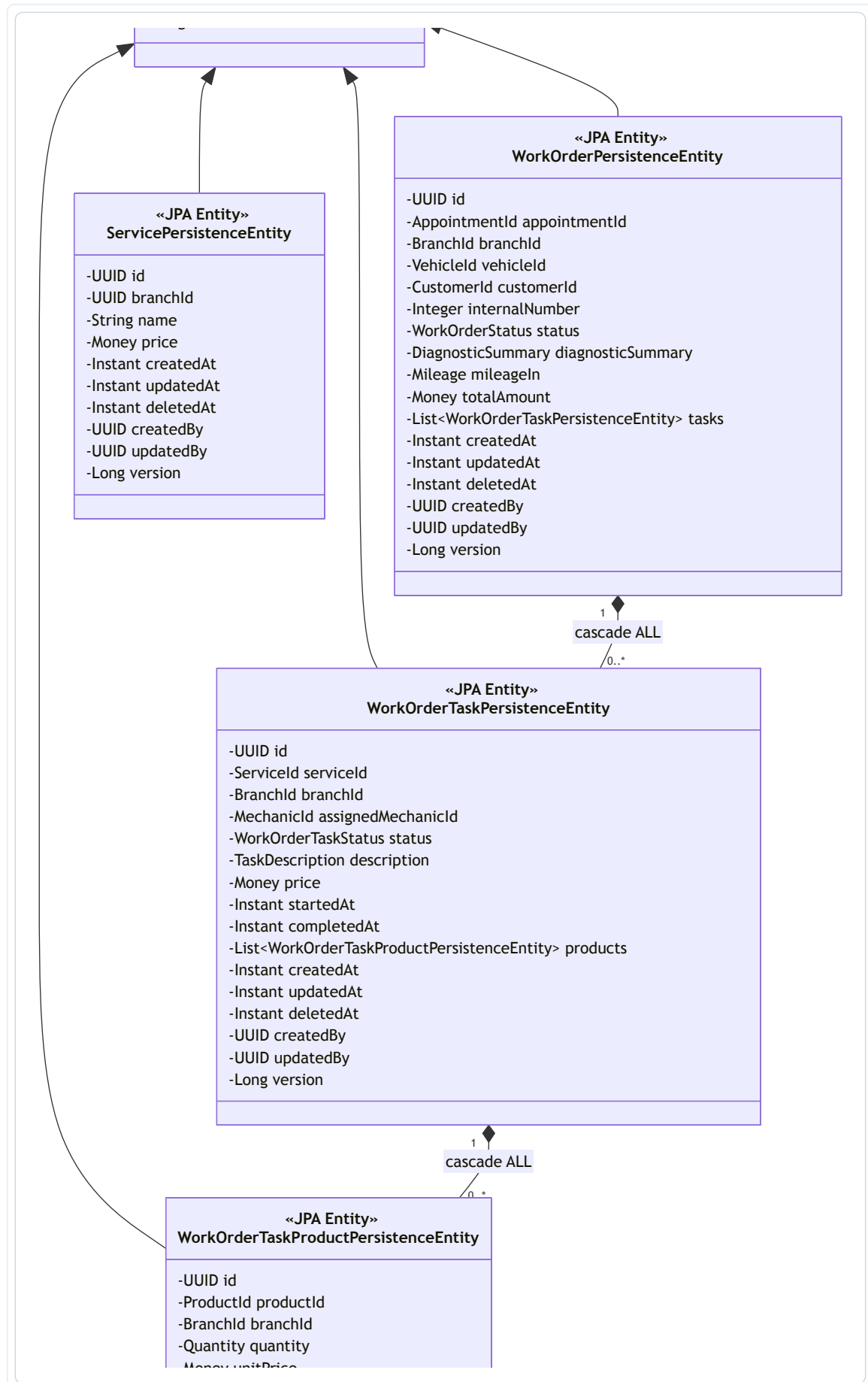


Figura 45: Diagrama de la Capa de Infraestructura – Operations

2.6.3.4.1. Persistence & Security Adapters

Tabla Relacional	Clase JPA Entity	Atributos & Campos Mapeados
work_orders	WorkOrderPersistenceEntity	id (UUID, PK), ap (UUID), branch_id (UUID), vehicle_id (UUID), customer_id (UUID), internal (INTEGER), status (VARCHAR), diagnost (TEXT), mileage_in (DECIMAL), total_amount (DECIMAL), created_at, updated_at, version
work_order_tasks	WorkOrderTaskPersistenceEntity	id (UUID, PK), work_order_id (UUID, FK), service_id (UUID), branch_id (UUID), assigned (UUID), status (VARCHAR), description (TEXT), price (DECIMAL), started_at, completed_at
work_order_task_products	WorkOrderTaskProductPersistenceEntity	id (UUID, PK), wc (UUID, FK), product_id (UUID),

Tabla Relacional	Clase JPA Entity	Atributos & Campos Mapeados
		branch_id (UUID), quantity (INTEGER), unit_price (DECIMAL), total_amount (DECIMAL)
services	ServicePersistenceEntity	id (UUID, PK), branch_id (UUID), name (VARCHAR), price (DECIMAL), created_at, updated_at, version

2.6.3.5. C4 Model Component Diagram (Container: Spring Boot REST API – Operations)

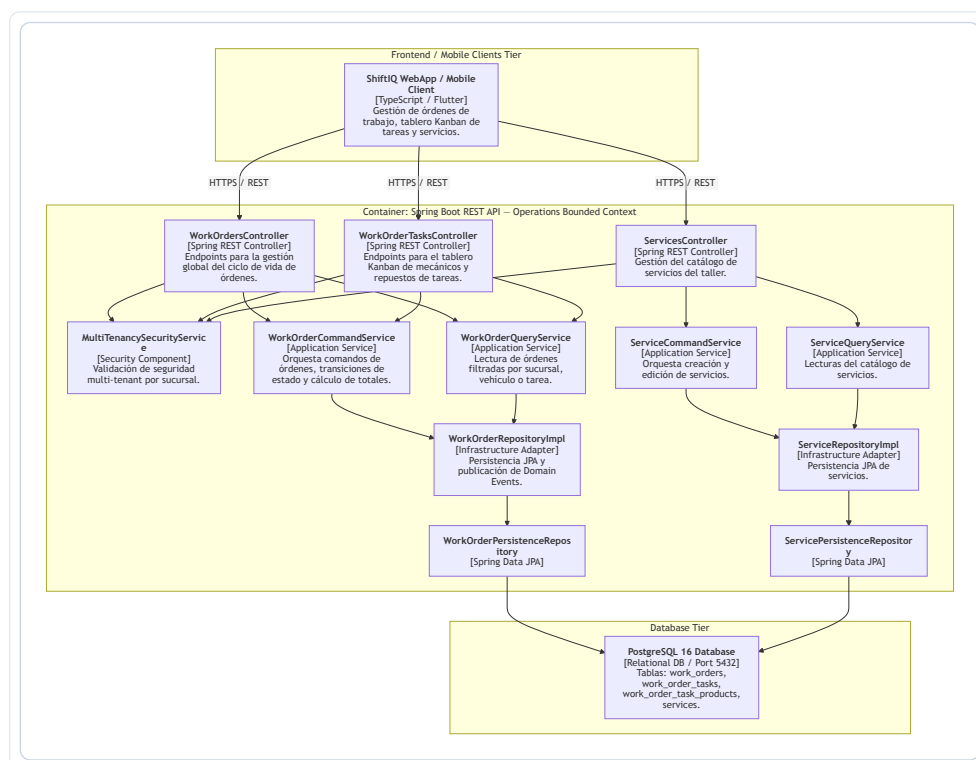


Figura 46: Diagrama de Componentes C4 Nivel 3 – Operations

2.6.3.6. Bounded Context Software Architecture Code Level Diagrams

2.6.3.6.1. Domain Layer Class Diagram

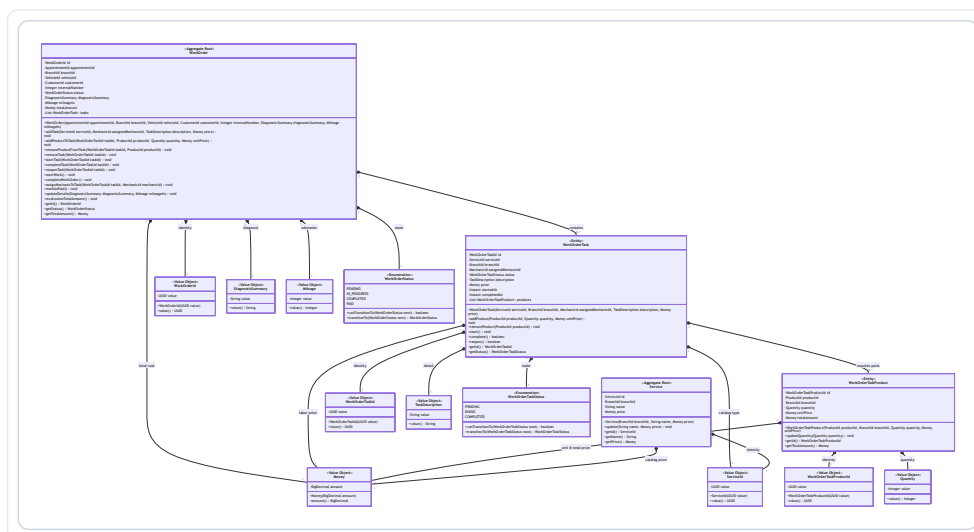


Figura 47: Diagrama de Clases del Dominio UML – Operations

2.6.3.6.2. Database Design Diagram (PostgreSQL 18)

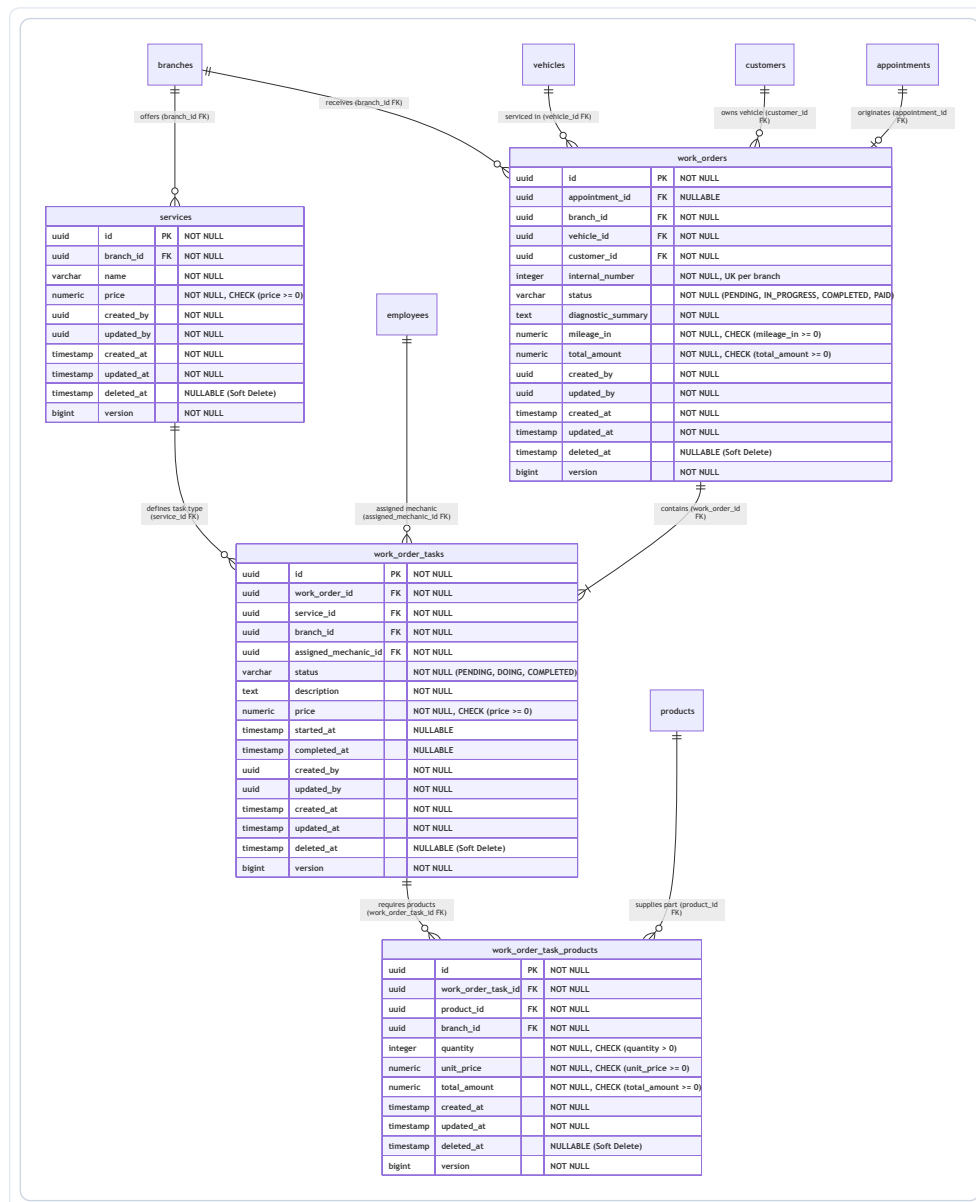


Figura 48: Diagrama de Base de Datos Relacional ER – Operations

Especificación de Tablas Relacionales (PostgreSQL 18):

Tabla: work_orders

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
branch_id	UUID	NOT NULL, FOREIGN KEY -> branches(id)
vehicle_id	UUID	NOT NULL, FOREIGN KEY -> vehicles(id)
customer_id	UUID	NOT NULL, FOREIGN KEY -> customers(id)
status	VARCHAR(20)	NOT NULL (DRAFT/IN_PROGRESS/COMPLETED/PAID)

Columna	Tipo de Dato	Constraints / Descripción
total_amount	DECIMAL(12,2)	NOT NULL, DEFAULT 0.00
created_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
updated_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
version	BIGINT	NOT NULL, DEFAULT 0

Tabla: services

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
name	VARCHAR(100)	NOT NULL, UNIQUE
price	DECIMAL(12,2)	NOT NULL, DEFAULT 0.00
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE

2.6.4. Bounded Context: Inventory (Stock & Products Management)

El **Bounded Context Inventory** administra el catálogo de repuestos, autopartes y consumibles del taller automotriz (**Product**), la gestión física de existencias mediante lotes de adquisición (**ProductBatch**), la evaluación automática de niveles de stock mínimo (**MinimumStockAlertEvaluationJob**), y la sincronización asíncrona de inventario respondiendo a las reservas y despachos producidos por las Órdenes de Trabajo del Bounded Context Operations.

2.6.4.1. Domain Layer (Capa de Dominio)

La Capa de Dominio define las reglas inmutables del inventario, gestionando el stock disponible, la deducción FIFO encapsulada en el agregado **Product**, los métodos de creación y reconstitución (patrón **Factory**), la activación de alertas de bajo stock y las validaciones de negocio sin dependencias tecnológicas externas.

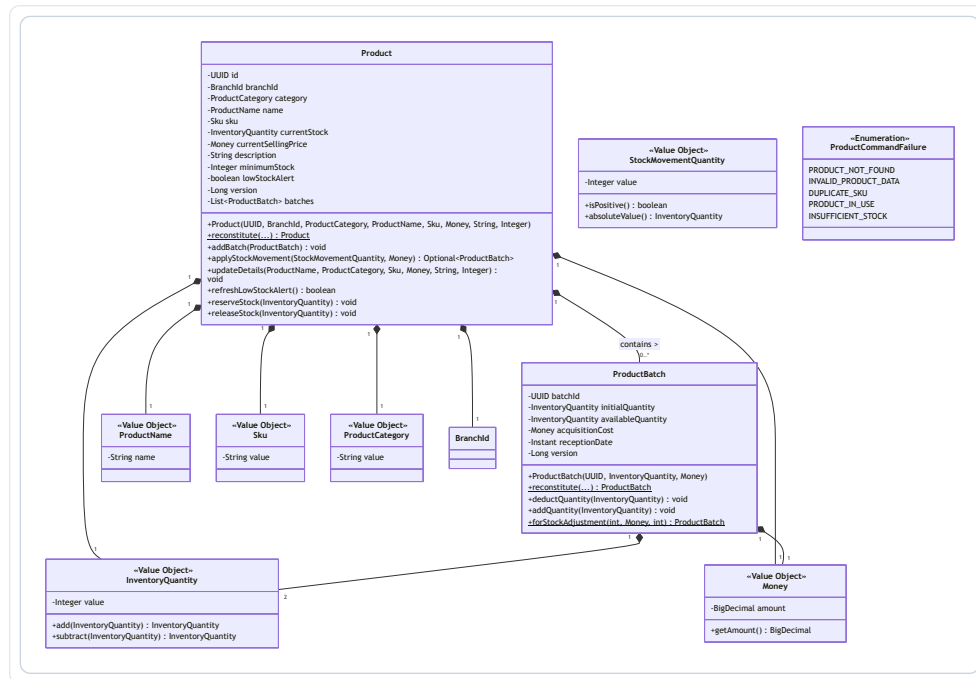


Figura 49: Diagrama de la Capa de Dominio – Inventory

2.6.4.1.1. Value Objects, Enums & Exceptions

Record: BranchId

Propósito: Identificador único fuertemente tipado de la sucursal de taller asociada al inventario.

Validaciones: No nulo.

Record: Money

Propósito: Representa montos monetarios para precios de venta y costos de adquisición de lotes.

Validaciones: amount no nulo y ≥ 0 .

Métodos: `getAmount()`.

Record: ProductName

Propósito: Nombre comercial de la autoparte o repuesto.

Validaciones: No nulo ni en blanco (`inventory.error.productName.required`).

Record: Sku

Propósito: Stock Keeping Unit (código único de producto por sucursal).

Validaciones: No nulo ni en blanco (`inventory.error.sku.required`).

Record: ProductCategory

Propósito: Categoría o familia del producto (ej. “Frenos”, “Filtros”, “Lubricantes”).

Validaciones: No nulo ni en blanco (`inventory.error.productCategory.required`).

Record: InventoryQuantity

Propósito: Cantidad entera no negativa en inventario.

Validaciones: No nulo y ≥ 0 .

Métodos: `add()`, `subtract()`.

Record: StockMovementQuantity(Integer value)

Propósito: Movimiento o ajuste de inventario (positivo para ingresos, negativo para salidas).

egresos).

Validaciones: No nulo y distinto de cero. **Métodos:** `isPositive()`, `absoluteValue()`.

Enum: `ProductCommandFailure`

Valores: `PRODUCT_NOT_FOUND`, `INVALID_PRODUCT_DATA`, `DUPLICATE_SKU`, `PRODUCT_IN_USE`, `INSUFFICIENT_STOCK`.

Excepción: `InsufficientStockException`

Propósito: Excepción de dominio lanzada cuando se intenta reservar o descontar más stock del disponible (`inventory.error.product.insufficientStock`).

2.6.4.1.2. Aggregates & Entities

Aggregate Root: `Product`

Hereda de: `AbstractAggregateRoot<Product>`

Propósito: Raíz del agregado que representa un producto del inventario en una sucursal (`BranchId`). Clave primaria `UUID id`.

Reglas de Negocio:

- Mantiene la lista de lotes físicos recibidos (`batches`).
- `reserveStock(InventoryQuantity amount)`: Recorre los lotes activos (`ProductBatch`) en estricto orden FIFO (`receptionDate` ascendente) descontando existencias. Lanza `InsufficientStockException` si `currentStock < amount`.
- `releaseStock(InventoryQuantity amount)`: Reingresa existencias a los lotes en caso de cancelación de reserva.
- `refreshLowStockAlert()`: Compara `currentStock <= minimumStock`. Si el estado cambia, emite `LowStockAlertTriggeredEvent` o `LowStockAlertClearedEvent`.

Entity: `ProductBatch`

Propósito: Entidad de dominio que representa un lote físico recibido con costo de adquisición y fecha de recepción. Clave primaria `UUID batchId`.

Atributos: `batchId` (`UUID`), `initialQuantity` (`InventoryQuantity`), `availableQuantity` (`InventoryQuantity`), `acquisitionCost` (`Money`), `receptionDate` (`Instant`), `version` (`Long`).

Comportamiento: `deductQuantity` y `addQuantity` actualizan `availableQuantity`.

2.6.4.1.3. Creation & Reconstitution Methods (Factory Pattern)

- **Constructor Público `Product(...)`:** `public Product(UUID id, BranchId branchId, ProductCategory category, ProductName name, Sku sku, Money currentSellingPrice, String description, Integer minimumStock)`. Asigna `UUID.randomUUID()` si `id` es nulo, inicializa `currentStock = 0`, `lowStockAlert = false` y publica `ProductCreatedEvent`.
- **`Product.reconstitute(...)`:** `public static Product reconstitute(UUID id, BranchId branchId, ProductCategory category, ProductName name, Sku sku, InventoryQuantity currentStock, Money currentSellingPrice, String description,`

Integer minimumStock, boolean lowStockAlert, Long version, List<ProductBatch> batches). Reconstruye el agregado desde infraestructura sin publicar eventos.

- **ProductBatch.forStockAdjustment(...):** public static ProductBatch forStockAdjustment(int signedQuantity, Money acquisitionCost, int resultingStock). Reconstituye un ProductBatch para ajustes manuales de almacén asignando initialQuantity = 0 y availableQuantity = resultingStock.

2.6.4.1.4. Domain Events

- ProductCreatedEvent: Notifica la creación de un nuevo producto en una sucursal.
- ProductUpdatedEvent: Notifica la actualización de los datos del producto.
- StockMovementAppliedEvent: Notifica la aplicación de un movimiento de stock manual o lote.
- StockReservedEvent: Notifica la reserva exitosa de stock solicitada desde Operations.
- StockReleasedEvent: Notifica la liberación de stock previamente reservado.
- LowStockAlertTriggeredEvent: Notifica cuando el stock cae por debajo del mínimo configurado.
- LowStockAlertClearedEvent: Notifica cuando el stock se recupera por encima del mínimo.

2.6.4.1.5. Domain Repositories (Interfaces)

ProductRepository (Domain Repository Interface)

Métodos de Contrato:

- Product save(Product product)
- Optional<Product> findById(UUID id)
- List<Product> findAllByBranchId(BranchId branchId)
- List<Product> findAllByBranchIdWithFilters(BranchId branchId, String name, String category, Boolean lowStockOnly)
- List<Product> findAll()
- boolean existsByBranchIdAndSku(BranchId branchId, String sku)
- boolean existsByBranchIdAndSkuAndIdNot(BranchId branchId, String sku, UUID productId)
- boolean existsById(UUID id)
- void deleteById(UUID id)

2.6.4.2. Application Layer (Capa de Aplicación)

La Capa de Aplicación expone la ejecución de casos de uso utilizando Result<T, ProductCommandFailure> para manejo funcional de fallos y ejecuta tareas programadas para la evaluación continua de alertas.

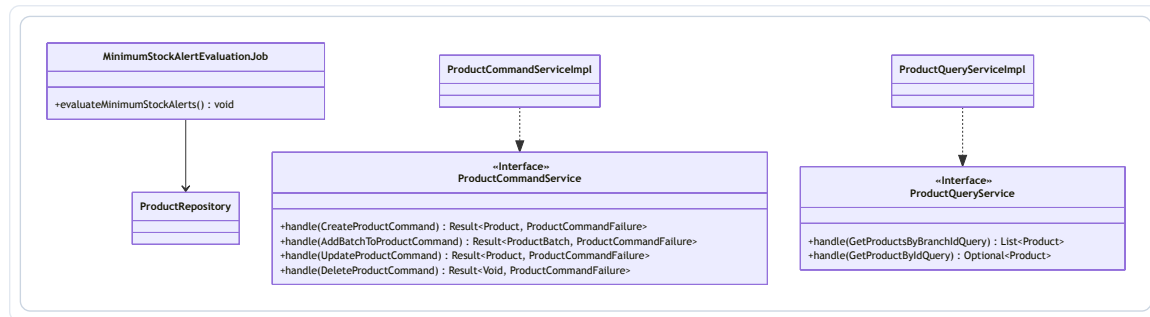


Figura 50: Diagrama de la Capa de Aplicación – Inventory

2.6.4.2.1. Commands & Queries (DTOs de Aplicación)

Comandos de Escritura (CQRS Commands):

- CreateProductCommand(BranchId branchId, ProductCategory category, ProductName name, Sku sku, String description, Money salePrice, InventoryQuantity minimumStock)
- UpdateProductCommand(UUID productId, ProductName name, ProductCategory category, Sku sku, String description, Money salePrice, InventoryQuantity minimumStock)
- DeleteProductCommand(UUID productId)
- AddBatchToProductCommand(UUID productId, StockMovementQuantity quantity, Money acquisitionCost)

Consultas y DTOs de Resultado (CQRS Queries):

- GetProductByIdQuery(UUID productId)
- GetProductsByBranchIdQuery(BranchId branchId, String name, String category, Boolean lowStockOnly)

2.6.4.2.2. Capabilities & Scheduled Tasks

MinimumStockAlertEvaluationJob (Scheduled Task / Job)

Tipo: Proceso de fondo programado (@Scheduled(cron = "0 0 * * * *")).

Responsabilidad: Inspecciona el estado de existencias de todos los productos por sucursal en la base de datos, invocando directamente product.refreshLowStockAlert() en cada agregado para actualizar el indicador lowStockAlert y publicar eventos LowStockAlertTriggeredEvent cuando el stock disponible cae por debajo de la reserva mínima configurada.

2.6.4.3. Interface Layer (Capa de Interfaz / REST & Events)

Exposición RESTful e integración asíncrona mediante listeners de eventos producidos por otros Bounded Contexts.

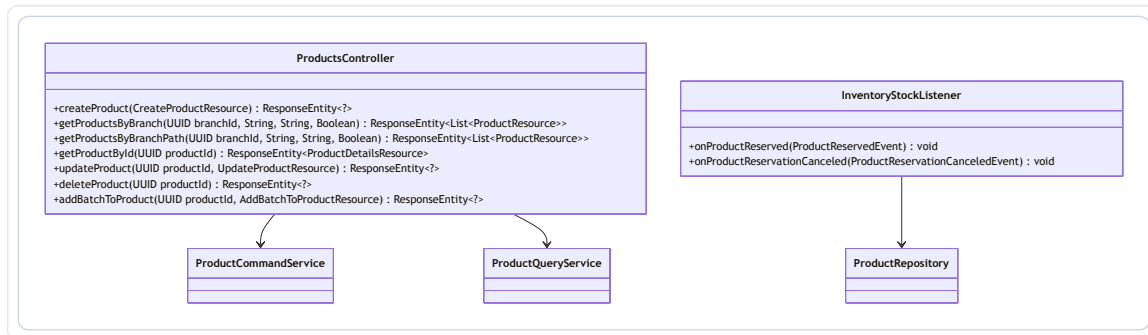


Figura 51: Diagrama de la Capa de Interfaz – Inventory

2.6.4.3.1. Endpoints & REST Controllers

ProductsController (/api/v1/inventory/products)

- POST /api/v1/inventory/products: Registra un nuevo producto en el inventario de una sucursal.
- GET /api/v1/inventory/products?branchId={branchId}: Consulta productos por sucursal con filtros opcionales (name, category, lowStockOnly).
- GET /api/v1/inventory/products/branch/{branchId}: Catálogo de productos por ruta de sucursal.
- GET /api/v1/inventory/products/{productId}: Consulta detalles completos de un producto incluyendo sus lotes.
- PUT /api/v1/inventory/products/{productId}: Actualiza información básica del producto.
- DELETE /api/v1/inventory/products/{productId}: Eliminación lógica (soft-delete vía deleted_at) del producto y sus lotes.
- POST /api/v1/inventory/products/{productId}/batches: Registra la entrada de un nuevo lote de stock o un ajuste manual de almacén.

Event Listener: InventoryStockListener

- Escucha ProductReservedEvent proveniente de Operations e invoca product.reserveStock(amount).
- Escucha ProductReservationCanceledEvent proveniente de Operations e invoca product.releaseStock(amount).

2.6.4.4. Infrastructure Layer (Capa de Infraestructura)

Mapeo ORM relacional a PostgreSQL 18 con Spring Data JPA y configuración de Jobs programados con @EnableScheduling.

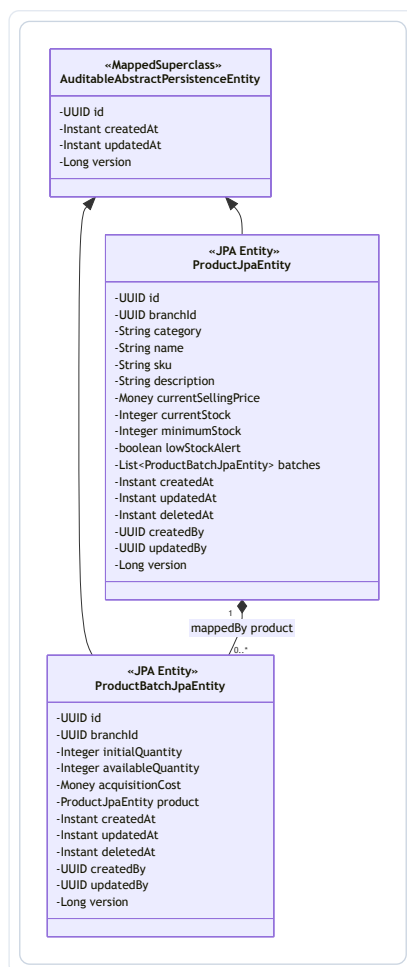


Figura 52: Diagrama de la Capa de Infraestructura – Inventory

2.6.4.4.1. Mapeo de Entidades Relacionales (JPA)

Tabla: products (ProductJpaEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
branch_id	UUID	NOT NULL, FOREIGN KEY -> branches(id)
category	VARCHAR(100)	NOT NULL
name	VARCHAR(150)	NOT NULL
sku	VARCHAR(100)	NOT NULL, UNIQUE
description	TEXT	NULLABLE
current_selling_price	NUMERIC(12,2)	NOT NULL, CHECK (current_selling_price >= 0)
current_stock	INTEGER	NOT NULL, CHECK (current_stock >= 0)
minimum_stock	INTEGER	NOT NULL, CHECK (minimum_stock >= 0)
low_stock_alert	BOOLEAN	NOT NULL DEFAULT false

Columna	Tipo de Dato	Constraints / Descripción
created_by	UUID	NOT NULL
updated_by	UUID	NOT NULL
created_at	TIMESTAMP	NOT NULL
updated_at	TIMESTAMP	NOT NULL
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)
version	BIGINT	NOT NULL

Tabla: product_batches (ProductBatchJpaEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
product_id	UUID	NOT NULL, FOREIGN KEY -> products(id)
branch_id	UUID	NOT NULL, FOREIGN KEY -> branches(id)
initial_quantity	INTEGER	NOT NULL, CHECK (initial_quantity > 0)
available_quantity	INTEGER	NOT NULL, CHECK (available_quantity >= 0)
acquisition_cost	NUMERIC(12,2)	NOT NULL, CHECK (acquisition_cost >= 0)
created_by	UUID	NOT NULL
updated_by	UUID	NOT NULL
created_at	TIMESTAMP	NOT NULL
updated_at	TIMESTAMP	NOT NULL
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)
version	BIGINT	NOT NULL

2.6.4.4.2. Repository Adapters & Infrastructure Components

ProductRepositoryAdapter (Infrastructure Repository Adapter)

Implementa: Contrato de dominio ProductRepository.

Inyecta: ProductJpaRepository (Spring Data JPA).

Responsabilidad: Provee persistencia relacional con aislamiento total del modelo de dominio. Realiza el mapeo bidireccional entre los agregados de dominio (Product, ProductBatch) y las entidades de persistencia JPA (ProductJpaEntity, ProductBatchJpaEntity).

Métodos Clave: save(), findById(), findAllByBranchId(), findAllByBranchIdWithFilters(), existsByBranchIdAndSku(), deleteById().

2.6.4.5. Software Architecture Component Level Diagrams (C4 Model - Level 3)

El siguiente diagrama C4 descompone el Container API en sus componentes principales para el Bounded Context **Inventory**.

2.6.4.5.1. C4 Model Component Diagram (Container: Spring Boot REST API – Inventory)

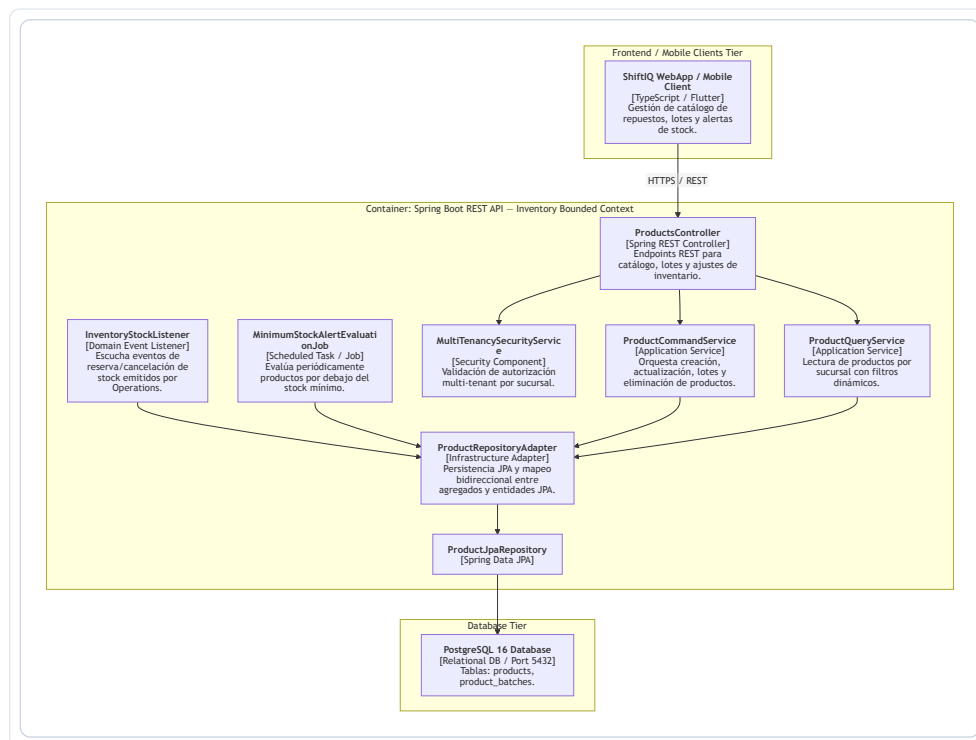


Figura 53: Diagrama de Componentes C4 Nivel 3 – Inventory

2.6.4.5.2. Descomposición y Responsabilidad de Componentes

- **ProductsController (Interface Layer):** Expone los endpoints RESTful para la creación, consulta filtrada, actualización y eliminación de productos y registro de lotes. **(Spring Web MVC, REST over HTTPS, Jackson JSON).**
- **InventoryStockListener (Interface Layer):** Escucha asíncronamente eventos de dominio emitidos por Operations (ProductReservedEvent, ProductReservationCanceledEvent) e invoca reglas de reserva. **(Spring Application Events).**
- **MinimumStockAlertEvaluationJob (Application Layer):** Capability programada en segundo plano que evalúa los umbrales de stock mínimo invocando `product.refreshLowStockAlert()`. **(Spring Scheduled Tasks).**
- **ProductCommandService (Application Layer):** Orquesta los comandos de creación, edición, borrado de productos y adición de lotes de inventario. **(Spring Service, Functional Result<T, E>).**
- **ProductQueryService (Application Layer):** Ejecuta consultas filtradas por sucursal, categoría y estado de alerta de bajo stock. **(Spring Service, Read-only Transactions).**

- **ProductRepositoryAdapter (Infrastructure Layer):** Adaptador de infraestructura que mapea agregados y entidades de dominio hacia/desde entidades relacionales JPA. (Spring Component, JPA Hibernate Mapping).
- **ProductJpaRepository (Infrastructure Layer):** Repositorio Spring Data JPA que interactúa directamente con PostgreSQL 18. (Spring Data JPA, Hibernate ORM, SQL Native Queries).

2.6.4.6. Bounded Context Software Architecture Code Level Diagrams

2.6.4.6.1. Domain Layer Class Diagram

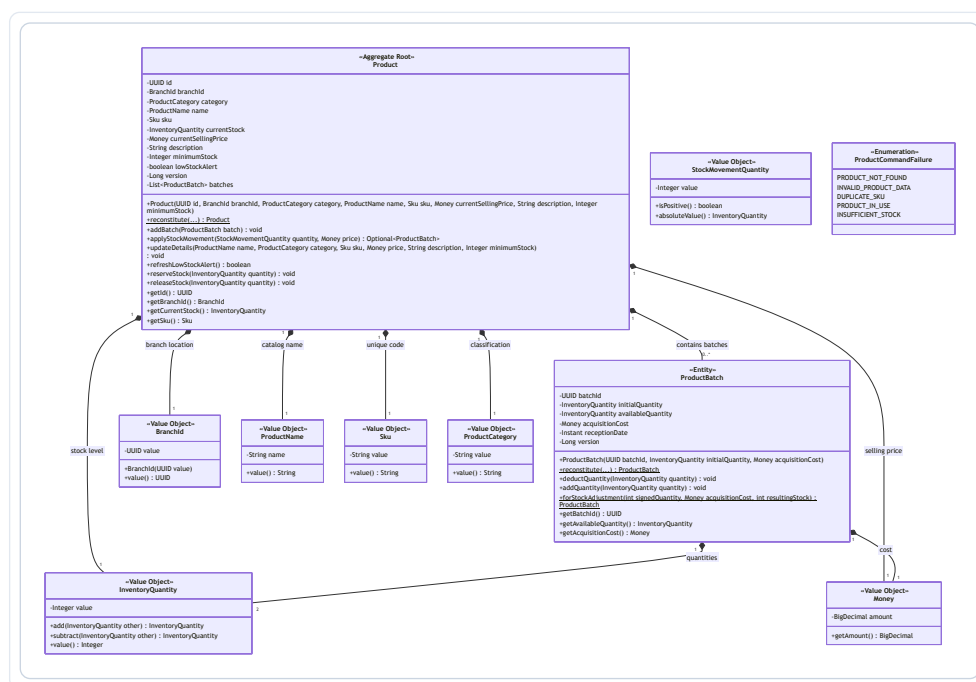


Figura 54: Diagrama de Clases del Dominio UML – Inventory

2.6.4.6.2. Database Design Diagram (PostgreSQL 18)

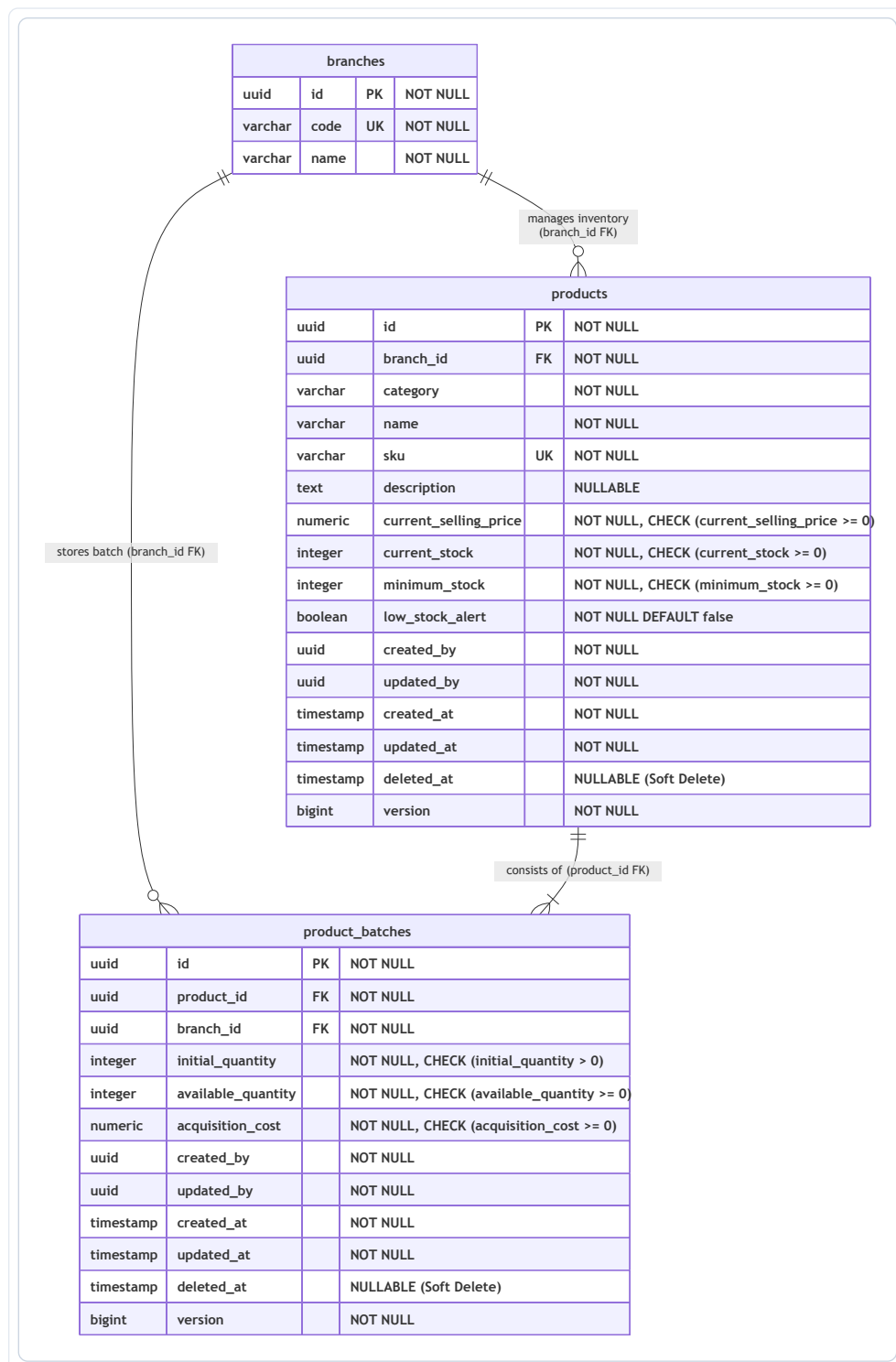


Figura 55: Diagrama de Base de Datos Relacional ER – Inventory

2.6.5. Bounded Context: IoT (Telemetry, Vehicles & OBD-II Devices)

El **Bounded Context IoT** administra la identidad telemática de los vehículos (Vehicle), la vinculación con sus conductores/proprietarios (VehicleRegistration), el inventario y estado

operativo de escáneres telemáticos OBD2 (Obd2Device), el emparejamiento activo entre escáneres y vehículos (Obd2DeviceRegistration), la ingesta remota de ráfagas telemáticas (TelemetrySnapshot), y la gestión inmutable de alertas por códigos de error computarizados (DtcAlert / Diagnostic Trouble Codes).

2.6.5.1. Domain Layer (Capa de Dominio)

La Capa de Dominio rige las reglas de lectura e ingesta telemática, constructores de dominio para la instanciación garantizada de Agregados (Vehicle, Obd2Device, TelemetrySnapshot), el servicio de contexto ActiveRegistrationContextService, la vinculación inmutable de dispositivos OBD2 con vehículos, las alertas automáticas según la gravedad de los códigos DTC (LOW, MEDIUM, HIGH, CRITICAL) y la validación de vin/placa vehicular.

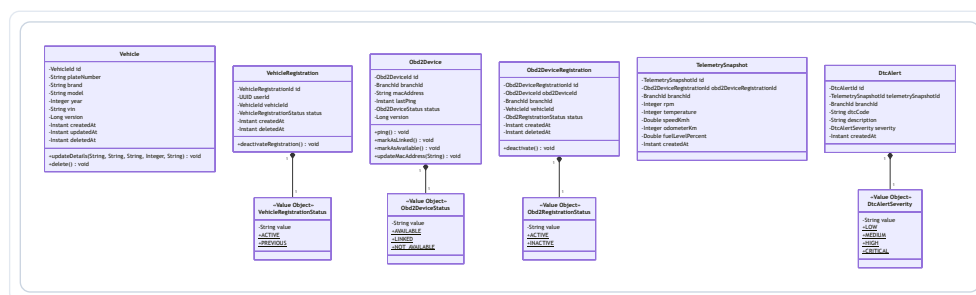


Figura 56: Diagrama de la Capa de Dominio – IoT

2.6.5.1.1. Value Objects, Enums & Exceptions

Record: Obd2DeviceStatus

Propósito: Estado de disponibilidad física de un escáner OBD2 en la sucursal.

Valores: AVAILABLE, LINKED, NOT_AVAILABLE.

Record: Obd2RegistrationStatus

Propósito: Estado del acoplamiento entre un escáner OBD2 y un vehículo.

Valores: ACTIVE, INACTIVE.

Record: VehicleRegistrationStatus

Propósito: Estado de la vinculación entre un usuario conductor y un vehículo.

Valores: ACTIVE, PREVIOUS.

Record: DtcAlertSeverity

Propósito: Severidad del código de error de diagnóstico telemático.

Valores: LOW, MEDIUM, HIGH, CRITICAL.

2.6.5.1.2. Aggregates & Entities

- **Aggregate Root Vehicle:** Representa un automóvil del parque vehicular. Valida placa, marca, modelo, VIN y año (1900 a año actual + 1). Emite VehicleDetailsUpdatedEvent.
- **Aggregate Root VehicleRegistration:** Enlace entre un conductor (userId) y vehículo (vehicleId). Método deactivateRegistration() emite VehicleRegistrationDeactivatedEvent.

- **Aggregate Root Obd2Device:** Dispositivo físico de diagnóstico registrado en sucursal. Métodos `ping()`, `markAsLinked()`, `markAsAvailable()`. Emite `Obd2DeviceStatusChangedEvent`.
- **Aggregate Root Obd2DeviceRegistration:** Emparejamiento activo escáner-vehículo. Método `deactivate()` emite `Obd2DeviceRegistrationDeactivatedEvent`.
- **Aggregate Root TelemetrySnapshot:** Captura puntual e inmutable de parámetros de motor (RPM, temperatura, velocidad km/h, odómetro, combustible %).
- **Aggregate Root DtcAlert:** Alerta de falla computarizada generada por el escáner (código DTC, descripción y severidad). Emite `DtcAlertTriggeredEvent`.

2.6.5.1.3. Domain Events

- `VehicleDetailsUpdatedEvent`: Notifica actualización de datos de un vehículo.
- `VehicleRegistrationDeactivatedEvent`: Notifica desvinculación de un conductor con un vehículo.
- `Obd2DeviceStatusChangedEvent`: Notifica cambios de estado de un escáner OBD2 (AVAILABLE / LINKED).
- `Obd2DeviceRegistrationDeactivatedEvent`: Notifica la desvinculación de un escáner OBD2 de un vehículo.
- `DtcAlertTriggeredEvent`: Notifica la detección de una falla telemática DTC en tiempo real.

2.6.5.1.4. Domain Repositories (Interfaces)

- **VehicleRepository:** `save()`, `findById()`, `findByVin()`, `findByPlateNumber()`, `delete()`, `findAllByIds()`.
- **VehicleRegistrationRepository:** `save()`, `findActiveByVehicleId()`, `findAllActiveByUserId()`, `findAllActiveByUserIds()`.
- **Obd2DeviceRepository:** `save()`, `findById()`, `findByMacAddress()`, `existsByMacAddress()`, `delete()`, `findAllByBranchId()`, `findAllByBranchIdAndStatus()`.
- **Obd2DeviceRegistrationRepository:** `save()`, `findById()`, `findActiveByObd2DeviceId()`, `findActiveByVehicleId()`, `findAllByBranchIdAndStatus()`, `findVehicleIdsWithActiveRegistration()`.
- **TelemetrySnapshotRepository:** `save()`, `saveAll()`, `findById()`, `findLatestByRegistrationId()`, `findAllByRegistrationId()`, `findAllByRegistrationIdAndCreatedAtGreaterThanOrEqualTo()`.
- **DtcAlertRepository:** `save()`, `saveAll()`, `findAllByRegistrationId()`, `findAllByRegistrationIdAndCreatedAtGreaterThanOrEqualTo()`.

2.6.5.2. Application Layer (Capa de Aplicación)

La Capa de Aplicación expone la ingesta de telemetría, emparejamiento de escáneres y consulta de alertas de motor.



Figura 57: Diagrama de la Capa de Aplicación – IoT

2.6.5.2.1. Commands & Queries (DTOs de Aplicación)

Comandos de Escritura (CQRS Commands):

- RegisterVehicleCommand, UpdateVehicleCommand, DeleteVehicleCommand
- CreateObd2DeviceCommand, UpdateObd2DeviceCommand, DeleteObd2DeviceCommand
- LinkObd2DeviceToVehicleCommand, DeactivateObd2DeviceRegistrationCommand
- IngestTelemetryBatchCommand

Consultas y DTOs de Resultado (CQRS Queries):

- GetVehicleByIdQuery, GetActiveVehiclesByCustomerIdQuery, GetVehiclesAvailableForLinkingQuery
- GetObd2DeviceByIdQuery, GetObd2DevicesByBranchIdQuery, GetAvailableObd2DevicesQuery
- GetObd2DeviceRegistrationsByBranchIdAndStatusQuery
- GetLatestTelemetrySnapshotQuery, GetTelemetrySnapshotHistoryQuery, GetTelemetrySnapshotsByRegistrationIdQuery, GetVehicleTelemetrySnapshotHistoryQuery
- GetDtcAlertsByRegistrationIdQuery, GetVehicleDtcAlertHistoryQuery

2.6.5.2.2. Command Failure ADTs (Sealed Interfaces)

- VehicleCommandFailure: NotFound, InvalidState, Duplicate
- Obd2DeviceCommandFailure: NotFound, InvalidState, Duplicate
- Obd2DeviceRegistrationCommandFailure: NotFound, InvalidState
- TelemetryCommandFailure: NotFound, InvalidState

2.6.5.3. Interface Layer (Capa de Interfaz / REST)

Exposición RESTful para ingesta telemática, alertas de motor y catálogo de vehículos.

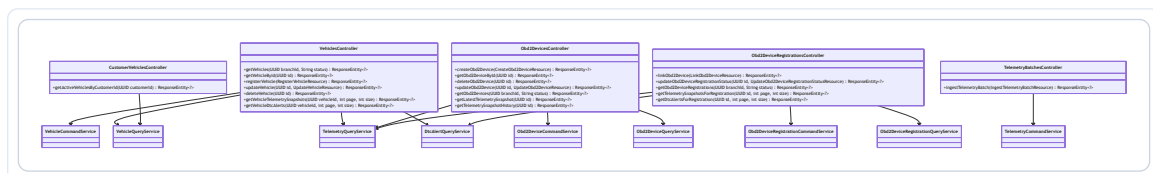


Figura 58: Diagrama de la Capa de Interfaz – IoT

2.6.5.3.1. Endpoints & REST Controllers

- **VehiclesController (/api/v1/vehicles):** GET / (filtro catálogo), GET /{id}, POST / (registro), PUT /{id}, DELETE /{id}, GET /{vehicleId}/telemetry-snapshots, GET /{vehicleId}/dtc-alerts.
- **CustomerVehiclesController (/api/v1/customers/{customerId}/vehicles):** GET / (vehículos activos del cliente).
- **Obd2DevicesController (/api/v1/obd2-devices):** POST /, GET /{id}, DELETE /{id}, PUT /{id}, GET / (filtro sucursal), GET /{id}/telemetry-snapshots/latest, GET /{id}/telemetry-snapshots.
- **Obd2DeviceRegistrationsController (/api/v1/obd2-device-registrations):** POST / (vincular), PATCH /{id} (desactivar), GET / (listar sucursal/estado), GET /{id}/telemetry-snapshots, GET /{id}/dtc-alerts.
- **TelemetryBatchesController (/api/v1/telemetry-batches):** POST / (ingesta masiva de hardware OBD2).

2.6.5.4. Infrastructure Layer (Capa de Infraestructura)

Mapeo relacional JPA a PostgreSQL 18 con adaptadores de repositorio, ensambladores de persistencia y listeners de eventos de dominio.

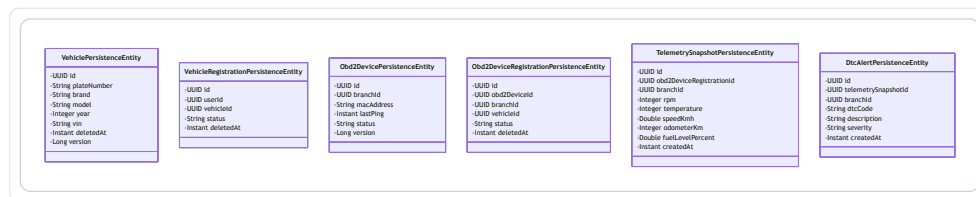


Figura 59: Diagrama de la Capa de Infraestructura – IoT

2.6.5.4.1. Mapeo de Entidades Relacionales (JPA)

Tabla: vehicles (VehiclePersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
plate_number	VARCHAR(20)	NOT NULL
brand	VARCHAR(50)	NOT NULL
model	VARCHAR(50)	NOT NULL
year	INTEGER	NOT NULL
vin	VARCHAR(50)	NOT NULL
created_at	TIMESTAMP	NOT NULL
updated_at	TIMESTAMP	NOT NULL
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)

Columna	Tipo de Dato	Constraints / Descripción
version	BIGINT	NOT NULL

Tabla: vehicle_registrations (VehicleRegistrationPersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
user_id	UUID	NOT NULL
vehicle_id	UUID	NOT NULL, FOREIGN KEY -> vehicles(id)
status	VARCHAR(20)	NOT NULL (ACTIVE / PREVIOUS)
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)

Tabla: obd2_devices (Obd2DevicePersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
branch_id	UUID	NOT NULL, FOREIGN KEY -> branches(id)
mac_address	VARCHAR(50)	NOT NULL, UNIQUE
last_ping	TIMESTAMP	NULLABLE
status	VARCHAR(20)	NOT NULL (AVAILABLE / LINKED)

Tabla: obd2_device_registrations (Obd2DeviceRegistrationPersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
obd2_device_id	UUID	NOT NULL, FOREIGN KEY -> obd2_devices(id)
branch_id	UUID	NOT NULL, FOREIGN KEY -> branches(id)
vehicle_id	UUID	NOT NULL, FOREIGN KEY -> vehicles(id)
status	VARCHAR(20)	NOT NULL (ACTIVE / INACTIVE)
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)

Tabla: telemetry_snapshots (TelemetrySnapshotPersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
obd2_device_registration_id	UUID	NOT NULL, FOREIGN KEY -> obd2_device_registrations(id)

Columna	Tipo de Dato	Constraints / Descripción
branch_id	UUID	NOT NULL
rpm	INTEGER	NOT NULL
temperature	INTEGER	NOT NULL
speed_kmh	DOUBLE PRECISION	NOT NULL
odometer_km	INTEGER	NOT NULL
fuel_level_percent	DOUBLE PRECISION	NOT NULL
created_at	TIMESTAMP	NOT NULL

Tabla: dtc_alerts (DtcAlertPersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
telemetry_snapshot_id	UUID	NOT NULL, FOREIGN KEY -> telemetry_snapshots(id)
branch_id	UUID	NOT NULL
dtc_code	VARCHAR(20)	NOT NULL
description	TEXT	NULLABLE
severity	VARCHAR(20)	NOT NULL (LOW/MEDIUM/HIGH/CRITICAL)
created_at	TIMESTAMP	NOT NULL

2.6.5.4.2. Adapters & Infrastructure Components

- **Persistence Adapters:** VehicleRepositoryImpl, VehicleRegistrationRepositoryImpl, Obd2DeviceRepositoryImpl, Obd2DeviceRegistrationRepositoryImpl, TelemetrySnapshotRepositoryImpl, DtcAlertRepositoryImpl.
- **Assemblers de Persistencia:** VehiclePersistenceAssembler, VehicleRegistrationPersistenceAssembler, Obd2DevicePersistenceAssembler, Obd2DeviceRegistrationPersistenceAssembler, TelemetrySnapshotPersistenceAssembler, DtcAlertPersistenceAssembler.
- **Outbound Services / Ports:** CustomerDirectoryPortImpl (ACL para validación de clientes en Core), ActiveRegistrationContextServiceImpl (resolución de contexto activo escáner-vehículo).
- **Event Listeners:** DtcAlertEventListener (escucha DtcAlertTriggeredEvent para integración con Órdenes de Trabajo).

2.6.5.5. Software Architecture Component Level Diagrams (C4 Model - Level 3)

Descomposición del Container API en sus componentes principales para el Bounded Context IoT.

2.6.5.5.1. C4 Model Component Diagram (Container: Spring Boot REST API – IoT)

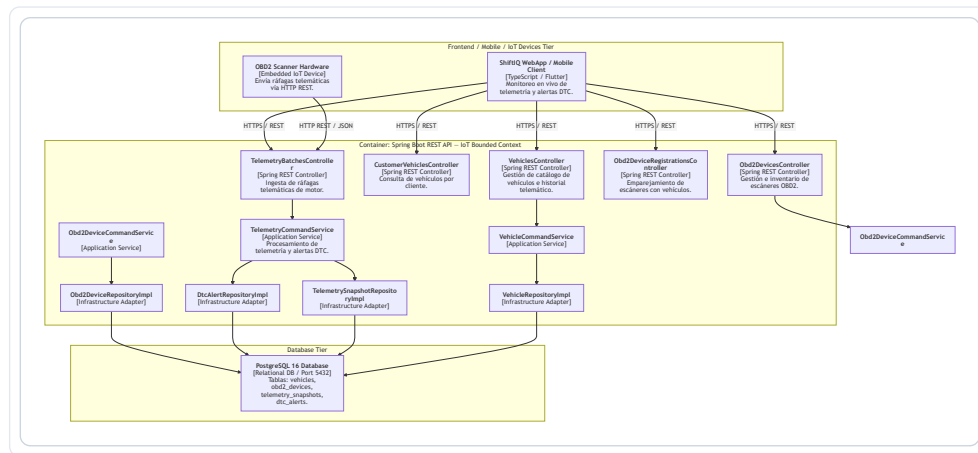


Figura 60: Diagrama de Componentes C4 Nivel 3 – IoT

2.6.5.6. Bounded Context Software Architecture Code Level Diagrams

2.6.5.6.1. Domain Layer Class Diagram

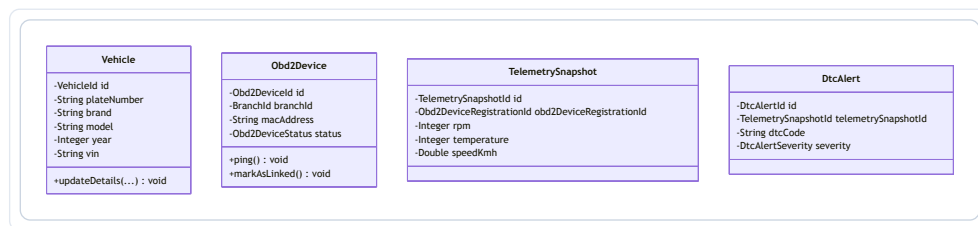


Figura 61: Diagrama de Clases del Dominio UML – IoT

2.6.5.6.2. Database Design Diagram (PostgreSQL 18)

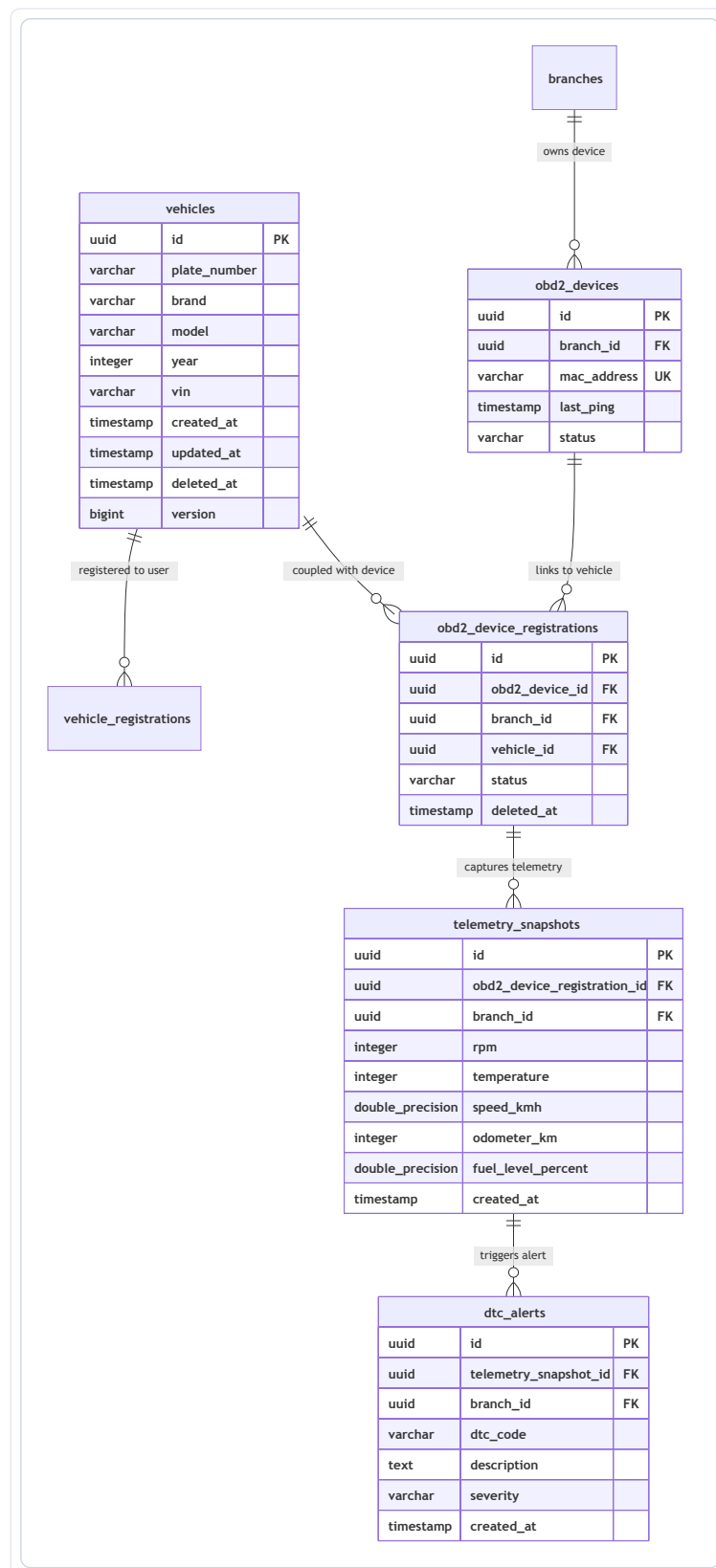


Figura 62: Diagrama de Base de Datos Relacional ER – IoT

2.6.6. Bounded Context: Core (Profiles, Workshops & Branches)

El Bounded Context **Core** constituye el núcleo relacional y organizacional de la plataforma **ShiftIQ**. Gestiona la identidad de los perfiles operacionales de los usuarios (Customer, Employee, Owner), las organizaciones y talleres mecánicos (Workshop), sus sedes o sucursales físicas (Branch), así como el modelo de monetización y suscripciones de la plataforma (SubscriptionPlan, BranchSubscription).

2.6.6.1. Domain Layer (Capa de Dominio)

La Capa de Dominio define el modelo de negocio inmutable, encapsulando reglas de validación, agregados principales, objetos de valor (Value Objects), métodos de creación (fábricas / constructores), eventos de dominio e interfaces de repositorios agnósticas a la tecnología de persistencia.

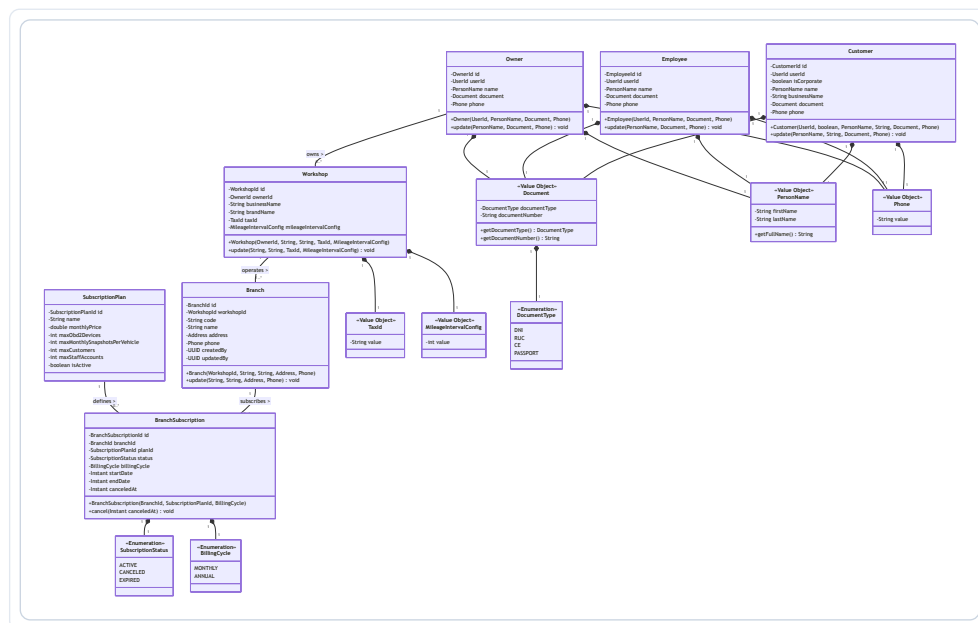


Figura 63: Diagrama de la Capa de Dominio – Core

2.6.6.1.1. Value Objects, Enums & Exceptions

Record: Document

(DocumentType documentType, String documentNumber)

Propósito: Encapsula la identidad legal del sujeto.

Validaciones: documentType no nulo (core.error.documentType.notNull); documentNumber no nulo ni en blanco (core.error.documentNumber.notBlank).

Record: PersonName

(String firstName, String lastName)

Propósito: Nombre y apellidos de personas naturales.

Validaciones: firstName y lastName no nulos ni vacíos. Método getFullName().

Record: Phone

(String value)

Propósito: Número telefónico de contac-

Record: TaxId

(String value)

Propósito: Identificador tributario (RUC de

to.

Validaciones: No nulo ni en blanco (`core.error.phone.required`).

11 dígitos).

Validaciones: Exactamente 11 dígitos numéricos (`core.error.taxId.invalid`).

Record: CreditCard

(`cardNumber`, `cardHolderName`, `expirationDate`, `cvv`)

Propósito: Datos de tarjeta para cobro simulado de suscripciones.

Validaciones: 16 dígitos, formato MM/YY, cvv de 3 dígitos.

Record: MileageIntervalConfig

(`int value`)

Propósito: Intervalo de kilometraje para mantenimientos sugeridos del taller.

Validaciones: Entero estrictamente positivo (`core.error.mileageIntervalConfig.mustBePositive`).

Strongly Typed IDs & Enumerations

- **Typed IDs:** `UserId`, `CustomerId`, `EmployeeId`, `OwnerId`, `WorkshopId`, `BranchId`, `BranchSubscriptionId`, `SubscriptionPlanId` (encapsulan UUID).
- **Enum DocumentType:** `DNI`, `RUC`, `CE`, `PASSPORT`.
- **Enum SubscriptionStatus:** `ACTIVE`, `CANCELED`, `EXPIRED`.
- **Enum BillingCycle:** `MONTHLY`, `ANNUAL`.

2.6.6.1.2. Aggregates & Entities

Customer (Aggregate Root)

Tipo: Raíz de Agregado (extends `AbstractDomainAggregateRoot<Customer>`)

Propósito: Agregado para clientes del sistema (persona natural `isCorporate = false` o persona jurídica `isCorporate = true`).

Atributos: `id` (`CustomerId`), `userId` (`UserId`), `isCorporate` (`boolean`), `name` (`PersonName`), `businessName` (`String`), `document` (`Document`), `phone` (`Phone`).

Reglas de Negocio & Métodos: Si es corporativo, `businessName` es obligatorio. En actualizaciones, el tipo de documento corporativo es inmutable. Emite `CustomerCreatedEvent` y `CustomerUpdatedEvent`.

Employee (Aggregate Root)

Tipo: Raíz de Agregado (extends `AbstractDomainAggregateRoot<Employee>`)

Propósito: Agregado para perfiles de empleados y personal técnico del taller vinculados a un `UserId`.

Atributos: `id` (`EmployeeId`), `userId` (`UserId`), `name` (`PersonName`), `document` (`Document`), `phone` (`Phone`).

Métodos: `update(PersonName, Document, Phone)`. Emite `EmployeeCreatedEvent` y `EmployeeUpdatedEvent`.

Owner (Aggregate Root)**Tipo:** Raíz de Agregado (extends `AbstractDomainAggregateRoot<Owner>`)**Propósito:** Agregado para propietarios y dueños de talleres automotrices.**Atributos:** id (`OwnerId`), userId (`UserId`), name (`PersonName`), document (`Document`), phone (`Phone`).**Métodos:** `update(PersonName, Document, Phone)`. Emite `OwnerCreatedEvent` y `OwnerUpdatedEvent`.**Workshop (Aggregate Root)****Tipo:** Raíz de Agregado (extends `AbstractDomainAggregateRoot<Workshop>`)**Propósito:** Representa la empresa o taller mecánico comercial propiedad de un `Owner`.**Atributos:** id (`WorkshopId`), ownerId (`OwnerId`), businessName (`String`), brandName (`String`), taxId (`TaxId`), mileageIntervalConfig (`MileageIntervalConfig`).**Reglas de Negocio:** businessName y brandName requeridos. Emite `WorkshopCreatedEvent` y `WorkshopUpdatedEvent`.**Branch & BranchSubscription (Aggregates)**

- **Branch:** Representa cada sucursal física (`BranchId`, `WorkshopId`, code, name, address, phone). Code único requerido. Emite `BranchCreatedEvent` y `BranchUpdatedEvent`.
- **BranchSubscription:** Suscripción contratada (`BranchSubscriptionId`, `BranchId`, `SubscriptionPlanId`, status, billingCycle, startDate, endDate). Calcula endDate automáticamente (+1 mes / +12 meses). `cancel(Instant)` cambia estado a CANCELED.
- **SubscriptionPlan:** Plan de comercialización SaaS (`SubscriptionPlanId`, name, monthlyPrice, maxObd2Devices, maxMonthlySnapshotsPerVehicle, maxCustomers, maxStaffAccounts, isActive).

2.6.6.1.3. Domain Repositories (Interfaces)**Interfaces de Repositorio del Dominio Core:**

- **CustomerRepository:** `save(Customer)`, `findById(CustomerId)`, `findByUserId(UserId)`, `existsByUserId(UserId)`, `findByDocumentNumber(String)`, `findProfileRolesByUserId(UserId)`, `delete(Customer)`.
- **EmployeeRepository:** `save(Employee)`, `findById(EmployeeId)`, `findByUserId(UserId)`, `existsByUserId(UserId)`, `findByDocumentNumber(String)`, `delete(Employee)`.
- **OwnerRepository:** `save(Owner)`, `findById(OwnerId)`, `findByUserId(UserId)`, `existsById(OwnerId)`, `existsByUserId(UserId)`, `findByDocumentNumber(String)`, `delete(Owner)`.
- **WorkshopRepository:** `save(Workshop)`, `findById(WorkshopId)`, `findAllByOwnerId(OwnerId)`, `existsById(WorkshopId)`.

- BranchRepository: `save(Branch), findById(BranchId), findAllByWorkshopId(WorkshopId), existsById(BranchId), existsByCode(String).`
- BranchSubscriptionRepository: `save(BranchSubscription), findById(BranchSubscriptionId), findAllByBranchId(BranchId), findActiveByBranchId(BranchId).`
- SubscriptionPlanRepository: `save(SubscriptionPlan), findById(SubscriptionPlanId), findByName(String), findAll().`

2.6.6.2. Application Layer (Capa de Aplicación)

La Capa de Aplicación orquesta los casos de uso, transformando los Commands y Queries provenientes de la capa de interfaz en operaciones del modelo de dominio.



Figura 64: Diagrama de la Capa de Aplicación – Core

2.6.6.2.1. Commands & Queries (DTOs)

Comandos de Escritura (CQRS Commands):

- CreateCustomerCommand(...), UpdateCustomerCommand(...), DeleteCustomerCommand(...)
- CreateEmployeeCommand(...), UpdateEmployeeCommand(...), DeleteEmployeeCommand(...)
- CreateOwnerCommand(...), UpdateOwnerCommand(...), DeleteOwnerCommand(...)
- CreateWorkshopCommand(...), UpdateWorkshopCommand(...)
- CreateBranchCommand(...), UpdateBranchCommand(...)
- AssignSubscriptionCommand(BranchId, SubscriptionPlanId, BillingCycle, CreditCard)
- CancelSubscriptionCommand(BranchId)

Consultas y DTOs de Resultado (CQRS Queries & Responses):

- GetCustomerByIdQuery, GetCustomerByUserIdQuery
- GetEmployeeByIdQuery, GetEmployeeByUserIdQuery, GetEmployeeByDocumentNumberQuery
- GetOwnerByIdQuery, GetOwnerByUserIdQuery
- GetWorkshopByIdQuery, GetAllWorkshopsByOwnerIdQuery
- GetBranchByIdQuery, GetAllBranchesByWorkshopIdQuery
- GetProfileRolesByUserIdQuery, GetProfileByDocumentNumberQuery
- ProfileSummary(profileId, userId, firstName, lastName, documentType, documentNumber, profileType)

2.6.6.3. Interface Layer (Capa de Interfaz / REST)

Expone los servicios de la plataforma a través de una API RESTful documentada con Swagger/OpenAPI y protegida mediante Spring Security.



Figura 65: Diagrama de la Capa de Interfaz – Core

2.6.6.3.1. Endpoints & REST Controllers

ProfilesController (/api/v1/profiles)

- GET /api/v1/profiles/roles?userId={userId}: Retorna lista de roles asignados al usuario (ej. ["CUSTOMER", "OWNER"]).
- GET /api/v1/profiles?documentNumber={documentNumber}: Búsqueda rápida de perfil por DNI/RUC.

CustomersController (/api/v1/customers)

- POST /api/v1/customers: Registra perfil de cliente (natural o corporativo).
- GET /api/v1/customers?userId={userId} / GET /api/v1/customers/{customerId}: Consultas de clientes.
- PUT /api/v1/customers/{customerId} / DELETE /api/v1/customers/{customerId}: Actualización y borrado lógico.

EmployeesController & OwnersController

- CRUD completo para perfiles de empleados (/api/v1/employees) y dueños de taller (/api/v1/owners).

WorkshopsController & BranchesController

- POST /api/v1/workshops & GET /api/v1/workshops?ownerId={ownerId}: Gestión de talleres.
- POST /api/v1/branches, GET /api/v1/branches?workshopId={workshopId}, POST /api/v1/branches/{branchId}/subscriptions, DELETE /api/v1/branches/{branchId}/subscription: Gestión de sucursales y suscripciones.

2.6.6.4. Infrastructure Layer (Capa de Infraestructura)

Implementa la persistencia física en PostgreSQL 18 utilizando Spring Data JPA, mapeando entidades de dominio inmutables a entidades de tabla relacional.

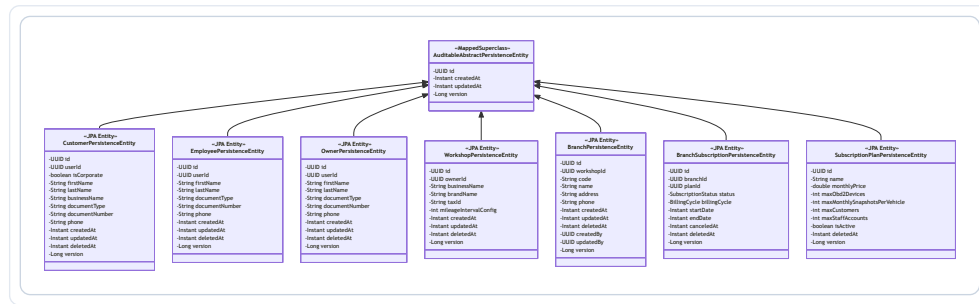


Figura 66: Diagrama de la Capa de Infraestructura – Core

2.6.6.4.1. Mapeo de Entidades Relacionales (JPA)

Tabla: customers (CustomerPersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
user_id	UUID	NOT NULL, UNIQUE
is_corporate	BOOLEAN	NOT NULL
first_name	VARCHAR	NOT NULL
last_name	VARCHAR	NOT NULL
business_name	VARCHAR	NULLABLE
document_type	VARCHAR	NOT NULL
document_number	VARCHAR	NOT NULL
phone	VARCHAR	NOT NULL
created_at	TIMESTAMP	NOT NULL
updated_at	TIMESTAMP	NOT NULL
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)
version	BIGINT	NOT NULL

Tabla: employees (EmployeePersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
user_id	UUID	NOT NULL, UNIQUE
first_name	VARCHAR	NOT NULL
last_name	VARCHAR	NOT NULL
document_type	VARCHAR	NOT NULL
document_number	VARCHAR	NOT NULL

Columna	Tipo de Dato	Constraints / Descripción
phone	VARCHAR	NOT NULL
created_at	TIMESTAMP	NOT NULL
updated_at	TIMESTAMP	NOT NULL
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)
version	BIGINT	NOT NULL

Tabla: owners (OwnerPersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
user_id	UUID	NOT NULL, UNIQUE
first_name	VARCHAR	NOT NULL
last_name	VARCHAR	NOT NULL
document_type	VARCHAR	NOT NULL
document_number	VARCHAR	NOT NULL
phone	VARCHAR	NOT NULL
created_at	TIMESTAMP	NOT NULL
updated_at	TIMESTAMP	NOT NULL
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)
version	BIGINT	NOT NULL

Tabla: workshops (WorkshopPersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
owner_id	UUID	NOT NULL, FOREIGN KEY (owners.id)
business_name	VARCHAR	NOT NULL
brand_name	VARCHAR	NOT NULL
tax_id	VARCHAR	NOT NULL
mileage_interval_config	INTEGER	NOT NULL
created_at	TIMESTAMP	NOT NULL
updated_at	TIMESTAMP	NOT NULL
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)
version	BIGINT	NOT NULL

Tabla: branches (BranchPersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
workshop_id	UUID	NOT NULL, FOREIGN KEY (workshops.id)
code	VARCHAR	NOT NULL, UNIQUE
name	VARCHAR	NOT NULL
address	VARCHAR	NOT NULL
phone	VARCHAR	NOT NULL
created_by	UUID	NOT NULL
updated_by	UUID	NOT NULL
created_at	TIMESTAMP	NOT NULL
updated_at	TIMESTAMP	NOT NULL
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)
version	BIGINT	NOT NULL

Tabla: branch_subscriptions (BranchSubscriptionPersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
branch_id	UUID	NOT NULL, FOREIGN KEY (branches.id)
plan_id	UUID	NOT NULL, FOREIGN KEY (subscription_plans.id)
status	VARCHAR	NOT NULL (Enum)
billing_cycle	VARCHAR	NOT NULL (Enum)
start_date	TIMESTAMP	NOT NULL
end_date	TIMESTAMP	NOT NULL
canceled_at	TIMESTAMP	NULLABLE
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)
version	BIGINT	NOT NULL

Tabla: subscription_plans (SubscriptionPlanPersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
name	VARCHAR	NOT NULL, UNIQUE
monthly_price	DOUBLE PRECISION	NOT NULL

Columna	Tipo de Dato	Constraints / Descripción
max_obd2_devices	INTEGER	NOT NULL
max_monthly_snapshots_per_vehicle	INTEGER	NOT NULL
max_customers	INTEGER	NOT NULL
max_staff_accounts	INTEGER	NOT NULL
is_active	BOOLEAN	NOT NULL
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)
version	BIGINT	NOT NULL

2.6.6.5. Software Architecture Component Level Diagrams (C4 Model - Level 3)

Descomposición del Container API en sus componentes principales para el Bounded Context Core.

2.6.6.5.1. C4 Model Component Diagram

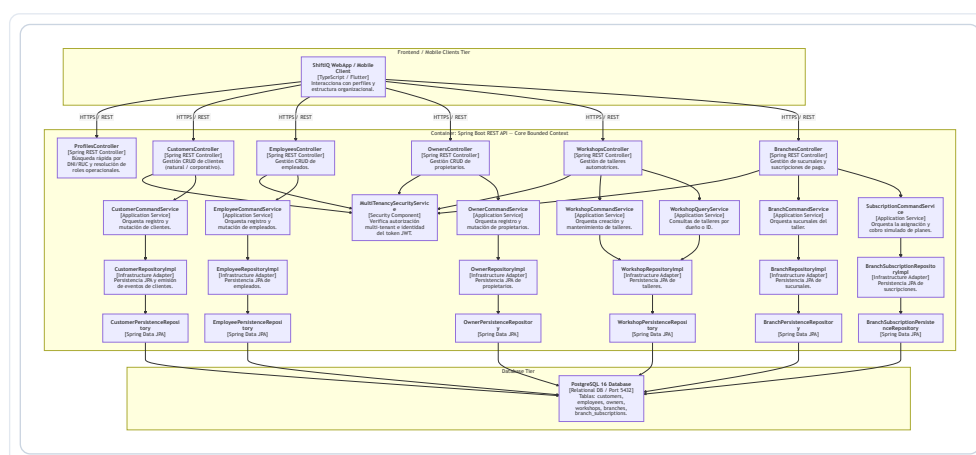


Figura 67: Diagrama de Componentes C4 Nivel 3 – Core

2.6.6.6. Code Level Diagrams

2.6.6.6.1. Domain Layer Class Diagram

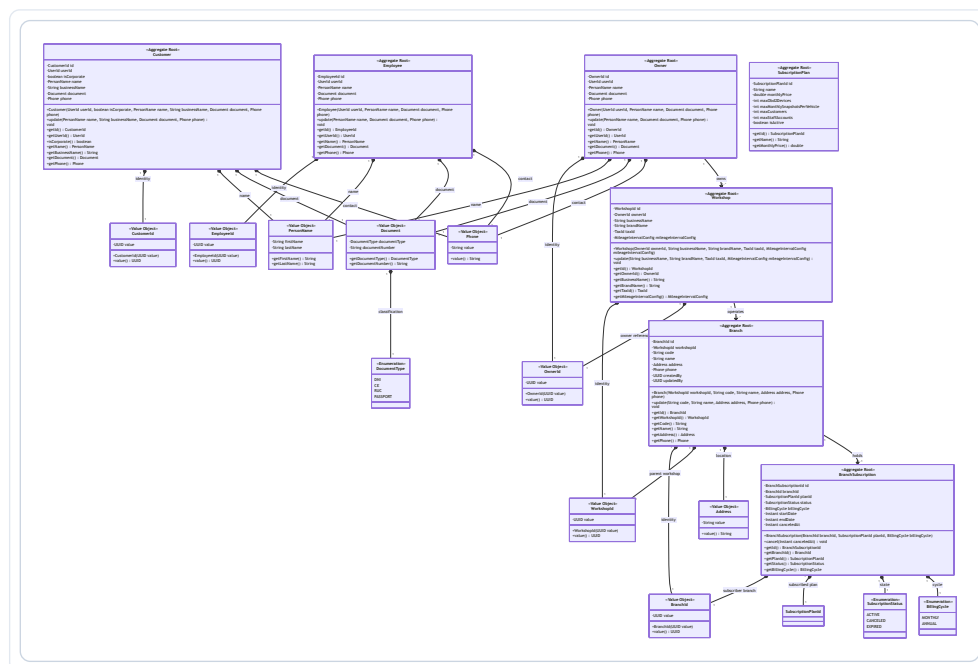


Figura 68: Diagrama de Clases del Dominio UML – Core

2.6.6.6.2. Database Design Diagram (PostgreSQL 18)

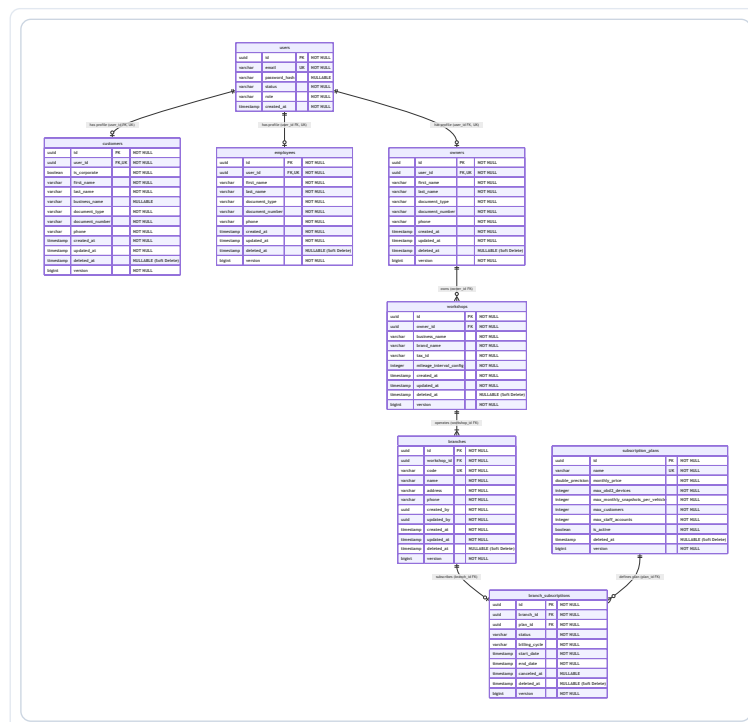


Figura 69: Diagrama de Base de Datos Relacional ER – Core

2.6.7. Bounded Context: Fleet (Appointments & Registrations)

El Bounded Context **Fleet** administra las citas programadas de atención mecánica (Appointment) en las distintas sucursales del taller, así como el registro y vinculación multi-tenant de clientes (CustomerRegistration) y empleados técnicos (EmployeeRegistration) con las sucursales del sistema. Interactúa mediante un Anti-Corruption Layer (ACL) con el Bounded Context **Core** para validar la existencia de clientes, empleados y sucursales.

2.6.7.1. Domain Layer (Capa de Dominio)

La Capa de Dominio define las reglas de agendamiento de citas mecánicas, duraciones estimadas predeterminadas (1 hora), métodos **Factory** para instanciación de agregados, validaciones de solapamiento de horarios en la capa de aplicación y la adscripción de clientes y empleados a las sedes activas del taller.

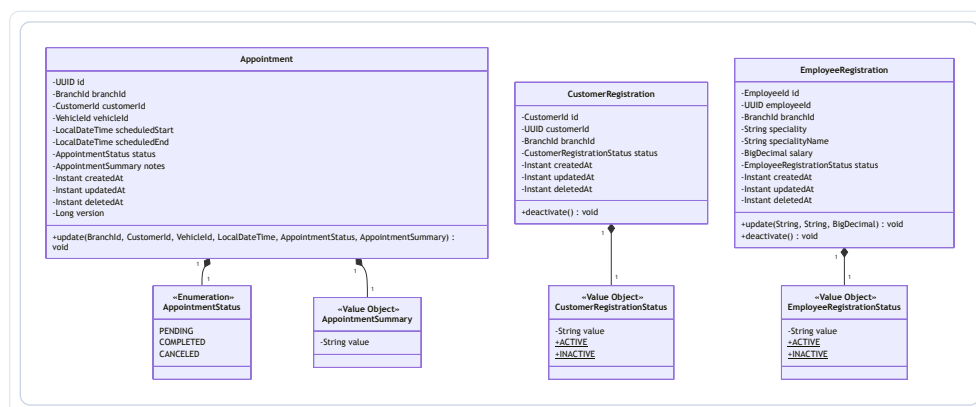


Figura 70: Diagrama de la Capa de Dominio – Fleet

2.6.7.1.1. Value Objects, Enums & Exceptions

Enum: AppointmentStatus

Valores: PENDING, COMPLETED, CANCELED

Propósito: Representa el estado del ciclo de vida de una cita programada.

Record: AppointmentSummary

(String value)

Propósito: Notas explicativas o resumen del motivo técnico de la cita. Max 2000 chars.

Record: CustomerRegistrationStatus

(String value)

Constantes: ACTIVE ("ACTIVE"), INACTIVE ("INACTIVE"). No nulo ni en blanco.

Record: EmployeeRegistrationStatus

(String value)

Constantes: ACTIVE ("ACTIVE"), INACTIVE ("INACTIVE"). No nulo ni en blanco.

2.6.7.1.2. Aggregates & Entities

Appointment (Aggregate Root)

Tipo: Raíz de Agregado (extends AbstractDomainAggregateRoot<Appointment>)

Propósito: Cita de servicio agendada para un cliente y vehículo en una sucursal específica.

Atributos: id (UUID), branchId (BranchId), customerId (CustomerId), vehicleId (VehicleId), scheduledStart (LocalDateTime), scheduledEnd (LocalDateTime), status (AppointmentStatus), notes (AppointmentSummary).

Reglas de Negocio: Al crearse calcula automáticamente $\text{scheduledEnd} = \text{scheduledStart} + 1$ hora. Inicia en estado PENDING. Emite AppointmentCreatedEvent.

CustomerRegistration (Aggregate Root)

Tipo: Raíz de Agregado (extends AbstractDomainAggregateRoot<CustomerRegistration>)

Propósito: Registro de asociación de un cliente (CustomerId) con una sucursal (BranchId).

Atributos: id (CustomerId), customerId (UUID), branchId (BranchId), status (CustomerRegistrationStatus).

Reglas de Negocio: Inicia en estado ACTIVE. Método deactivate() cambia a INACTIVE y setea deletedAt. Emite CustomerRegistrationCreatedEvent.

EmployeeRegistration (Aggregate Root)

Tipo: Raíz de Agregado (extends AbstractDomainAggregateRoot<EmployeeRegistration>)

Propósito: Registro de adscripción de un empleado (EmployeeId) a una sucursal con especialidad técnica y salario.

Atributos: id (EmployeeId), employeeId (UUID), branchId (BranchId), speciality (String), specialityName (String), salary (BigDecimal), status (EmployeeRegistrationStatus).

Reglas de Negocio: Permite actualizar especialidad y salario. deactivate() deshabilita la adscripción. Emite EmployeeRegistrationCreatedEvent.

2.6.7.1.4. Domain Repositories (Interfaces)

Interfaces de Repositorio del Dominio Fleet:

- AppointmentRepository: save(Appointment), findById(UUID), existsById(UUID), deleteById(UUID), existsByScheduledStartLessThanAndScheduledEndGreaterThan(...), findByBranchId(...), findByCustomerId(...), findByVehicleId(...), findByBranchIdAndStatus(...).
- CustomerRegistrationRepository: save(...), findById(...), findByCustomerId(...), findByCustomerIdAndBranchId(...), findByBranchIdAndStatus(...), existsByCustomerIdAndBranchId(...).
- EmployeeRegistrationRepository: save(...), findById(...), findByEmployeeId(...), findByBranchId(...), findByBranchIdAndStatus(...), existsByEmployeeIdAndBranchId(...).

2.6.7.2. Application Layer (Capa de Aplicación)

La Capa de Aplicación expone la ejecución de casos de uso mediante servicios de comando y consulta.



Figura 71: Diagrama de la Capa de Aplicación – Fleet

2.6.7.2.1. Commands & Queries (DTOs de Aplicación)

Comandos de Escritura (CQRS Commands):

- CreateAppointmentCommand(BranchId, CustomerId, VehicleId, LocalDateTime, AppointmentSummary)
- UpdateAppointmentCommand(UUID, BranchId, CustomerId, VehicleId, LocalDateTime, AppointmentStatus, AppointmentSummary)
- DeleteAppointmentCommand(UUID appointmentId)
- CreateCustomerRegistrationCommand(CustomerId, BranchId)
- UpdateCustomerRegistrationCommand(UUID registrationId, CustomerRegistrationStatus)
- DeleteCustomerRegistrationCommand(UUID registrationId)
- CreateEmployeeRegistrationCommand(EmployeeId, BranchId, String, String, BigDecimal)
- UpdateEmployeeRegistrationCommand(EmployeeId, String, String, BigDecimal)
- DeleteEmployeeRegistrationCommand(EmployeeId registrationId)

Consultas y DTOs de Resultado (CQRS Queries & ACL Outbound):

- AppointmentQueryService (búsquedas por ID, sucursal, cliente, vehículo y estado).
- GetCustomerRegistrationByCustomerIdQuery(UUID customerId)
- GetEmployeeRegistrationByIdQuery, GetEmployeeRegistrationByEmployeeIdQuery
- GetEmployeeRegistrationsByBranchIdQuery, GetEmployeeRegistrationsByBranchIdAndStatusQuery
- **Outbound Services (ACL):** ExternalCoreService (valida Clientes/Empleados/Sucursales en Core), ExternalVehicleService (valida Vehículos).

2.6.7.3. Interface Layer (Capa de Interfaz / REST)

Exposición RESTful para agendamiento de citas y registros de clientes/empleados por sucursal.

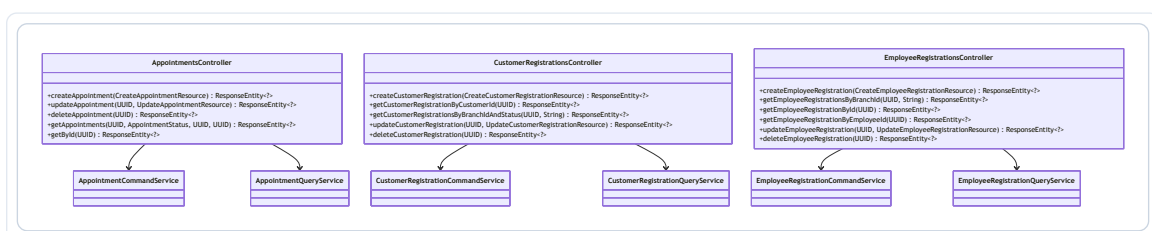


Figura 72: Diagrama de la Capa de Interfaz – Fleet

2.6.7.3.1. Endpoints & REST Controllers

AppointmentsController (/api/v1/appointments)

- POST /api/v1/appointments: Agenda una nueva cita mecánica.
- GET /api/v1/appointments: Obtiene citas filtradas opcionalmente por branchId, status, customerId O vehicleId.
- GET /api/v1/appointments/{appointmentId}: Obtiene el detalle de una cita específica.
- PUT /api/v1/appointments/{appointmentId} / DELETE /api/v1/appointments/{appointmentId}: Edición y soft-delete.

CustomerRegistrationsController (/api/v1/customer-registrations)

- POST /api/v1/customer-registrations: Vincula a un cliente con una sucursal.
- GET /api/v1/customer-registrations?customerId={customerId} / ? branchId={branchId}&status={status}: Consultas.
- PUT /api/v1/customer-registrations/{id} / DELETE /api/v1/customer-registrations/{id}: Edición y desactivación.

EmployeeRegistrationsController (/api/v1/employee-registrations)

- POST /api/v1/employee-registrations: Adscribe a un empleado técnico a una sucursal.
- GET /api/v1/employee-registrations?branchId={branchId}&status={status} / {id}: Consultas de adscripción.
- PUT /api/v1/employee-registrations/{id} / DELETE /api/v1/employee-registrations/{id}: Actualización y desactivación.

2.6.7.4. Infrastructure Layer (Capa de Infraestructura)

Mapeo relacional JPA a PostgreSQL 18 con soporte de eliminación lógica (@SQLDelete setean-do deleted_at).

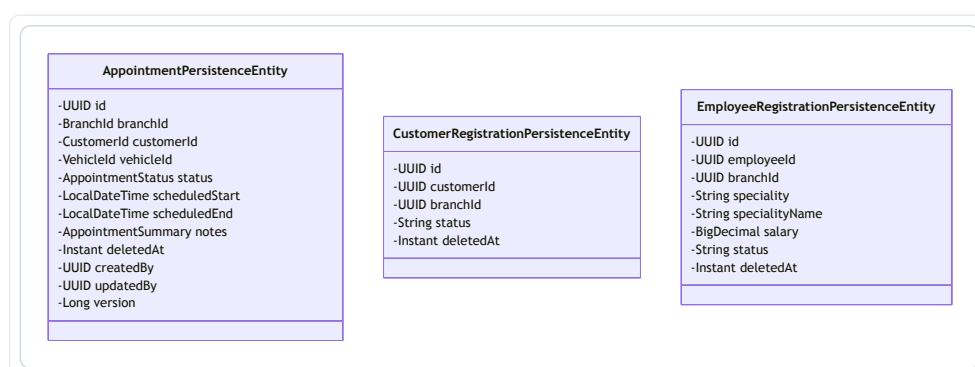


Figura 73: Diagrama de la Capa de Infraestructura – Fleet

2.6.7.4.1. Mapeo de Entidades Relacionales (JPA)

Tabla: appointments (AppointmentPersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
branch_id	UUID	NOT NULL
customer_id	UUID	NOT NULL
vehicle_id	UUID	NOT NULL
status	VARCHAR(20)	NOT NULL (PENDING, COMPLETED, CANCELED)
scheduled_start	TIMESTAMP	NOT NULL
scheduled_end	TIMESTAMP	NOT NULL, CHECK (scheduled_end > scheduled_start)
notes	TEXT	NULLABLE (AppointmentSummaryAttributeConverter)
created_by	UUID	NOT NULL
updated_by	UUID	NOT NULL
created_at	TIMESTAMP	NOT NULL
updated_at	TIMESTAMP	NOT NULL
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)
version	BIGINT	NOT NULL

Tabla: customer_registrations (CustomerRegistrationPersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
customer_id	UUID	NOT NULL
branch_id	UUID	NOT NULL
status	VARCHAR(20)	NOT NULL (ACTIVE, INACTIVE)
created_at	TIMESTAMP	NOT NULL
updated_at	TIMESTAMP	NOT NULL
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)
version	BIGINT	NOT NULL

Tabla: employee_registrations (EmployeeRegistrationPersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL

Columna	Tipo de Dato	Constraints / Descripción
employee_id	UUID	NOT NULL
branch_id	UUID	NOT NULL
speciality	VARCHAR(50)	NOT NULL
speciality_name	VARCHAR(50)	NULLABLE
salary	NUMERIC(10,2)	NOT NULL, CHECK (salary >= 0)
status	VARCHAR(20)	NOT NULL (ACTIVE, INACTIVE)
created_at	TIMESTAMP	NOT NULL
updated_at	TIMESTAMP	NOT NULL
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)
version	BIGINT	NOT NULL

2.6.7.5. Software Architecture Component Level Diagrams (C4 Model - Level 3)

Descomposición del Container API en sus componentes principales para el Bounded Context Fleet.

2.6.7.5.1. C4 Model Component Diagram

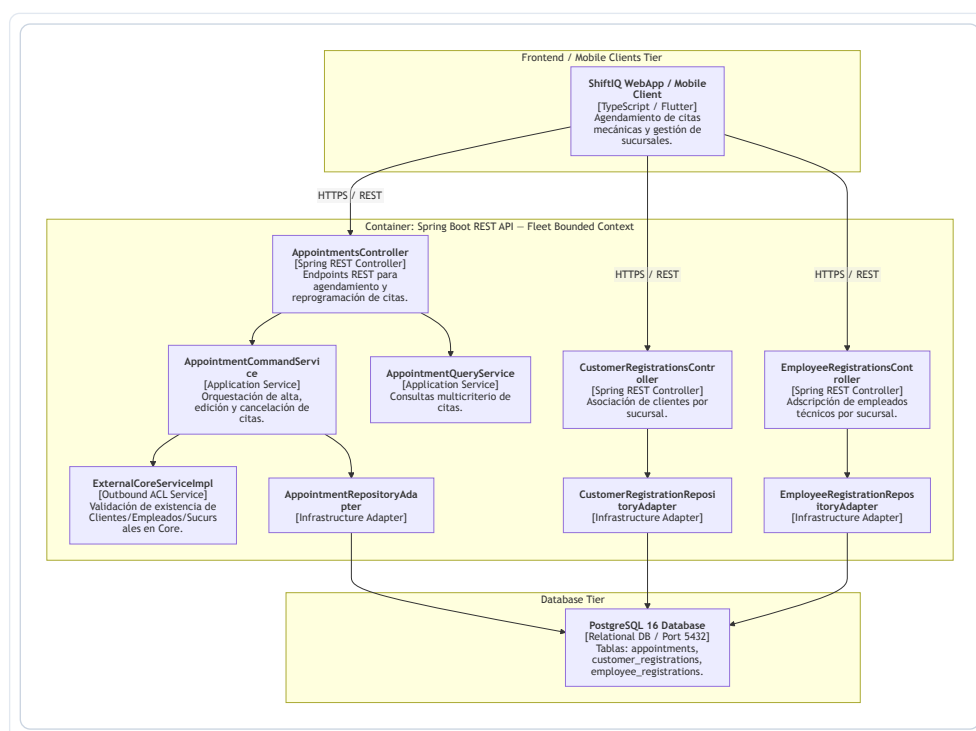


Figura 74: Diagrama de Componentes C4 Nivel 3 – Fleet

2.6.7.6. Code Level Diagrams

2.6.7.6.1. Domain Layer Class Diagram

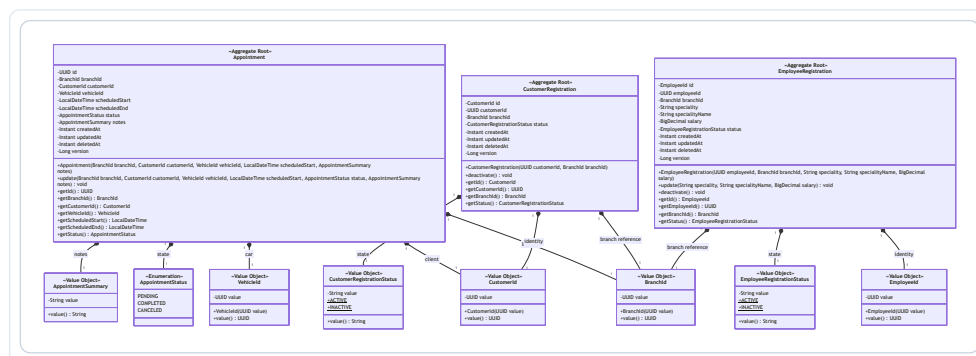


Figura 75: Diagrama de Clases del Dominio UML – Fleet

2.6.7.6.2. Database Design Diagram (PostgreSQL 18)

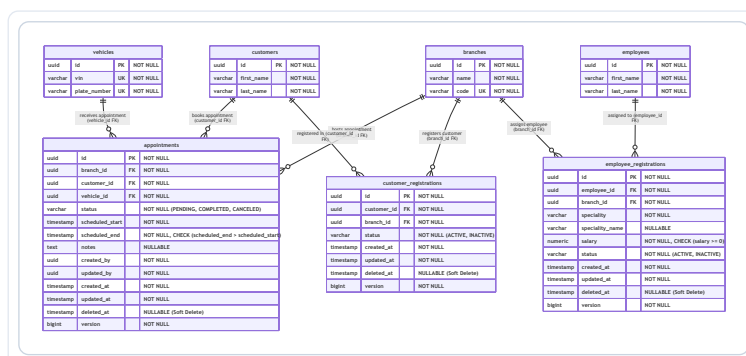


Figura 76: Diagrama de Base de Datos Relacional ER – Fleet

2.6.8. Bounded Context: Billing (Quotes, Vouchers & Payments)

El Bounded Context **Billing** gestiona el ciclo de vida financiero posterior a la prestación de servicios en el taller automotriz. Comprende la cotización preliminar de órdenes de trabajo (Quote), la emisión de comprobantes de pago electrónicos autorizados por SUNAT (Voucher: Facturas/Boletas) mediante la integración con la API externa **Factos**, el registro y amortización de pagos multicanal (Payment), la integración de cobros con tarjeta mediante **Stripe**, y los flujos de facturación inmediata (**Checkout**).

2.6.8.1. Domain Layer (Capa de Dominio)

La Capa de Dominio encapsula el cálculo estricto de subtotales, impuestos (IGV 18%), descuentos porcentuales, montos totales mediante métodos **Factory**, reglas financieras encapsuladas en los Agregados Quote y Voucher, transiciones de estado inmutables, la validación del saldo deudor de comprobantes y la emisión de eventos de dominio financieros.

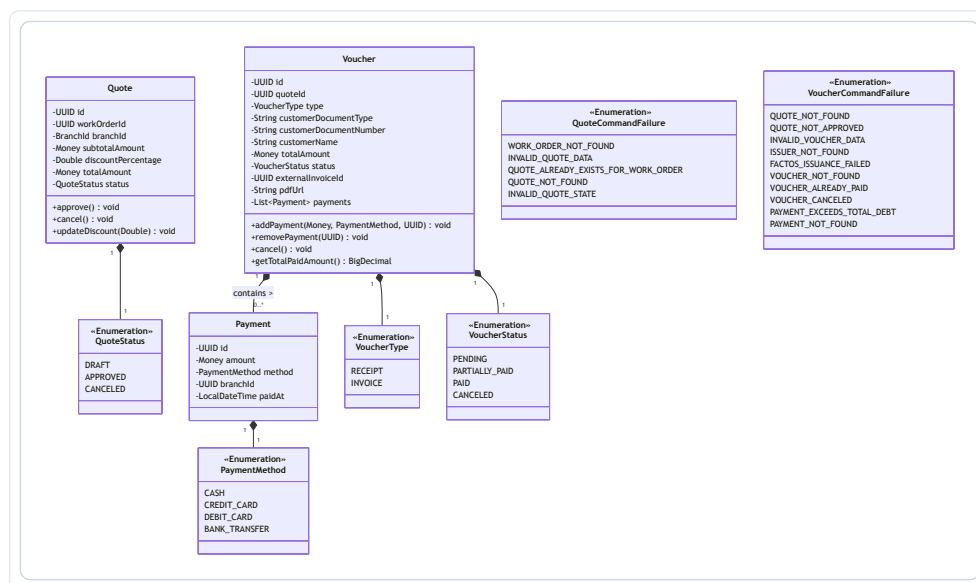


Figura 77: Diagrama de la Capa de Dominio – Billing

2.6.8.1.1. Value Objects, Enums & Exceptions

Enum: QuoteStatus

Valores: DRAFT, APPROVED, CANCELED

Propósito: Ciclo de vida de la cotización preliminar. APPROVED es requisito obligatorio para emitir Voucher.

Enum: VoucherType

Valores: RECEIPT, INVOICE

Propósito: Tipo de comprobante fiscal SUNAT (RECEIPT: Boleta B001 para DNI; INVOICE: Factura F001 para RUC).

Enum: VoucherStatus

Valores: PENDING, PARTIALLY_PAID, PAID, CANCELED

Propósito: Estado de saldo deudor. Al saldar 100%, emite VoucherPaidEvent.

Enum: PaymentMethod

Valores: CASH, CREDIT_CARD, DEBIT_CARD, BANK_TRANSFER

Propósito: Canal transaccional del abono registrado.

Excepciones y Fallos del Dominio Financiero:

- **QuoteCommandFailure:** WORK_ORDER_NOT_FOUND, INVALID_QUOTE_DATA, QUOTE_ALREADY_EXISTS_FOR_WORK_ORDER, QUOTE_NOT_FOUND, INVALID_QUOTE_STATE.
- **VoucherCommandFailure:** QUOTE_NOT_FOUND, QUOTE_NOT_APPROVED, INVALID_VOUCHER_DATA, ISSUER_NOT_FOUND, FACTOS_ISSUANCE_FAILED, VOUCHER_NOT_FOUND, VOUCHER_ALREADY_PAID, VOUCHER_CANCELED, PAYMENT_EXCEEDS_TOTAL_DEBT, PAYMENT_NOT_FOUND.

2.6.8.1.2. Aggregates & Entities

Quote (Aggregate Root)

Tipo: Raíz de Agregado (extends AbstractDomainAggregateRoot<Quote>)

Propósito: Representa la cotización de servicios y repuestos de una Orden de Trabajo.

Atributos: id (UUID), workOrderId (UUID), branchId (BranchId), subtotalAmount (Money), discountPercentage (Double), totalAmount (Money), status (QuoteStatus).

Reglas de Negocio: Aplica descuento porcentual sobre el subtotal (0% a 100%) para calcular $\text{totalAmount} = \text{subtotal} * (1 - \text{discount}/100)$. La aprobación `approve()` requiere estar previamente en DRAFT.

Voucher (Aggregate Root) & Payment (Entity)

- **Voucher:** Agregado principal de facturación (quoteId, type, customerDocumentType, customerDocumentNumber, customerName, totalAmount, status, externalInvoiceId, pdfUrl, payments).
- **Payment:** Entidad de abono (id, amount, method, branchId, paidAt).
- **Reglas de Negocio:** `addPayment(...)` valida que el monto no exceda la deuda restante. Transiciona automáticamente a PARTIALLY_PAID o PAID. Al cubrir el 100% emite `VoucherPaidEvent`. `removePayment(UUID)` elimina abonos y recalcula dinámicamente el estado.

2.6.8.1.4. Domain Repositories (Interfaces)

Interfaces de Repositorio del Dominio Billing:

- **QuoteRepository:** `save(Quote)`, `findById(UUID)`, `findAllByBranchId(BranchId)`, `existsByWorkOrderId(UUID)`.
- **VoucherRepository:** `save(Voucher)`, `findById(UUID)`, `findByBranchId(BranchId)`.

2.6.8.2. Application Layer (Capa de Aplicación)

La Capa de Aplicación expone la ejecución de casos de uso mediante servicios de comando (`QuoteCommandService`, `VoucherCommandService`, `StripePaymentCommandService`) y servicios de consulta (`QuoteQueryService`, `VoucherQueryService`).

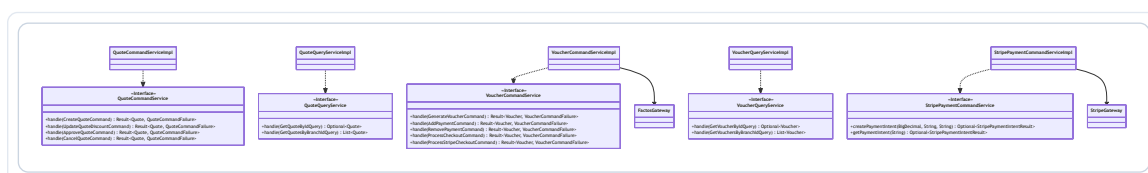


Figura 78: Diagrama de la Capa de Aplicación – Billing

2.6.8.2.1. Commands, Queries & Outbound Gateways (DTOs & Ports)

Comandos de Escritura (CQRS Commands):

- `CreateQuoteCommand(workOrderId, discountPercentage, branchId, status)`, `UpdateQuoteDiscountCommand(...)`, `ApproveQuoteCommand(...)`, `CancelQuoteCommand(...)`

- `GenerateVoucherCommand(quoteId, type, customerDocumentType, customerDocumentNumber, customerName)`
- `AddPaymentCommand(voucherId, amount, method)`, `RemovePaymentCommand(voucherId, paymentId)`
- `ProcessCheckoutCommand(...)`, `ProcessStripeCheckoutCommand(...)`

Consultas & Outbound Gateways (CQRS Queries & Ports):

- `GetQuoteByIdQuery`, `GetQuotesByBranchIdQuery`, `GetVoucherByIdQuery`, `GetVouchersByBranchIdQuery`
- **FactosGateway:** `issueVoucher(issuerRuc, documentType, customerDocumentType, customerDocumentNumber, customerName, items)`
- **StripeGateway / PaymentGateway:** `createStripePaymentIntent(amount, currency, description)`, `getStripePaymentIntent(paymentIntentId)`

2.6.8.3. Interface Layer (Capa de Interfaz / REST & Events)

Exposición RESTful e integración de listeners para eventos internos de facturación.



Figura 79: Diagrama de la Capa de Interfaz – Billing

2.6.8.3.1. Endpoints & REST Controllers

QuotesController (/api/v1/quotes)

- `POST /api/v1/quotes`: Registra cotización basada en Orden de Trabajo.
- `GET /api/v1/quotes?branchId={branchId}` / `GET /api/v1/quotes/{id}`: Consultas.
- `PUT /api/v1/quotes/{id}`: Actualiza descuento en DRAFT.
- `POST /api/v1/quotes/{id}/approvals`: Aprueba cotización.
- `POST /api/v1/quotes/{id}/cancellations`: Anula cotización.

VouchersController & CheckoutsController

- `POST /api/v1/vouchers`: Emisión de Boleta/Factura vía Factos API SUNAT.
- `GET /api/v1/vouchers?branchId={branchId}` / `{voucherId}`: Consultas de comprobantes y sus abonos.
- `POST /api/v1/vouchers/{voucherId}/payments`: Registro de abonos parciales/totales.
- `POST /api/v1/checkouts` & `POST /api/v1/checkouts/stripe`: Flujos de cobro inmediato y checkout con Stripe.

StripePaymentsController & Event Listener

- `POST /api/v1/payments/stripe/payment-intents`: Genera PaymentIntent para tarjetas en app web/móvil.

- **VoucherPaidListener:** Escucha **VoucherPaidEvent** para auditoría y finalización de la Orden de Trabajo.

2.6.8.4. Infrastructure Layer (Capa de Infraestructura)

Mapeo relacional JPA a tablas PostgreSQL 18 e integración de clientes HTTP REST (FactosGatewayImpl y StripeGatewayImpl).

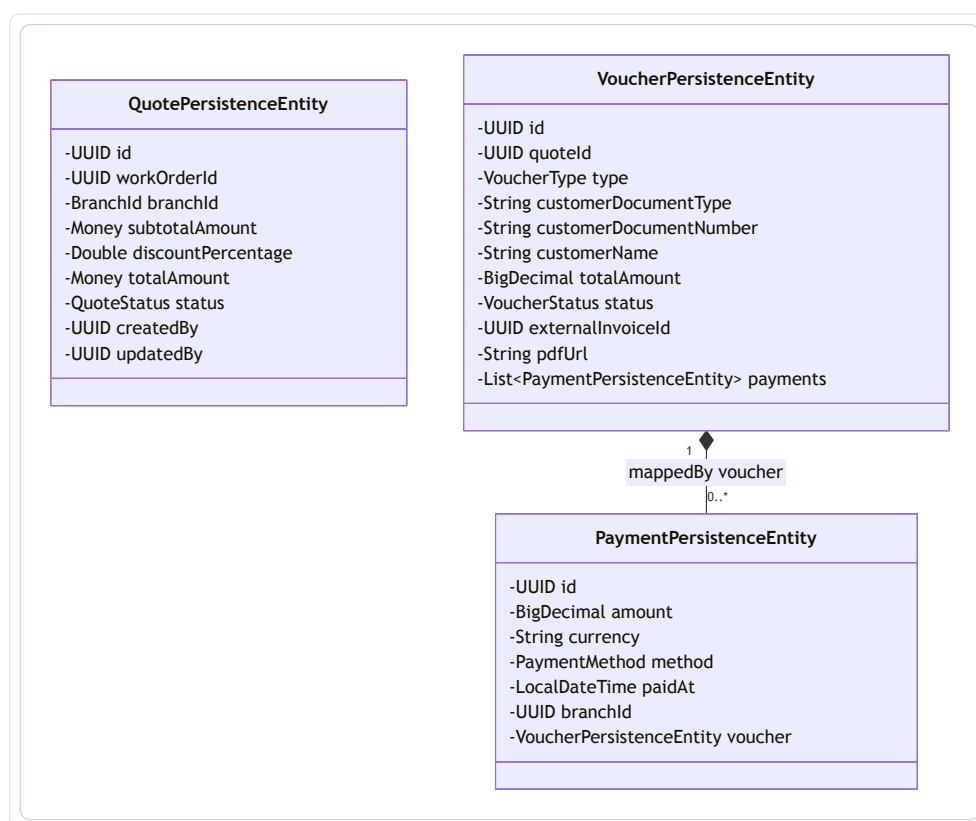


Figura 80: Diagrama de la Capa de Infraestructura – Billing

2.6.8.4.1. Mapeo de Entidades Relacionales (JPA)

Tabla: quotes (QuotePersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
work_order_id	UUID	NOT NULL, UNIQUE
branch_id	UUID	NOT NULL
subtotal_amount	DECIMAL	NOT NULL, CHECK (subtotal_amount >= 0)
discount_percentage	DOUBLE PRECISION	NOT NULL, CHECK (0 <= discount <= 100)
total_amount	DECIMAL	NOT NULL, CHECK (total_amount >= 0)
status	VARCHAR(20)	NOT NULL (DRAFT, APPROVED, CANCELED)

Columna	Tipo de Dato	Constraints / Descripción
created_by	UUID	NOT NULL
updated_by	UUID	NOT NULL
created_at	TIMESTAMP	NOT NULL
updated_at	TIMESTAMP	NOT NULL
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)
version	BIGINT	NOT NULL

Tabla: vouchers (VoucherPersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
quote_id	UUID	NOT NULL
type	VARCHAR(20)	NOT NULL (RECEIPT, INVOICE)
customer_document_type	VARCHAR(20)	NOT NULL
customer_document_number	VARCHAR(20)	NOT NULL
customer_name	VARCHAR(150)	NOT NULL
total_amount	DECIMAL(10,2)	NOT NULL, CHECK (total_amount >= 0)
status	VARCHAR(20)	NOT NULL (PENDING, PARTIALLY_PAID, PAID, CANCELED)
external_invoice_id	UUID	NOT NULL
pdf_url	VARCHAR(500)	NULLABLE
created_at	TIMESTAMP	NOT NULL
updated_at	TIMESTAMP	NOT NULL
deleted_at	TIMESTAMP	NULLABLE (Soft Delete)
version	BIGINT	NOT NULL

Tabla: payments (PaymentPersistenceEntity)

Columna	Tipo de Dato	Constraints / Descripción
id	UUID	PRIMARY KEY, NOT NULL
voucher_id	UUID	NOT NULL, FOREIGN KEY (vouchers.id)
amount	DECIMAL	NOT NULL, CHECK (amount > 0)
currency	VARCHAR(3)	NOT NULL DEFAULT 'PEN'
method	VARCHAR(20)	NOT NULL (CASH, CREDIT_CARD, DEBIT_CARD, BANK_TRANSFER)

Columna	Tipo de Dato	Constraints / Descripción
branch_id	UUID	NOT NULL
paid_at	TIMESTAMP	NOT NULL

2.6.8.5. Software Architecture Component Level Diagrams (C4 Model - Level 3)

Descomposición del Container API en sus componentes principales para el Bounded Context Billing.

2.6.8.5.1. C4 Model Component Diagram

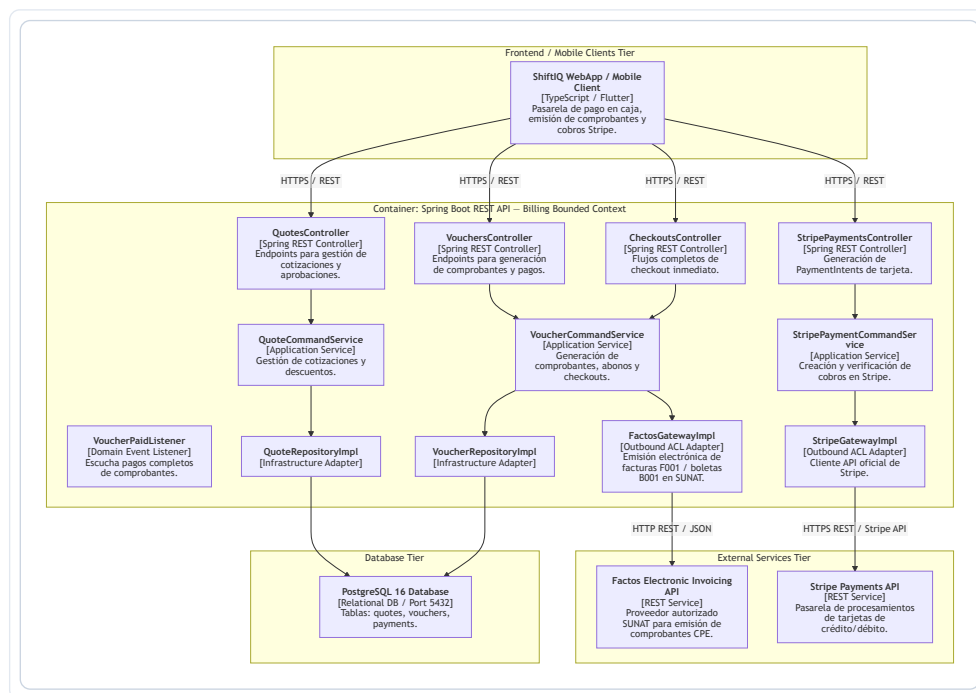


Figura 81: Diagrama de Componentes C4 Nivel 3 – Billing

2.6.8.6. Code Level Diagrams

2.6.8.6.1. Domain Layer Class Diagram

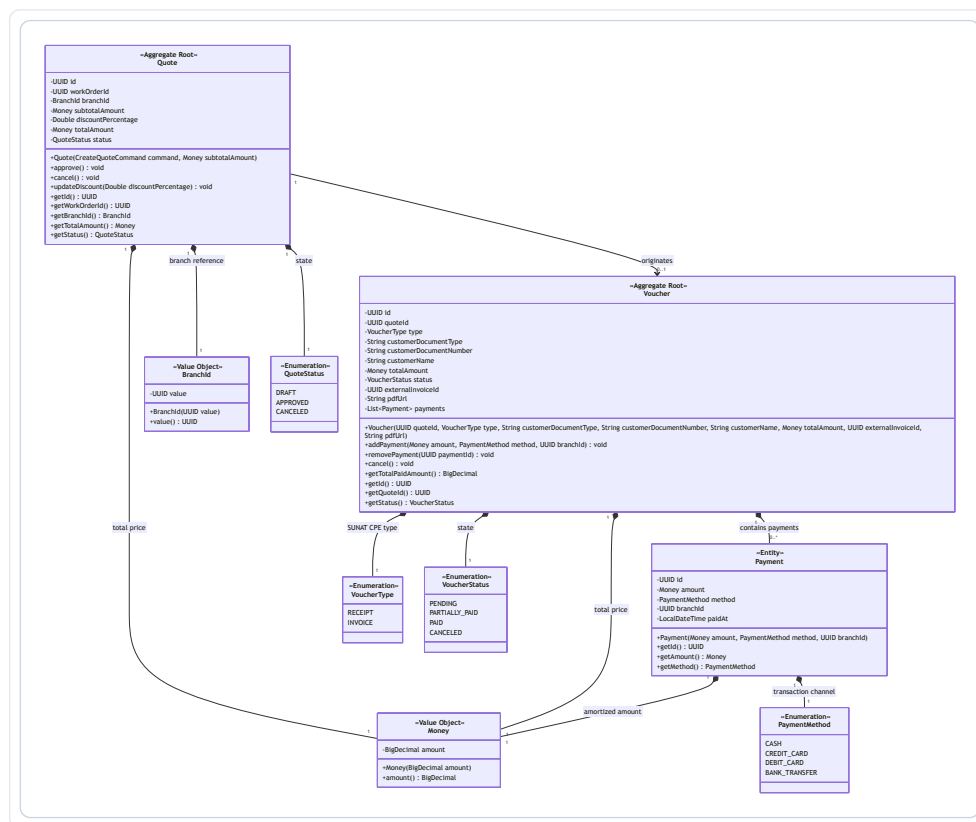


Figura 82: Diagrama de Clases del Dominio UML – Billing

2.6.8.6.2. Database Design Diagram (PostgreSQL 18)

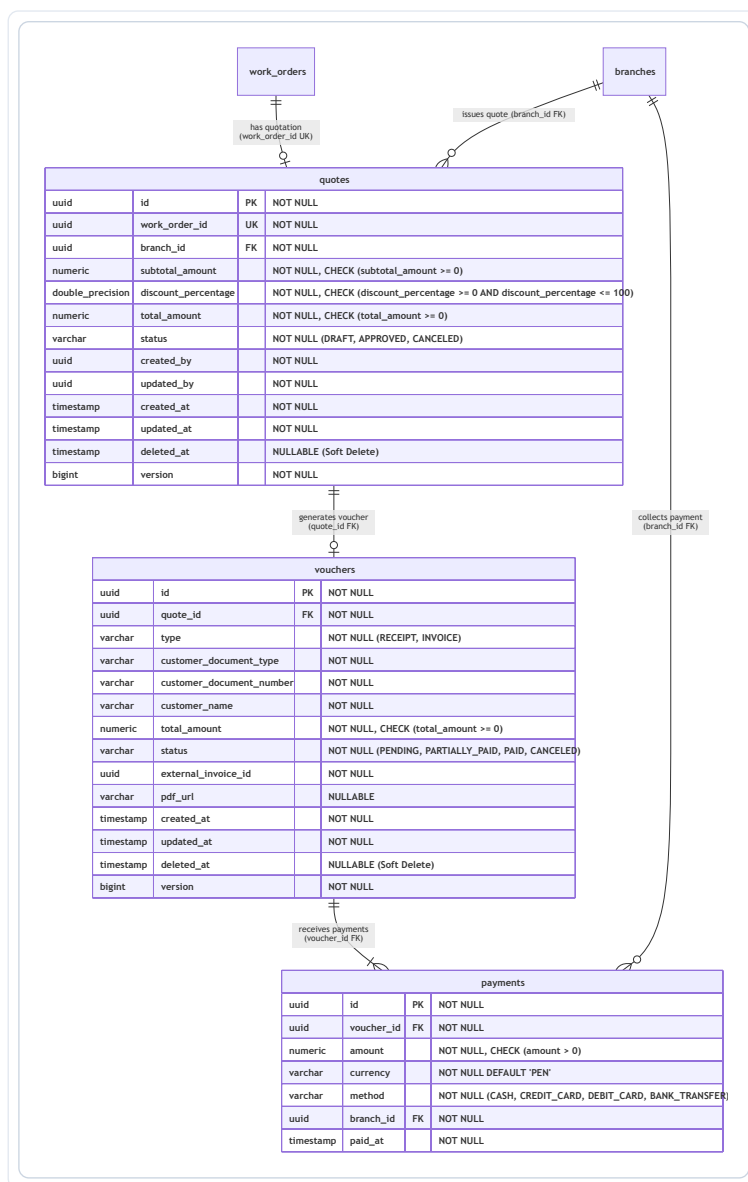


Figura 83: Diagrama de Base de Datos Relacional ER – Billing

...